

TripJar

Save Together. Travel Together.

Full Codebase — July 30, 2026

Backend:	Node.js · Express · TypeScript · Drizzle ORM · PostgreSQL
Mobile:	Expo · React Native · React Query · Expo Router
Auth:	JWT Bearer tokens · 30-day expiry
Payments:	Stripe-ready (simulated for MVP)

Table of Contents

- **Database Schema**

- lib/db/src/index.ts

- lib/db/src/schema/index.ts

- **API Server — App & Config**

- artifacts/api-server/src/app.ts

- artifacts/api-server/src/index.ts

- **API Server — Auth & Utilities**

- artifacts/api-server/src/lib/auth.ts

- artifacts/api-server/src/lib/jar-health.ts

- artifacts/api-server/src/lib/activity.ts

- artifacts/api-server/src/lib/notifications.ts

- artifacts/api-server/src/lib/email.ts

- artifacts/api-server/src/lib/schedule-utils.ts

- artifacts/api-server/src/lib/logger.ts

- **API Server — Routes**

- artifacts/api-server/src/routes/index.ts

- artifacts/api-server/src/routes/auth.ts

- artifacts/api-server/src/routes/profile.ts

- artifacts/api-server/src/routes/dashboard.ts

- artifacts/api-server/src/routes/jars.ts

- artifacts/api-server/src/routes/members.ts

- artifacts/api-server/src/routes/invitations.ts

- artifacts/api-server/src/routes/contributions.ts

- artifacts/api-server/src/routes/milestones.ts

- artifacts/api-server/src/routes/schedules.ts

- artifacts/api-server/src/routes/reminders.ts

- artifacts/api-server/src/routes/commitments.ts

- artifacts/api-server/src/routes/agreements.ts

- artifacts/api-server/src/routes/notifications.ts

- artifacts/api-server/src/routes/activity.ts

- artifacts/api-server/src/routes/health.ts

- **Mobile App — Contexts**

- artifacts/mobile/contexts/auth-context.tsx

- artifacts/mobile/contexts/create-jar-context.tsx

- **Mobile App — Auth Screens**

- artifacts/mobile/app/(auth)/index.tsx

- artifacts/mobile/app/(auth)/login.tsx

- artifacts/mobile/app/(auth)/register.tsx

- artifacts/mobile/app/(auth)/forgot-password.tsx

- artifacts/mobile/app/(auth)/profile-setup.tsx

- **Mobile App — Tab Screens**

- artifacts/mobile/app/(tabs)/index.tsx

- artifacts/mobile/app/(tabs)/jars.tsx

- artifacts/mobile/app/(tabs)/activity.tsx

- artifacts/mobile/app/(tabs)/notifications.tsx

- artifacts/mobile/app/(tabs)/profile.tsx

- **Mobile App — Detail Screens**

- artifacts/mobile/app/jar/[id].tsx

- artifacts/mobile/app/contribution/[jarId].tsx

- artifacts/mobile/app/invite/[token].tsx

- **Mobile App — Create Jar Wizard**

- artifacts/mobile/app/create-jar/index.tsx

- artifacts/mobile/app/create-jar/dates.tsx

- artifacts/mobile/app/create-jar/goal.tsx

artifacts/mobile/app/create-jar/milestones.tsx
artifacts/mobile/app/create-jar/members.tsx
artifacts/mobile/app/create-jar/plan.tsx
artifacts/mobile/app/create-jar/rules.tsx
artifacts/mobile/app/create-jar/review.tsx

• **Mobile App — Components**

artifacts/mobile/components/CalendarPicker.tsx
artifacts/mobile/components/DateInput.tsx
artifacts/mobile/components/CurrencyInput.tsx
artifacts/mobile/components/CircularProgress.tsx
artifacts/mobile/components/ProgressBar.tsx
artifacts/mobile/components/JarCard.tsx
artifacts/mobile/components/JarHealthBadge.tsx
artifacts/mobile/components/MemberAvatar.tsx
artifacts/mobile/components/ScheduleSetupSheet.tsx
artifacts/mobile/components/EmptyState.tsx
artifacts/mobile/components/SkeletonLoader.tsx
artifacts/mobile/components/ErrorBoundary.tsx
artifacts/mobile/components/ErrorFallback.tsx

• **Mobile App — Constants & Hooks**

artifacts/mobile/constants/colors.ts
artifacts/mobile/hooks/useColors.ts

Database Schema

lib/db/src/index.ts

```

1 import { drizzle } from "drizzle-orm/node-postgres";
2 import pg from "pg";
3 import * as schema from "./schema";
4
5 const { Pool } = pg;
6
7 if (!process.env.DATABASE_URL) {
8   throw new Error(
9     "DATABASE_URL must be set. Did you forget to provision a database?",
10  );
11 }
12
13 export const pool = new Pool({ connectionString: process.env.DATABASE_URL });
14 export const db = drizzle(pool, { schema });
15
16 export * from "./schema";
17

```

lib/db/src/schema/index.ts

[illegible]

lib/db/src/schema/index.ts (continued)

```
33     profile: one(profiles, { fields: [users.id], references: [profiles.userId] }),
34     jarMembers: many(jarMembers),
35     notifications: many(notifications),
36     activityEvents: many(activityEvents),
37     agreementAcceptances: many(agreementAcceptances),
38   }));
```

[illegible]

```

42 export const profiles = pgTable("profiles", {
43   id: uuid("id").primaryKey().defaultRandom(),
44   userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
45   firstName: text("first_name").notNull(),
46   lastName: text("last_name").notNull(),
47   displayName: text("display_name").notNull(),
48   avatarUrl: text("avatar_url"),
49   phone: text("phone"),
50   timeZone: text("time_zone").notNull().default("America/New_York"),
51   defaultCurrency: text("default_currency").notNull().default("USD"),
52   createdAt: timestamp("created_at").notNull().defaultNow(),
53   updatedAt: timestamp("updated_at").notNull().defaultNow(),
54 }, (t) => [
55   uniqueIndex("profiles_user_id_idx").on(t.userId),
56 ]);

```

```
58 export const profilesRelations = relations(profiles, ({ one }) => ({
59   user: one(users, { fields: [profiles.userId], references: [users.id] }),
60 }));
```

[illegible]

```

64 export const jars = pgTable("jars", {
65   id: uuid("id").primaryKey().defaultRandom(),
66   organizerId: uuid("organizer_id").notNull().references(() => users.id),
67   name: text("name").notNull(),
68   slug: text("slug").notNull(),
69   category: text("category").notNull().default("Vacation"),
70   description: text("description"),
71   destination: text("destination"),
72   coverImageUrl: text("cover_image_url"),
73   startDate: date("start_date"),
74   endDate: date("end_date"),
75   targetDate: date("target_date").notNull(),
76   goalAmountCents: integer("goal_amount_cents").notNull(),
77   currency: text("currency").notNull().default("USD"),
78   status: text("status").notNull().default("Draft"),
79   approvalThreshold: numeric("approval_threshold", { precision: 4, scale:
80     3 }).notNull().default(0.167),
81   createdAt: timestamp("created_at").notNull().defaultNow(),
82   updatedAt: timestamp("updated_at").notNull().defaultNow(),
83 }, (t) => [
84   uniqueIndex("jars_slug_idx").on(t.slug),
85   index("jars_organizer_idx").on(t.organizerId),
86 ]);

```

```
88 export const jarsRelations = relations(jars, ({ one, many }) => ({
89   organizer: one(users, { fields: [jars.organizerId], references: [users.id] }),
90   members: many(jarMembers),
91   invitations: many(invitations),
```

lib/db/src/schema/index.ts (continued)

lib/db/src/schema/index.ts (continued)

lib/db/src/schema/index.ts (continued)

[illegible]

lib/db/src/schema/index.ts (continued)

```

269     id: uuid("id").primaryKey().defaultRandom(),
270     commitmentRequestId: uuid("commitment_request_id").notNull().references(() => commitmentRequests.id,
271     memberId: uuid("member_id").notNull().references(() => jarMembers.id),
272     vote: text("vote").notNull(),
273     note: text("note"),
274     createdAt: timestamp("created_at").notNull().defaultNow(),
275 }, (t) => [
276     uniqueIndex("commitment_votes_request_member_idx").on(t.commitmentRequestId, t.memberId),
277 ]);

```

```
279 export const commitmentVotesRelations = relations(commitmentVotes, ({ one }) => ({
280   commitmentRequest: one(commitmentRequests, { fields: [commitmentVotes.commitmentRequestId],
281   member: one(jarMembers, { fields: [commitmentVotes.memberId], references: [jarMembers.id] })),
282 }));
```

[illegible]

```
286 export const agreements = pgTable("agreements", {
287   id: uuid("id").primaryKey().defaultRandom(),
288   jarId: uuid("jar_id").notNull().references(() => jars.id, { onDelete: "cascade" }),
289   version: text("version").notNull(),
290   content: text("content").notNull(),
291   effectiveDate: date("effective_date").notNull(),
292   createdAt: timestamp("created_at").notNull().defaultNow(),
293 }, (t) => [
294   index("agreements_jar_idx").on(t.jarId),
295 ]);
```

```
297 export const agreementsRelations = relations(agreements, ({ one, many }) => ({
298   jar: one(jars, { fields: [agreements.jarId], references: [jars.id] }),
299   acceptances: many(agreementAcceptances),
300 }));
```

```
302 // % % % Agreement Acceptances % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

```

304 export const agreementAcceptances = pgTable("agreement_acceptances", {
305   id: uuid("id").primaryKey().defaultRandom(),
306   agreementId: uuid("agreement_id").notNull().references(() => agreements.id, { onDelete: "cascade" }),
307   userId: uuid("user_id").notNull().references(() => users.id),
308   acceptedAt: timestamp("accepted_at").notNull().defaultNow(),
309 }, (t) => [
310   uniqueIndex("agreement_acceptances_agreement_user_idx").on(t.agreementId, t.userId),
311 ]);

```

```
313 export const agreementAcceptancesRelations = relations(agreementAcceptances, ({ one }) => ({
314   agreement: one(agreements, { fields: [agreementAcceptances.agreementId], references:
315     user: one(users, { fields: [agreementAcceptances.userId], references: [users.id] })),
316 }));
```

[illegible]

```
320 export const notifications = pgTable("notifications", {
321   id: uuid("id").primaryKey().defaultRandom(),
322   userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
323   type: text("type").notNull().default("general"),
324   title: text("title").notNull(),
325   message: text("message").notNull(),
326   isRead: boolean("is_read").notNull().default(false),
327   relatedJarId: uuid("related_jar_id").references(() => jars.id),
```



```
387   jarId: uuid("jar_id").notNull().references(() => jars.id),  
388   memberId: uuid("member_id").notNull().references(() => jarMembers.id),  
389   amountCents: integer("amount_cents").notNull(),  
390   reason: text("reason"),  
391   status: text("status").notNull().default("pending"),  
392   createdAt: timestamp("created_at").notNull().defaultNow(),  
393   updatedAt: timestamp("updated_at").notNull().defaultNow(),  
394 });  
395  
396 export const refundRequestPlaceholdersRelations = relations(refundRequestPlaceholders, ({ one }) => ({  
397   jar: one(jars, { fields: [refundRequestPlaceholders.jarId], references: [jars.id] }),  
398   member: one(jarMembers, { fields: [refundRequestPlaceholders.memberId], references: [jarMembers.id] })),  
399 ));  
400  
401 // % % % Type exports % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  
402  
403 export type User = typeof users.$inferSelect;  
404 export type NewUser = typeof users.$inferInsert;  
405 export type Profile = typeof profiles.$inferSelect;  
406 export type NewProfile = typeof profiles.$inferInsert;  
407 export type Jar = typeof jars.$inferSelect;  
408 export type NewJar = typeof jars.$inferInsert;  
409 export type JarMember = typeof jarMembers.$inferSelect;  
410 export type NewJarMember = typeof jarMembers.$inferInsert;  
411 export type Invitation = typeof invitations.$inferSelect;  
412 export type NewInvitation = typeof invitations.$inferInsert;  
413 export type Contribution = typeof contributions.$inferSelect;  
414 export type NewContribution = typeof contributions.$inferInsert;  
415 export type ContributionSchedule = typeof contributionSchedules.$inferSelect;  
416 export type NewContributionSchedule = typeof contributionSchedules.$inferInsert;  
417 export type Milestone = typeof milestones.$inferSelect;  
418 export type NewMilestone = typeof milestones.$inferInsert;  
419 export type MilestoneAllocation = typeof milestoneAllocations.$inferSelect;  
420 export type NewMilestoneAllocation = typeof milestoneAllocations.$inferInsert;  
421 export type CommitmentRequest = typeof commitmentRequests.$inferSelect;  
422 export type NewCommitmentRequest = typeof commitmentRequests.$inferInsert;  
423 export type CommitmentVote = typeof commitmentVotes.$inferSelect;  
424 export type NewCommitmentVote = typeof commitmentVotes.$inferInsert;  
425 export type Agreement = typeof agreements.$inferSelect;  
426 export type NewAgreement = typeof agreements.$inferInsert;  
427 export type AgreementAcceptance = typeof agreementAcceptances.$inferSelect;  
428 export type NewAgreementAcceptance = typeof agreementAcceptances.$inferInsert;  
429 export type Notification = typeof notifications.$inferSelect;  
430 export type NewNotification = typeof notifications.$inferInsert;  
431 export type ActivityEvent = typeof activityEvents.$inferSelect;  
432 export type NewActivityEvent = typeof activityEvents.$inferInsert;  
433 export type PaymentMethodPlaceholder = typeof paymentMethodPlaceholders.$inferSelect;  
434 export type RefundRequestPlaceholder = typeof refundRequestPlaceholders.$inferSelect;
```

API Server — App & Config

artifacts/api-server/src/app.ts

```
1  import express, { type Express } from "express";
2  import cors from "cors";
3  import pinoHttp from "pino-http";
4  import router from "../routes/index.js";
5  import { logger } from "../lib/logger.js";
6
7  const app: Express = express();
8
9  // Trust proxy for deployed environments
10 app.set("trust proxy", 1);
11
12 app.use(
13   pinoHttp({
14     logger,
15     serializers: {
16       req(req) {
17         return {
18           id: req.id,
19           method: req.method,
20           url: req.url?.split("?")[0],
21         };
22       },
23       res(res) {
24         return {
25           statusCode: res.statusCode,
26         };
27       },
28     },
29   }),
30 );
31
32 // CORS: allow all origins in dev, restrict in production
33 const allowedOrigins = process.env["ALLOWED_ORIGINS"]
34   ? process.env["ALLOWED_ORIGINS"].split(",")
35   : null;
36
37 app.use(
38   cors({
39     origin: allowedOrigins
40       ? (origin, callback) => {
41         if (!origin || allowedOrigins.some((o) => origin.includes(o))) {
42           callback(null, true);
43         } else {
44           callback(new Error("Not allowed by CORS"));
45         }
46       }
47     : true,
48     credentials: true,
49     methods: ["GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"],
50     allowedHeaders: ["Content-Type", "Authorization"],
51   }),
52 );
```

artifacts/api-server/src/app.ts (continued)

```
53
54 app.use(express.json({ limit: "1mb" }));
55 app.use(express.urlencoded({ extended: true, limit: "1mb" }));
56
57 app.use("/api", router);
58
59 // Global error handler
60 app.use(
61   (
62     err: Error,
63     _req: express.Request,
64     res: express.Response,
65     _next: express.NextFunction,
66   ) => {
67     logger.error({ err }, "Unhandled error");
68     res.status(500).json({
69       error: "InternalServerError",
70       message:
71         process.env["NODE_ENV"] === "production"
72           ? "An internal error occurred"
73           : err.message,
74     });
75   },
76 );
77
78 export default app;
79
```

artifacts/api-server/src/index.ts

```
1 import app from "./app";
2 import { logger } from "./lib/logger";
3
4 const rawPort = process.env["PORT"];
5
6 if (!rawPort) {
7   throw new Error(
8     "PORT environment variable is required but was not provided.",
9   );
10 }
11
12 const port = Number(rawPort);
13
14 if (Number.isNaN(port) || port <= 0) {
15   throw new Error(`Invalid PORT value: "${rawPort}"`);
16 }
17
18 app.listen(port, (err) => {
19   if (err) {
20     logger.error({ err }, "Error listening on port");
21     process.exit(1);
22   }
23
24   logger.info({ port }, "Server listening");
25 });
26
```

API Server — Auth & Utilities

artifacts/api-server/src/lib/auth.ts

```
1  import jwt from "jsonwebtoken";
2  import { type Request, type Response, type NextFunction } from "express";
3
4  const JWT_SECRET =
5    process.env["JWT_SECRET"] ?? "tripjar-dev-secret-change-in-production";
6  const TOKEN_EXPIRY = "30d";
7
8  export interface AuthenticatedRequest extends Request {
9    userId: string;
10   userEmail: string;
11 }
12
13 export function requireAuth(
14   req: Request,
15   res: Response,
16   next: NextFunction,
17 ): void {
18   const authHeader = req.headers.authorization;
19   if (!authHeader?.startsWith("Bearer ")) {
20     res
21       .status(401)
22       .json({ error: "Unauthorized", message: "Missing or invalid auth token" });
23     return;
24   }
25
26   const token = authHeader.slice(7);
27   try {
28     const payload = jwt.verify(token, JWT_SECRET) as {
29       userId: string;
30       email: string;
31     };
32     (req as AuthenticatedRequest).userId = payload.userId;
33     (req as AuthenticatedRequest).userEmail = payload.email;
34     next();
35   } catch {
36     res
37       .status(401)
38       .json({ error: "Unauthorized", message: "Invalid or expired token" });
39   }
40 }
41
42 export function signToken(userId: string, email: string): string {
43   return jwt.sign({ userId, email }, JWT_SECRET, { expiresIn: TOKEN_EXPIRY });
44 }
45
```

artifacts/api-server/src/lib/jar-health.ts

```
1  export type HealthStatus = "Ahead" | "OnTrack" | "NeedsAttention" | "AtRisk";
2
3  export interface JarHealthResult {
4    status: HealthStatus;
```

```

5    message: string;
6    actualPercentage: number;
7    expectedPercentage: number;
8    daysElapsed: number;
9    totalDays: number;
10   amountBehindCents: number;
11   estimatedCompletionDate: string | null;
12 }
13
14 // Configurable thresholds (percentage points ahead/behind)
15 const THRESHOLDS = {
16   aheadMin: 10,
17   onTrackMin: -10,
18   needsAttentionMin: -20,
19 } as const;
20
21 export function calculateJarHealth(
22   goalAmountCents: number,
23   totalSavedCents: number,
24   launchedAt: Date,
25   targetDate: Date,
26 ): JarHealthResult {
27   const now = new Date();
28   const totalMs = targetDate.getTime() - launchedAt.getTime();
29   const elapsedMs = Math.max(0, now.getTime() - launchedAt.getTime());
30
31   const totalDays = Math.max(1, Math.ceil(totalMs / 86_400_000));
32   const daysElapsed = Math.ceil(elapsedMs / 86_400_000);
33
34   const expectedPercentage = Math.min(100, (daysElapsed / totalDays) * 100);
35   const actualPercentage =
36     goalAmountCents > 0
37       ? Math.min(100, (totalSavedCents / goalAmountCents) * 100)
38       : 0;
39
40   const diff = actualPercentage - expectedPercentage;
41
42   let status: HealthStatus;
43   if (diff >= THRESHOLDS.aheadMin) status = "Ahead";
44   else if (diff >= THRESHOLDS.onTrackMin) status = "OnTrack";
45   else if (diff >= THRESHOLDS.needsAttentionMin) status = "NeedsAttention";
46   else status = "AtRisk";
47
48   const expectedAmountCents = Math.round(
49     (expectedPercentage / 100) * goalAmountCents,
50   );
51   const amountBehindCents = Math.max(0, expectedAmountCents - totalSavedCents);
52   const amountAheadCents = Math.max(0, totalSavedCents - expectedAmountCents);
53
54   let message = "";
55   if (status === "Ahead") {
56     const daysAhead = Math.round((amountAheadCents / goalAmountCents) * totalDays);
57     message =
58       daysAhead > 0
59         ? `Your group is on track to reach the goal ${daysAhead} days early.`
60         : "Your group is ahead of schedule!";
61   } else if (status === "OnTrack") {
62     message = "Your group is on track to reach the goal.";
63   } else if (status === "NeedsAttention") {

```


artifacts/api-server/src/lib/jar-health.ts (continued)

```
64     const behindDollars = Math.round(amountBehindCents / 100);
65     message = `Your group is approximately ${behindDollars} behind the current savings target.`;
66   } else {
67     const behindDollars = Math.round(amountBehindCents / 100);
68     message = `Your group is ${behindDollars} behind schedule. Consider increasing contributions soon.`;
69   }
70
71   // Estimate completion date based on current daily saving rate
72   let estimatedCompletionDate: string | null = null;
73   if (totalSavedCents > 0 && daysElapsed > 0 && totalSavedCents < goalAmountCents) {
74     const dailyRate = totalSavedCents / daysElapsed;
75     const remainingCents = goalAmountCents - totalSavedCents;
76     const daysToComplete = Math.ceil(remainingCents / dailyRate);
77     const completionDate = new Date(now);
78     completionDate.setDate(completionDate.getDate() + daysToComplete);
79     estimatedCompletionDate = completionDate.toISOString().split("T")[0]!;
80   }
81
82   return {
83     status,
84     message,
85     actualPercentage,
86     expectedPercentage,
87     daysElapsed,
88     totalDays,
89     amountBehindCents,
90     estimatedCompletionDate,
91   };
92 }
93
94 export function calculateMemberHealth(
95   contributionTargetCents: number,
96   contributedAmountCents: number,
97   daysElapsed: number,
98   totalDays: number,
99 ): "Ahead" | "OnTrack" | "NeedsAttention" | "AtRisk" | "NotStarted" {
100   if (contributedAmountCents === 0) return "NotStarted";
101
102   const expectedPercentage = Math.min(100, (daysElapsed / Math.max(1, totalDays)) * 100);
103   const actualPercentage =
104     contributionTargetCents > 0
105       ? Math.min(100, (contributedAmountCents / contributionTargetCents) * 100)
106       : 0;
107
108   const diff = actualPercentage - expectedPercentage;
109
110   if (diff >= THRESHOLDS.aheadMin) return "Ahead";
111   if (diff >= THRESHOLDS.onTrackMin) return "OnTrack";
112   if (diff >= THRESHOLDS.needsAttentionMin) return "NeedsAttention";
113   return "AtRisk";
114 }
115
```

artifacts/api-server/src/lib/activity.ts

```
1  import { db } from "@workspace/db";
2  import { activityEvents } from "@workspace/db";
3
4  export type ActivityEventType =
```

artifacts/api-server/src/lib/activity.ts (continued)

```
5 | "jar_created"
6 | "invitation_sent"
7 | "member_joined"
8 | "contribution_added"
9 | "contribution_reversed"
10 | "target_changed"
11 | "milestone_created"
12 | "milestone_funded"
13 | "commitment_requested"
14 | "approval_submitted"
15 | "jar_locked"
16 | "jar_completed"
17 | "jar_cancelled"
18 | "member_left"
19 | "agreement_accepted";
20
21 export async function logActivity(params: {
22   jarId: string;
23   userId?: string;
24   eventType: ActivityEventType;
25   description: string;
26   amountCents?: number;
27   metadata?: Record<string, unknown>;
28 }): Promise<void> {
29   try {
30     await db.insert(activityEvents).values({
31       jarId: params.jarId,
32       userId: params.userId ?? null,
33       eventType: params.eventType,
34       description: params.description,
35       amountCents: params.amountCents ?? null,
36       metadata: params.metadata ?? null,
37     });
38   } catch (err) {
39     // Activity logging is fire-and-forget; never block main request
40     console.error("Failed to log activity:", err);
41   }
42 }
43
```

artifacts/api-server/src/lib/notifications.ts

```
1 import { db } from "@workspace/db";
2 import { notifications } from "@workspace/db";
3
4 export type NotificationType =
5 | "invitation_received"
6 | "member_joined"
7 | "contribution_recorded"
8 | "contribution_due"
9 | "contribution_overdue"
10 | "jar_halfway_funded"
11 | "milestone_funded"
12 | "commitment_requested"
13 | "member_approved_commitment"
14 | "member_rejected_commitment"
15 | "lock_date_approaching"
16 | "goal_fully_funded"
17 | "general";
```

artifacts/api-server/src/lib/notifications.ts (continued)

```
18
19 export async function createNotification(params: {
20   userId: string;
21   type: NotificationType;
22   title: string;
23   message: string;
24   relatedJarId?: string;
25   actionUrl?: string;
26 }): Promise<void> {
27   try {
28     await db.insert(notifications).values({
29       userId: params.userId,
30       type: params.type,
31       title: params.title,
32       message: params.message,
33       relatedJarId: params.relatedJarId ?? null,
34       actionUrl: params.actionUrl ?? null,
35       isRead: false,
36     });
37   } catch (err) {
38     // Notification creation is fire-and-forget
39     console.error("Failed to create notification:", err);
40   }
41 }
42
43 export async function notifyAllMembers(params: {
44   jarId: string;
45   memberUserIds: string[];
46   type: NotificationType;
47   title: string;
48   message: string;
49   excludeUserId?: string;
50 }): Promise<void> {
51   const targets = params.excludeUserId
52     ? params.memberUserIds.filter((id) => id !== params.excludeUserId)
53     : params.memberUserIds;
54
55   await Promise.all(
56     targets.map((userId) =>
57       createNotification({
58         userId,
59         type: params.type,
60         title: params.title,
61         message: params.message,
62         relatedJarId: params.jarId,
63       }),
64     ),
65   );
66 }
67
```

artifacts/api-server/src/lib/email.ts

```
1 import { Resend } from "resend";
2 import { logger } from "../logger.js";
3
4 let resend: Resend | null = null;
5
6 function getResendClient(): Resend | null {
```

```

7   const apiKey = process.env["RESEND_API_KEY"];
8   if (!apiKey) {
9     logger.warn("RESEND_API_KEY is not set – email delivery is disabled");
10    return null;
11  }
12  if (!resend) {
13    resend = new Resend(apiKey);
14  }
15  return resend;
16 }
17
18 function getAppBaseUrl(): string {
19   // In Replit dev the mobile app is served at the dev domain root
20   const domain = process.env["REPLIT_DEV_DOMAIN"];
21   if (domain) return `https://${domain}`;
22   // Allow explicit override for production
23   return process.env["APP_BASE_URL"] ?? "https://tripjar.app";
24 }
25
26 export async function sendInvitationEmail(opts: {
27   toEmail: string;
28   jarName: string;
29   inviterName: string;
30   token: string;
31 }): Promise<void> {
32   const client = getResendClient();
33   if (!client) {
34     logger.info(
35       { toEmail: opts.toEmail, token: opts.token },
36       "Email delivery skipped (no RESEND_API_KEY) – invitation token logged for manual testing"
37     );
38     return;
39   }
40
41   const inviteUrl = `${getAppBaseUrl()}/invite/${opts.token}`;
42   const fromAddress =
43     process.env["EMAIL_FROM"] ?? "TripJar <noreply@updates.tripjar.app>";
44
45   try {
46     const { error } = await client.emails.send({
47       from: fromAddress,
48       to: opts.toEmail,
49       subject: `${opts.inviterName} invited you to join "${opts.jarName}" on TripJar`,
50       html: `
51 <!DOCTYPE html>
52 <html>
53 <head>
54   <meta charset="utf-8" />
55   <meta name="viewport" content="width=device-width, initial-scale=1" />
56 </head>
57 <body style="margin:0;padding:0;background:#f4f7f6;font-family:sans-serif;">
58   <table width="100%" cellpadding="0" cellspacing="0" style="background:#f4f7f6;padding:40px 0;">
59     <tr>
60       <td align="center">
61         <table width="520" cellpadding="0" cellspacing="0" style="background:#ffffff;border-
62 radius:16px;overflow:hidden">
63           <tr>
64             <td style="background:#178B57;padding:32px;text-align:center;">
65               <p style="margin:0;color:#ffffff;font-size:28px;font-weight:bold;">>ð TripJar</p>

```

artifacts/api-server/src/lib/email.ts (continued)

```

66         <p style="margin:8px 0 0;color:#E8F6EF;font-size:15px;">Group Travel Savings</p>
67     </td>
68 </tr>
69 <!-- Body -->
70 <tr>
71     <td style="padding:40px 32px;">
72         <h1 style="margin:0 0 16px;font-size:24px;color:#111827;font-weight:bold;">You're invited!
73         <p style="margin:0 0 24px;font-size:16px;color:#6B7280;line-height:1.6;">
74             <strong style="color:#111827;">${opts.inviterName}</strong> has invited you to join
75             <strong style="color:#111827;">${opts.jarName}</strong> on TripJar – the easiest way
76 to save for ... </p>
77         <div style="text-align:center;margin:32px 0;">
78             <a href="${inviteUrl}"
79                 style="display:inline-block;background:#178B57;color:#ffffff;font-size:17px;font-
80 weight:bold;
81                 text-decoration:none;padding:16px 40px;border-radius:12px;">
82                 Accept Invitation
83             </a>
84         </div>
85         <p style="margin:0;font-size:14px;color:#9CA3AF;text-align:center;">
86             This link expires in 7 days. If you didn't expect this, you can safely ignore it.
87         </p>
88     </td>
89 <!-- Footer -->
90 <tr>
91     <td style="padding:24px 32px;border-top:1px solid #E5E7EB;text-align:center;">
92         <p style="margin:0;font-size:12px;color:#9CA3AF;">
93             Or copy this link: <a href="${inviteUrl}" style="color:#178B57;">${inviteUrl}</a>
94         </p>
95     </td>
96 </tr>
97 </table>
98 </td>
99 </tr>
100 </table>
101 </body>
102 </html>`,
103     });
104
105     if (error) {
106         logger.error({ error, toEmail: opts.toEmail }, "Resend API error sending invitation email");
107     } else {
108         logger.info({ toEmail: opts.toEmail }, "Invitation email sent successfully");
109     }
110 } catch (err) {
111     logger.error({ err, toEmail: opts.toEmail }, "Failed to send invitation email");
112 }
113 }
114

```

artifacts/api-server/src/lib/schedule-utils.ts

```

1  /**
2   * Utilities for computing contribution schedule due dates.
3   */
4
5  export interface ScheduleLike {
6      startDate: string; // ISO date string yyyy-MM-dd
7      frequency: string;

```

```

8     preferredDay?: number | null;
9 }
10
11 /**
12  * Given a schedule, find the next due date that is:
13  * - >= afterDate (defaults to today)
14  * - >= schedule.startDate
15  *
16  * Returns null for manual/custom frequencies.
17  */
18 export function computeNextDueDate(schedule: ScheduleLike, afterDate: Date = new Date()): Date | null {
19     const start = parseLocalDate(schedule.startDate);
20     const { frequency, preferredDay } = schedule;
21
22     // Never return a date before the schedule has started
23     const effectiveAfter = laterDate(afterDate, start);
24
25     if (frequency === "weekly") {
26         return advanceByDays(start, effectiveAfter, 7);
27     }
28
29     if (frequency === "biweekly") {
30         return advanceByDays(start, effectiveAfter, 14);
31     }
32
33     if (frequency === "monthly") {
34         return advanceMonthly(start, effectiveAfter, preferredDay ?? null);
35     }
36
37     if (frequency === "twiceMonthly") {
38         return advanceTwiceMonthly(start, effectiveAfter, preferredDay ?? null);
39     }
40
41     return null;
42 }
43
44 /**
45  * Find the most recent due date that is < afterDate (i.e. a due date that has already passed)
46  * and is >= schedule.startDate.
47  * Returns null if no due date has passed yet.
48  */
49 export function computePreviousDueDate(schedule: ScheduleLike, afterDate: Date = new Date()): Date |
50 null {
51     const start = parseLocalDate(schedule.startDate);
52     const next = computeNextDueDate(schedule, afterDate);
53     if (!next) return null;
54
55     const { frequency, preferredDay } = schedule;
56
57     let prev: Date;
58     if (frequency === "weekly") {
59         prev = new Date(next);
60         prev.setDate(prev.getDate() - 7);
61     } else if (frequency === "biweekly") {
62         prev = new Date(next);
63         prev.setDate(prev.getDate() - 14);
64     } else if (frequency === "monthly") {
65         // Step back one month, keeping the same clamped day
66         prev = safeMonthDate(next.getFullYear(), next.getMonth() - 1, next.getDate());
67     } else if (frequency === "twiceMonthly") {

```

[illegible]

artifacts/api-server/src/lib/schedule-utils.ts (continued)

```
126
127 /** Advance monthly (same day-of-month as start, or preferredDay) until >= afterDate */
128 function advanceMonthly(start: Date, afterDate: Date, preferredDay: number | null): Date {
129     const dom = preferredDay ?? start.getDate();
130     const after = stripTime(afterDate);
131
132     // Try same month first (clamped to valid day)
133     const candidate = safeMonthDate(after.getFullYear(), after.getMonth(), dom);
134     if (candidate >= after) return candidate;
135
136     // Move to next month
137     return safeMonthDate(after.getFullYear(), after.getMonth() + 1, dom);
138 }
139
140 /** Advance twice-monthly: days at preferredDay and preferredDay+15 (capped at 28) */
141 function advanceTwiceMonthly(start: Date, afterDate: Date, preferredDay: number | null): Date {
142     const firstDay = preferredDay ?? start.getDate();
143     const secondDay = Math.min(firstDay + 15, 28);
144     const after = stripTime(afterDate);
145
146     const c1 = safeMonthDate(after.getFullYear(), after.getMonth(), firstDay);
147     const c2 = safeMonthDate(after.getFullYear(), after.getMonth(), secondDay);
148
149     if (c1 >= after) return c1;
150     if (c2 >= after) return c2;
151
152     // Neither this month – use next month's first day
153     return safeMonthDate(after.getFullYear(), after.getMonth() + 1, firstDay);
154 }
155
156 export function stripTime(d: Date): Date {
157     return new Date(d.getFullYear(), d.getMonth(), d.getDate());
158 }
159
160 /** Format a Date as yyyy-MM-dd */
161 export function toISODate(d: Date): string {
162     const y = d.getFullYear();
163     const m = String(d.getMonth() + 1).padStart(2, "0");
164     const day = String(d.getDate()).padStart(2, "0");
165     return `${y}-${m}-${day}`;
166 }
167
```

artifacts/api-server/src/lib/logger.ts

```
1 import pino from "pino";
2
3 const isProduction = process.env.NODE_ENV === "production";
4
5 export const logger = pino({
6     level: process.env.LOG_LEVEL ?? "info",
7     redact: [
8         "req.headers.authorization",
9         "req.headers.cookie",
10        "res.headers['set-cookie']",
11    ],
12    ...(isProduction
13        ? {}
14        : {
```


artifacts/api-server/src/lib/logger.ts (continued)

```
15     transport: {
16       target: "pino-pretty",
17       options: { colorize: true },
18     },
19   }},
20 });
21
```

API Server — Routes

artifacts/api-server/src/routes/index.ts

```
1  import { Router, type IRouter } from "express";
2  import healthRouter from "../health.js";
3  import authRouter from "../auth.js";
4  import profileRouter from "../profile.js";
5  import dashboardRouter from "../dashboard.js";
6  import jarsRouter from "../jars.js";
7  import membersRouter from "../members.js";
8  import invitationsRouter from "../invitations.js";
9  import contributionsRouter from "../contributions.js";
10 import schedulesRouter from "../schedules.js";
11 import milestonesRouter from "../milestones.js";
12 import commitmentsRouter from "../commitments.js";
13 import agreementsRouter from "../agreements.js";
14 import notificationsRouter from "../notifications.js";
15 import activityRouter from "../activity.js";
16 import remindersRouter from "../reminders.js";
17
18 const router: IRouter = Router();
19
20 router.use(healthRouter);
21 router.use(authRouter);
22 router.use(profileRouter);
23 router.use(dashboardRouter);
24 router.use(jarsRouter);
25 router.use(membersRouter);
26 router.use(invitationsRouter);
27 router.use(contributionsRouter);
28 router.use(schedulesRouter);
29 router.use(milestonesRouter);
30 router.use(commitmentsRouter);
31 router.use(agreementsRouter);
32 router.use(notificationsRouter);
33 router.use(activityRouter);
34 router.use(remindersRouter);
35
36 export default router;
37
```

artifacts/api-server/src/routes/auth.ts

```
1  import { Router } from "express";
2  import bcrypt from "bcryptjs";
3  import { db } from "@workspace/db";
4  import { users, profiles } from "@workspace/db";
5  import { eq } from "drizzle-orm";
6  import { requireAuth, signToken, type AuthenticatedRequest } from "../lib/auth.js";
7  import crypto from "node:crypto";
8
9  const router = Router();
10
11 // POST /auth/register
12 router.post("/auth/register", async (req, res) => {
```

```
13   const { email, password, firstName, lastName } = req.body as {
14     email?: string;
15     password?: string;
16     firstName?: string;
17     lastName?: string;
18   };
19
20   if (!email || !password || !firstName || !lastName) {
21     res.status(400).json({ error: "BadRequest", message: "All fields are required" });
22     return;
23   }
24
25   if (password.length < 8) {
26     res.status(400).json({ error: "BadRequest", message: "Password must be at least 8 characters" });
27     return;
28   }
29
30   const existing = await db.select().from(users).where(eq(users.email, email.toLowerCase())).limit(1);
31   if (existing.length > 0) {
32     res.status(409).json({ error: "Conflict", message: "An account with this email already exists" });
33     return;
34   }
35
36   const passwordHash = await bcrypt.hash(password, 12);
37   const displayName = `${firstName} ${lastName}`.trim();
38
39   const [newUser] = await db.insert(users).values({
40     email: email.toLowerCase(),
41     passwordHash,
42     emailVerified: false,
43     lastLoginAt: new Date(),
44   }).returning();
45
46   if (!newUser) {
47     res.status(500).json({ error: "InternalServerError", message: "Failed to create user" });
48     return;
49   }
50
51   const [newProfile] = await db.insert(profiles).values({
52     userId: newUser.id,
53     firstName,
54     lastName,
55     displayName,
56     timeZone: "America/New_York",
57     defaultCurrency: "USD",
58   }).returning();
59
60   if (!newProfile) {
61     res.status(500).json({ error: "InternalServerError", message: "Failed to create profile" });
62     return;
63   }
64
65   const token = signToken(newUser.id, newUser.email);
66
67   res.status(201).json({
68     token,
69     user: {
70       id: newUser.id,
71       email: newUser.email,
```

```
72     emailVerified: newUser.emailVerified,
73     createdAt: newUser.createdAt,
74   },
75   profile: {
76     id: newProfile.id,
77     userId: newProfile.userId,
78     firstName: newProfile.firstName,
79     lastName: newProfile.lastName,
80     displayName: newProfile.displayName,
81     avatarUrl: newProfile.avatarUrl,
82     phone: newProfile.phone,
83     timeZone: newProfile.timeZone,
84     defaultCurrency: newProfile.defaultCurrency,
85     createdAt: newProfile.createdAt,
86   },
87   });
88 });
89
90 // POST /auth/login
91 router.post("/auth/login", async (req, res) => {
92   const { email, password } = req.body as { email?: string; password?: string };
93
94   if (!email || !password) {
95     res.status(400).json({ error: "BadRequest", message: "Email and password are required" });
96     return;
97   }
98
99   const [user] = await db.select().from(users).where(eq(users.email, email.toLowerCase())).limit(1);
100   if (!user || !user.passwordHash) {
101     res.status(401).json({ error: "Unauthorized", message: "Invalid email or password" });
102     return;
103   }
104
105   const passwordValid = await bcrypt.compare(password, user.passwordHash);
106   if (!passwordValid) {
107     res.status(401).json({ error: "Unauthorized", message: "Invalid email or password" });
108     return;
109   }
110
111   await db.update(users).set({ lastLoginAt: new Date() }).where(eq(users.id, user.id));
112
113   const [profile] = await db.select().from(profiles).where(eq(profiles.userId, user.id)).limit(1);
114
115   const token = signToken(user.id, user.email);
116
117   res.json({
118     token,
119     user: {
120       id: user.id,
121       email: user.email,
122       emailVerified: user.emailVerified,
123       createdAt: user.createdAt,
124     },
125     profile: profile
126     ? {
127       id: profile.id,
128       userId: profile.userId,
129       firstName: profile.firstName,
130       lastName: profile.lastName,
```

```
131         displayName: profile.displayName,
132         avatarUrl: profile.avatarUrl,
133         phone: profile.phone,
134         timeZone: profile.timeZone,
135         defaultCurrency: profile.defaultCurrency,
136         createdAt: profile.createdAt,
137     }
138     : null,
139 });
140 });
141
142 // POST /auth/logout
143 router.post("/auth/logout", (_req, res) => {
144     res.json({ message: "Logged out successfully" });
145 });
146
147 // GET /auth/me
148 router.get("/auth/me", requireAuth, async (req, res) => {
149     const userId = (req as AuthenticatedRequest).userId;
150
151     const [user] = await db.select().from(users).where(eq(users.id, userId)).limit(1);
152     if (!user) {
153         res.status(401).json({ error: "Unauthorized", message: "User not found" });
154         return;
155     }
156
157     const [profile] = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
158
159     res.json({
160         token: req.headers.authorization?.slice(7) ?? "",
161         user: {
162             id: user.id,
163             email: user.email,
164             emailVerified: user.emailVerified,
165             createdAt: user.createdAt,
166         },
167         profile: profile
168         ? {
169             id: profile.id,
170             userId: profile.userId,
171             firstName: profile.firstName,
172             lastName: profile.lastName,
173             displayName: profile.displayName,
174             avatarUrl: profile.avatarUrl,
175             phone: profile.phone,
176             timeZone: profile.timeZone,
177             defaultCurrency: profile.defaultCurrency,
178             createdAt: profile.createdAt,
179         }
180         : null,
181     });
182 });
183
184 // POST /auth/forgot-password
185 router.post("/auth/forgot-password", async (req, res) => {
186     const { email } = req.body as { email?: string };
187     if (!email) {
188         res.status(400).json({ error: "BadRequest", message: "Email is required" });
189         return;
190     }
191 });
```

artifacts/api-server/src/routes/auth.ts (continued)

```
190   }
191
192   const [user] = await db.select().from(users).where(eq(users.email, email.toLowerCase())).limit(1);
193   if (user) {
194     const resetToken = crypto.randomBytes(32).toString("hex");
195     const expiresAt = new Date(Date.now() + 3600 * 1000); // 1 hour
196     await db.update(users)
197       .set({ resetToken, resetTokenExpiresAt: expiresAt })
198       .where(eq(users.id, user.id));
199     // In production: send email with reset link
200     // For dev: token is in DB, accessible via logs
201   }
202
203   // Always return 200 to prevent email enumeration
204   res.json({ message: "If an account exists with this email, you will receive reset instructions." });
205 });
206
207 // POST /auth/reset-password
208 router.post("/auth/reset-password", async (req, res) => {
209   const { token, password } = req.body as { token?: string; password?: string };
210
211   if (!token || !password) {
212     res.status(400).json({ error: "BadRequest", message: "Token and password are required" });
213     return;
214   }
215
216   if (password.length < 8) {
217     res.status(400).json({ error: "BadRequest", message: "Password must be at least 8 characters" });
218     return;
219   }
220
221   const [user] = await db.select().from(users).where(eq(users.resetToken, token)).limit(1);
222   if (!user || !user.resetTokenExpiresAt || user.resetTokenExpiresAt < new Date()) {
223     res.status(400).json({ error: "BadRequest", message: "Invalid or expired reset token" });
224     return;
225   }
226
227   const passwordHash = await bcrypt.hash(password, 12);
228   await db.update(users)
229     .set({ passwordHash, resetToken: null, resetTokenExpiresAt: null })
230     .where(eq(users.id, user.id));
231
232   res.json({ message: "Password reset successfully. Please log in with your new password." });
233 });
234
235 export default router;
236
```

artifacts/api-server/src/routes/profile.ts

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { profiles } from "@workspace/db";
4 import { eq } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7 const router = Router();
8
9 // GET /profile
```

```
10  router.get("/profile", requireAuth, async (req, res) => {
11    const userId = (req as AuthenticatedRequest).userId;
12
13    const [profile] = await db
14      .select()
15      .from(profiles)
16      .where(eq(profiles.userId, userId))
17      .limit(1);
18
19    if (!profile) {
20      res.status(404).json({ error: "NotFound", message: "Profile not found" });
21      return;
22    }
23
24    res.json({
25      id: profile.id,
26      userId: profile.userId,
27      firstName: profile.firstName,
28      lastName: profile.lastName,
29      displayName: profile.displayName,
30      avatarUrl: profile.avatarUrl,
31      phone: profile.phone,
32      timeZone: profile.timeZone,
33      defaultCurrency: profile.defaultCurrency,
34      createdAt: profile.createdAt,
35    });
36  });
37
38  // PATCH /profile
39  router.patch("/profile", requireAuth, async (req, res) => {
40    const userId = (req as AuthenticatedRequest).userId;
41    const { firstName, lastName, displayName, avatarUrl, phone, timeZone, defaultCurrency } =
42      req.body as {
43        firstName?: string;
44        lastName?: string;
45        displayName?: string;
46        avatarUrl?: string | null;
47        phone?: string | null;
48        timeZone?: string;
49        defaultCurrency?: string;
50      };
51
52    const [existing] = await db
53      .select()
54      .from(profiles)
55      .where(eq(profiles.userId, userId))
56      .limit(1);
57
58    if (!existing) {
59      res.status(404).json({ error: "NotFound", message: "Profile not found" });
60      return;
61    }
62
63    const updates: Partial<typeof existing> = { updatedAt: new Date() };
64    if (firstName !== undefined) updates.firstName = firstName;
65    if (lastName !== undefined) updates.lastName = lastName;
66    if (displayName !== undefined) updates.displayName = displayName;
67    if (avatarUrl !== undefined) updates.avatarUrl = avatarUrl;
68    if (phone !== undefined) updates.phone = phone;
```

artifacts/api-server/src/routes/profile.ts (continued)

```
69   if (timeZone !== undefined) updates.timeZone = timeZone;
70   if (defaultCurrency !== undefined) updates.defaultCurrency = defaultCurrency;
71
72   const [updated] = await db
73     .update(profiles)
74     .set(updates)
75     .where(eq(profiles.userId, userId))
76     .returning();
77
78   if (!updated) {
79     res.status(500).json({ error: "InternalError", message: "Failed to update profile" });
80     return;
81   }
82
83   res.json({
84     id: updated.id,
85     userId: updated.userId,
86     firstName: updated.firstName,
87     lastName: updated.lastName,
88     displayName: updated.displayName,
89     avatarUrl: updated.avatarUrl,
90     phone: updated.phone,
91     timeZone: updated.timeZone,
92     defaultCurrency: updated.defaultCurrency,
93     createdAt: updated.createdAt,
94   });
95 });
96
97 export default router;
98
```

artifacts/api-server/src/routes/dashboard.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import {
4    jars,
5    jarMembers,
6    contributions,
7    contributionSchedules,
8    profiles,
9    notifications,
10   activityEvents,
11 } from "@workspace/db";
12 import { eq, and, desc, sql, inArray, count, gte, lte } from "drizzle-orm";
13 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
14 import { calculateJarHealth, calculateMemberHealth } from "../lib/jar-health.js";
15 import { computeNextDueDate, computePreviousDueDate, toISODate } from "../lib/schedule-utils.js";
16
17 const router = Router();
18
19 // GET /dashboard
20 router.get("/dashboard", requireAuth, async (req, res) => {
21   const userId = (req as AuthenticatedRequest).userId;
22
23   // Get all jars for this user
24   const userMemberships = await db
25     .select()
26     .from(jarMembers)
```



```

27     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))));
28
29     const memberJarIds = userMemberships.map((m) => m.jarId);
30
31     const userJars = memberJarIds.length > 0
32       ? await db
33         .select()
34         .from(jars)
35         .where(inArray(jars.id, memberJarIds))
36         .orderBy(desc(jars.updatedAt))
37       : [];
38
39     // Active jars in saving phase
40     const activeJars = userJars.filter((j) => ["Saving", "CommitmentPending",
41 "Completed", "Simulated"].includes(j.status));
42     const activeJarsCount = activeJars.length;
43
44     // Unread notifications
45     const unreadResult = await db
46       .select({ count: sql<number>`count(*)` })
47       .from(notifications)
48       .where(and(eq(notifications.userId, userId), eq(notifications.isRead, false)));
49     const unreadNotifications = Number(unreadResult[0]?.count ?? 0);
50
51     // Featured jar: most recently active saving jar
52     const featuredJar = activeJars[0] ?? userJars[0] ?? null;
53
54     let featuredJarData = null;
55     let personalProgress = null;
56     let memberProgress: unknown[] = [];
57     let upcomingActivity: unknown[] = [];
58
59     if (featuredJar) {
60       // Total saved for featured jar
61       const savedResult = await db
62         .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
63         .from(contributions)
64         .where(
65           and(
66             eq(contributions.jarId, featuredJar.id),
67             inArray(contributions.status, ["completed", "simulated"]),
68           ),
69         );
70       const totalSavedCents = Number(savedResult[0]?.total ?? 0);
71       const percentFunded = featuredJar.goalAmountCents > 0
72         ? Math.min(100, Math.round((totalSavedCents / featuredJar.goalAmountCents) * 1000) / 10)
73         : 0;
74
75       const memberCount = await db
76         .select({ count: sql<number>`count(*)` })
77         .from(jarMembers)
78         .where(and(eq(jarMembers.jarId, featuredJar.id), eq(jarMembers.status, "active")));
79
80       let health = null;
81       if (featuredJar.launchesAt) {
82         health = calculateJarHealth(
83           featuredJar.goalAmountCents,
84           totalSavedCents,
85           featuredJar.launchesAt,

```

```

86         new Date(featuredJar.targetDate),
87     );
88 }
89
90     const daysRem = Math.max(0, Math.ceil((new Date(featuredJar.targetDate).getTime() - Date.now()) /
91     1000 * 60 * 60 * 24));
92     featuredJarData = {
93         id: featuredJar.id,
94         organizerId: featuredJar.organizerId,
95         name: featuredJar.name,
96         category: featuredJar.category,
97         destination: featuredJar.destination,
98         coverImageUrl: featuredJar.coverImageUrl,
99         targetDate: featuredJar.targetDate,
100         goalAmountCents: featuredJar.goalAmountCents,
101         currency: featuredJar.currency,
102         status: featuredJar.status,
103         memberCount: Number(memberCount[0]?.count ?? 0),
104         totalSavedCents,
105         percentFunded,
106         daysRemaining: daysRem,
107         userRole: userMemberships.find((m) => m.jarId === featuredJar.id)?.role ?? null,
108         health,
109     };
110
111     // Personal progress for featured jar
112     const myMembership = userMemberships.find((m) => m.jarId === featuredJar.id);
113     if (myMembership) {
114         const myContribs = await db
115             .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
116             .from(contributions)
117             .where(
118                 and(
119                     eq(contributions.jarId, featuredJar.id),
120                     eq(contributions.memberId, myMembership.id),
121                     inArray(contributions.status, ["completed", "simulated"]),
122                 ),
123             );
124         const contributedAmountCents = Number(myContribs[0]?.total ?? 0);
125         const remainingAmountCents = Math.max(0, myMembership.contributionTargetCents -
126         contributedAmountCents);
127         const percentComplete = myMembership.contributionTargetCents > 0
128             ? Math.min(100, (contributedAmountCents / myMembership.contributionTargetCents) * 100)
129             : 0;
130
131         // Fetch active schedule for this member
132         const scheduleRows = await db
133             .select()
134             .from(contributionSchedules)
135             .where(
136                 and(
137                     eq(contributionSchedules.jarId, featuredJar.id),
138                     eq(contributionSchedules.memberId, myMembership.id),
139                     eq(contributionSchedules.isActive, true),
140                 ),
141             )
142             .limit(1);
143         const activeSchedule = scheduleRows[0] ?? null;
144         let nextScheduledDate: string | null = null;

```

```

145     let nextScheduledAmountCents: number | null = null;
146     let hasMissedContribution = false;
147
148     if (activeSchedule && !activeSchedule.isPaused) {
149         const nextDue = computeNextDueDate(activeSchedule, new Date());
150         if (nextDue) {
151             nextScheduledDate = toISODate(nextDue);
152             nextScheduledAmountCents = activeSchedule.amountCents;
153         }
154
155         // Check if the previous due date was missed (no contribution in ±1 day window)
156         const prevDue = computePreviousDueDate(activeSchedule, new Date());
157         if (prevDue) {
158             const windowStart = toISODate(new Date(prevDue.getFullYear(), prevDue.getMonth(),
159             ----- const windowEnd = toISODate(new Date(prevDue.getFullYear(), prevDue.getMonth(),
160 prevDue.getDate() + 1));
161
162             const contribsNearDue = await db
163                 .select({ id: contributions.id })
164                 .from(contributions)
165                 .where(
166                     and(
167                         eq(contributions.memberId, myMembership.id),
168                         eq(contributions.jarId, featuredJar.id),
169                         gte(contributions.contributionDate, windowStart),
170                         lte(contributions.contributionDate, windowEnd),
171                         inArray(contributions.status, ["completed", "simulated"]),
172                     ),
173                 )
174                 .limit(1);
175
176             hasMissedContribution = !contribsNearDue[0];
177         }
178
179         const baseIsOnTrack = percentComplete >= (featuredJarData.health?.expectedPercentage ?? 0) - 10;
180
181         personalProgress = {
182             targetAmountCents: myMembership.contributionTargetCents,
183             contributedAmountCents,
184             remainingAmountCents,
185             percentComplete,
186             nextScheduledDate,
187             nextScheduledAmountCents,
188             estimatedCompletionDate: null,
189             isOnTrack: baseIsOnTrack && !hasMissedContribution,
190             hasMissedScheduledContribution: hasMissedContribution,
191         };
192     }
193
194     // Member progress for featured jar
195     const allMembers = await db
196         .select()
197         .from(jarMembers)
198         .where(and(eq(jarMembers.jarId, featuredJar.id), eq(jarMembers.status, "active")));
199
200     const memberProfiles = await db
201         .select()
202         .from(profiles)
203         .where(inArray(profiles.userId, allMembers.map((m) => m.userId)));

```

```

204
205     const profileMap = new Map(memberProfiles.map((p) => [p.userId, p]));
206
207     const daysElapsed = featuredJar.launchesAt
208       ? Math.ceil((Date.now() - featuredJar.launchesAt.getTime()) / 86_400_000)
209       : 0;
210     const totalDays = Math.ceil((new Date(featuredJar.targetDate).getTime() -
211       featuredJar.launchesAt.getTime()) / 86_400_000);
212     memberProgress = await Promise.all(
213       allMembers.map(async (m) => {
214         const memberContribs = await db
215           .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
216           .from(contributions)
217           .where(
218             and(
219               eq(contributions.jarId, featuredJar.id),
220               eq(contributions.memberId, m.id),
221               inArray(contributions.status, ["completed", "simulated"]),
222             ),
223           );
224         const contributedAmountCents = Number(memberContribs[0]?.total ?? 0);
225         const percentComplete = m.contributionTargetCents > 0
226           ? Math.min(100, Math.round((contributedAmountCents / m.contributionTargetCents) * 1000) / 10)
227           : 0;
228         const status = calculateMemberHealth(
229           m.contributionTargetCents,
230           contributedAmountCents,
231           daysElapsed,
232           totalDays,
233         );
234         const prof = profileMap.get(m.userId);
235
236         return {
237           memberId: m.id,
238           userId: m.userId,
239           displayName: prof?.displayName ?? "Unknown",
240           avatarUrl: prof?.avatarUrl ?? null,
241           role: m.role,
242           contributionTargetCents: m.contributionTargetCents,
243           contributedAmountCents,
244           percentComplete,
245           status,
246         };
247       }),
248     );
249
250     // Upcoming activity items
251     const targetDate = new Date(featuredJar.targetDate);
252     upcomingActivity = [
253       {
254         type: "jar_target_date",
255         title: "Savings Target Date",
256         description: `${featuredJar.name} savings goal deadline`,
257         date: featuredJar.targetDate,
258         jarId: featuredJar.id,
259         jarName: featuredJar.name,
260       },
261     ].filter((item) => new Date(item.date) > new Date());
262   }

```

artifacts/api-server/src/routes/dashboard.ts (continued)

```
263
264 // Recent contributions across all jars
265 const recentContribs = memberJarIds.length > 0
266   ? await db
267     .select()
268     .from(contributions)
269     .where(inArray(contributions.jarId, memberJarIds))
270     .orderBy(desc(contributions.createdAt))
271     .limit(5)
272   : [];
273
274 res.json({
275   featuredJar: featuredJarData,
276   personalProgress,
277   memberProgress,
278   upcomingActivity,
279   recentContributions: recentContribs,
280   totalJars,
281   activeJars: activeJarsCount,
282   unreadNotifications,
283 });
284 });
285
286 export default router;
287
```

artifacts/api-server/src/routes/jars.ts

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import {
4   jars,
5   jarMembers,
6   contributions,
7   profiles,
8   agreements,
9   agreementAcceptances,
10 } from "@workspace/db";
11 import { eq, and, or, desc, sql, inArray } from "drizzle-orm";
12 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
13 import { calculateJarHealth } from "../lib/jar-health.js";
14 import { logActivity } from "../lib/activity.js";
15 import { notifyAllMembers } from "../lib/notifications.js";
16
17 const router = Router();
18
19 // Helper: check jar access and return member info
20 async function getJarAccess(jarId: string, userId: string) {
21   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
22   if (!jar[0]) return { jar: null, member: null, isOrganizer: false };
23
24   const member = await db
25     .select()
26     .from(jarMembers)
27     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status,
28 "active"))))
29     .limit(1);
30
31   const isOrganizer = jar[0].organizerId === userId;
32   return { jar: jar[0], member: member[0] ?? null, isOrganizer };
33 }
```

```

32   }
33
34   // Helper: get total saved for a jar
35   async function getTotalSaved(jarId: string): Promise<number> {
36     const result = await db
37       .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
38       .from(contributions)
39       .where(
40         and(
41           eq(contributions.jarId, jarId),
42           inArray(contributions.status, ["completed", "simulated"]),
43         ),
44       );
45     return Number(result[0]?.total ?? 0);
46   }
47
48   // Helper: compute daysRemaining
49   function daysRemaining(targetDate: string): number | null {
50     if (!targetDate) return null;
51     const diff = new Date(targetDate).getTime() - Date.now();
52     return Math.max(0, Math.ceil(diff / 86_400_000));
53   }
54
55   // Helper: build jar summary
56   async function buildJarSummary(jar: typeof jars.$inferSelect, userId: string) {
57     const totalSavedCents = await getTotalSaved(jar.id);
58     const percentFunded = jar.goalAmountCents > 0
59       ? Math.min(100, Math.round((totalSavedCents / jar.goalAmountCents) * 1000) / 10)
60       : 0;
61
62     const memberCount = await db
63       .select({ count: sql<number>`count(*)` })
64       .from(jarMembers)
65       .where(and(eq(jarMembers.jarId, jar.id), eq(jarMembers.status, "active")));
66
67     const userMember = await db
68       .select()
69       .from(jarMembers)
70       .where(and(eq(jarMembers.jarId, jar.id), eq(jarMembers.userId, userId)))
71       .limit(1);
72
73     let health = null;
74     if (jar.launchedException && jar.status === "Saving") {
75       health = calculateJarHealth(
76         jar.goalAmountCents,
77         totalSavedCents,
78         jar.launchedException,
79         new Date(jar.targetDate),
80       );
81     }
82
83     return {
84       id: jar.id,
85       organizerId: jar.organizerId,
86       name: jar.name,
87       category: jar.category,
88       destination: jar.destination,
89       coverImageUrl: jar.coverImageUrl,
90       targetDate: jar.targetDate,

```

```

91     goalAmountCents: jar.goalAmountCents,
92     currency: jar.currency,
93     status: jar.status,
94     memberCount: Number(memberCount[0]?.count ?? 0),
95     totalSavedCents,
96     percentFunded,
97     daysRemaining: daysRemaining(jar.targetDate),
98     userRole: userMember[0]?.role ?? null,
99     health,
100   };
101 }
102
103 // GET /jars
104 router.get("/jars", requireAuth, async (req, res) => {
105   const userId = (req as AuthenticatedRequest).userId;
106   const { status } = req.query as { status?: string };
107
108   // Find all jars where user is organizer or active member
109   const userMembers = await db
110     .select({ jarId: jarMembers.jarId })
111     .from(jarMembers)
112     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active")));
113
114   const memberJarIds = userMembers.map((m) => m.jarId);
115
116   let query = db.select().from(jars).where(
117     memberJarIds.length > 0
118     ? or(eq(jars.organizerId, userId), inArray(jars.id, memberJarIds))
119     : eq(jars.organizerId, userId),
120   ).orderBy(desc(jars.updatedAt));
121
122   const jarList = await query;
123   const filtered = status ? jarList.filter((j) => j.status === status) : jarList;
124
125   const summaries = await Promise.all(
126     filtered.map((jar) => buildJarSummary(jar, userId)),
127   );
128
129   res.json(summaries);
130 });
131
132 // POST /jars
133 router.post("/jars", requireAuth, async (req, res) => {
134   const userId = (req as AuthenticatedRequest).userId;
135   const {
136     name,
137     category = "Vacation",
138     description,
139     destination,
140     coverImageUrl,
141     startDate,
142     endDate,
143     targetDate,
144     goalAmountCents,
145     currency = "USD",
146     approvalThreshold = 0.67,
147   } = req.body as {
148     name?: string;
149     category?: string;

```

```

150     description?: string;
151     destination?: string;
152     coverImageUrl?: string;
153     startDate?: string;
154     endDate?: string;
155     targetDate?: string;
156     goalAmountCents?: number;
157     currency?: string;
158     approvalThreshold?: number;
159   };
160
161   if (!name || !targetDate || !goalAmountCents) {
162     res.status(400).json({ error: "BadRequest", message: "name, targetDate, and goalAmountCents are
163       required" });
164     return;
165   }
166
167   if (goalAmountCents < 100) {
168     res.status(400).json({ error: "BadRequest", message: "Goal must be at least $1.00" });
169     return;
170   }
171
172   // Generate slug
173   const baseSlug = name.toLowerCase().replace(/^[^a-z0-9]+/g, "-").slice(0, 40);
174   const slug = `${baseSlug}-${Date.now().toString(36)}`;
175
176   const [jar] = await db
177     .insert(jars)
178     .values({
179       organizerId: userId,
180       name,
181       slug,
182       category,
183       description: description ?? null,
184       destination: destination ?? null,
185       coverImageUrl: coverImageUrl ?? null,
186       startDate: startDate ?? null,
187       endDate: endDate ?? null,
188       targetDate,
189       goalAmountCents,
190       currency,
191       status: "Draft",
192       approvalThreshold: String(approvalThreshold),
193     })
194     .returning();
195
196   if (!jar) {
197     res.status(500).json({ error: "InternalServerError", message: "Failed to create jar" });
198     return;
199   }
200
201   // Add organizer as member
202   await db.insert(jarMembers).values({
203     jarId: jar.id,
204     userId,
205     role: "organizer",
206     contributionTargetCents: 0,
207     status: "active",
208     joinedAt: new Date(),
209   });

```



```

209
210 // Create default agreement
211 await db.insert(agreements).values({
212   jarId: jar.id,
213   version: "1.0",
214   content: DEFAULT_AGREEMENT_TEXT,
215   effectiveDate: new Date().toISOString().split("T")[0]!,
216 });
217
218 await logActivity({ jarId: jar.id, userId, eventType: "jar_created", description: `${name} was
219 created` });
220 res.status(201).json(await buildJarSummary(jar, userId));
221 });
222
223 // GET /jars/:jarId
224 router.get("/jars/:jarId", requireAuth, async (req, res) => {
225   const userId = (req as AuthenticatedRequest).userId;
226   const { jarId } = req.params as { jarId: string };
227
228   const { jar, member, isOrganizer } = await getJarAccess(jarId, userId);
229   if (!jar) {
230     res.status(404).json({ error: "NotFound", message: "Jar not found" });
231     return;
232   }
233
234   if (!isOrganizer && !member) {
235     res.status(403).json({ error: "Forbidden", message: "You do not have access to this jar" });
236     return;
237   }
238
239   const totalSavedCents = await getTotalSaved(jarId);
240   const percentFunded = jar.goalAmountCents > 0
241     ? Math.min(100, Math.round((totalSavedCents / jar.goalAmountCents) * 1000) / 10)
242     : 0;
243
244   const memberCount = await db
245     .select({ count: sql<number>`count(*)` })
246     .from(jarMembers)
247     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
248
249   let health = null;
250   if (jar.launchesAt && ["Saving", "CommitmentPending", "Committed"].includes(jar.status)) {
251     health = calculateJarHealth(
252       jar.goalAmountCents,
253       totalSavedCents,
254       jar.launchesAt,
255       new Date(jar.targetDate),
256     );
257   }
258
259   res.json({
260     id: jar.id,
261     organizerId: jar.organizerId,
262     name: jar.name,
263     slug: jar.slug,
264     category: jar.category,
265     description: jar.description,
266     destination: jar.destination,
267     coverImageUrl: jar.coverImageUrl,

```

```

268     startDate: jar.startDate,
269     endDate: jar.endDate,
270     targetDate: jar.targetDate,
271     goalAmountCents: jar.goalAmountCents,
272     currency: jar.currency,
273     status: jar.status,
274     approvalThreshold: Number(jar.approvalThreshold),
275     memberCount: Number(memberCount[0]?.count ?? 0),
276     totalSavedCents,
277     percentFunded,
278     daysRemaining: daysRemaining(jar.targetDate),
279     launchedAt: jar.launchedAt,
280     createdAt: jar.createdAt,
281     updatedAt: jar.updatedAt,
282     health,
283     userRole: member?.role ?? (isOrganizer ? "organizer" : null),
284   });
285   });
286
287   // PATCH /jars/:jarId
288   router.patch("/jars/:jarId", requireAuth, async (req, res) => {
289     const userId = (req as AuthenticatedRequest).userId;
290     const { jarId } = req.params as { jarId: string };
291
292     const { jar, isOrganizer } = await getJarAccess(jarId, userId);
293     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
294     if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can edit the"
295     }); return; }
296
297     const updates: Partial<typeof jar> = { updatedAt: new Date() };
298     const allowed = ["name", "description", "destination", "coverImageUrl", "startDate", "endDate",
299     "targetDate", "goalAmountCents", "currency"];
300     if (req.body[key] !== undefined) (updates as Record<string, unknown>)[key] = req.body[key];
301   }
302
303   const [updated] = await db.update(jars).set(updates).where(eq(jars.id, jarId)).returning();
304   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to update jar" });
305   return; }
306   res.json(await buildJarSummary(updated, userId));
307   });
308
309   // POST /jars/:jarId/launch
310   router.post("/jars/:jarId/launch", requireAuth, async (req, res) => {
311     const userId = (req as AuthenticatedRequest).userId;
312     const { jarId } = req.params as { jarId: string };
313
314     const { jar, isOrganizer } = await getJarAccess(jarId, userId);
315     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
316     if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can launch the
317     jar" }); return; }
318     if (!["Draft", "Inviting"].includes(jar.status)) {
319       res.status(400).json({ error: "BadRequest", message: "Jar cannot be launched from its current
320       status" }); return;
321     }
322
323     const [updated] = await db
324       .update(jars)
325       .set({ status: "Saving", launchedAt: new Date(), updatedAt: new Date() })
326       .where(eq(jars.id, jarId))
327       .returning();

```

```

327   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to launch jar" });
328   return; }
329   await logActivity({ jarId, userId, eventType: "jar_created", description: `${jar.name} is now open for
330   contributi...
331   // Get all member user IDs to notify
332   const members = await db
333     .select({ userId: jarMembers.userId })
334     .from(jarMembers)
335     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
336
337   await notifyAllMembers({
338     jarId,
339     memberUserIds: members.map((m) => m.userId),
340     type: "general",
341     title: `${jar.name} is now active!`,
342     message: "Your jar is now open for contributions. Start saving!",
343     excludeUserId: userId,
344   });
345
346   res.json(await buildJarSummary(updated, userId));
347 });
348
349 // POST /jars/:jarId/cancel
350 router.post("/jars/:jarId/cancel", requireAuth, async (req, res) => {
351   const userId = (req as AuthenticatedRequest).userId;
352   const { jarId } = req.params as { jarId: string };
353
354   const { jar, isOrganizer } = await getJarAccess(jarId, userId);
355   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
356   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can cancel the
357   if (["Cancelled", "Completed"].includes(jar.status)) {
358     res.status(400).json({ error: "BadRequest", message: "Jar is already cancelled or completed" });
359     return;
360   }
361
362   const [updated] = await db
363     .update(jars)
364     .set({ status: "Cancelled", updatedAt: new Date() })
365     .where(eq(jars.id, jarId))
366     .returning();
367
368   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to cancel jar" });
369
370   await logActivity({ jarId, userId, eventType: "jar_cancelled", description: `${jar.name} was
371   res.json(await buildJarSummary(updated, userId));
372 });
373
374
375 // GET /jars/:jarId/health
376 router.get("/jars/:jarId/health", requireAuth, async (req, res) => {
377   const userId = (req as AuthenticatedRequest).userId;
378   const { jarId } = req.params as { jarId: string };
379
380   const { jar, member, isOrganizer } = await getJarAccess(jarId, userId);
381   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
382   if (!isOrganizer && !member) { res.status(403).json({ error: "Forbidden", message: "Access denied" });
383
384   if (!jar.launchingAt) {
385     res.json({

```

artifacts/api-server/src/routes/jars.ts (continued)

```
386     status: "OnTrack",
387     message: "Jar has not launched yet.",
388     actualPercentage: 0,
389     expectedPercentage: 0,
390     daysElapsed: 0,
391     totalDays: Math.ceil((new Date(jar.targetDate).getTime() - Date.now()) / 86_400_000),
392     amountBehindCents: 0,
393     estimatedCompletionDate: null,
394   });
395   return;
396 }
397
398 const totalSavedCents = await getTotalSaved(jarId);
399 const health = calculateJarHealth(
400   jar.goalAmountCents,
401   totalSavedCents,
402   jar.launchedAt,
403   new Date(jar.targetDate),
404 );
405
406 res.json(health);
407 });
408
409 const DEFAULT_AGREEMENT_TEXT = `TripJar Savings Agreement
410
411 1. SAVING PHASE: All contributions are made voluntarily. During the Saving Phase, contributions remain
412 under the co...
413
414 2. COMMITMENT REQUEST: Before any funds are designated for a specific purchase, the Organizer must
415 submit a Commitm...
416
417 3. COMMITTED FUNDS: Once a Commitment Request is approved and the Lock Date has passed, designated funds
418 become com...
419
420 4. CANCELLATION: If the Jar is cancelled before any Commitment Request is approved, all members may
421 request a full ...
422
423 5. REFUNDS: Refund requests are subject to review. TripJar is not responsible for amounts already
424 transferred to ex...
425
426 6. SIMULATED PAYMENTS: This version of TripJar uses simulated payment data. No real money is
427 transferred. This agre...
428
429 7. DISCLAIMER: The exact legal and financial terms governing real-money transactions will be provided
430 separately be...
431
432 All members agree to treat each other fairly and communicate openly about any inability to meet
433 contribution schedu...
434
435 export default router;
```

artifacts/api-server/src/routes/members.ts

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, contributions, profiles } from "@workspace/db";
4 import { eq, and, sql, inArray } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6 import { calculateMemberHealth } from "../lib/jar-health.js";
7
8 const router = Router();
9
10 // GET /jars/:jarId/members
11 router.get("/jars/:jarId/members", requireAuth, async (req, res) => {
12   const userId = (req as AuthenticatedRequest).userId;
13   const { jarId } = req.params as { jarId: string };
```

```

14
15 // Check access
16 const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
17 if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
18
19 const userMember = await db
20   .select()
21   .from(jarMembers)
22   .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status,
23     "active").limit(1);
24
25 if (jar[0].organizerId !== userId && !userMember[0]) {
26   res.status(403).json({ error: "Forbidden", message: "Access denied" });
27   return;
28 }
29
30 const allMembers = await db
31   .select()
32   .from(jarMembers)
33   .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
34
35 const memberProfiles = await db
36   .select()
37   .from(profiles)
38   .where(inArray(profiles.userId, allMembers.map((m) => m.userId)));
39
40 const profileMap = new Map(memberProfiles.map((p) => [p.userId, p]));
41
42 const daysElapsed = jar[0].launchedAt
43   ? Math.ceil((Date.now() - jar[0].launchedAt.getTime()) / 86_400_000)
44   : 0;
45 const totalDays = jar[0].launchedAt
46   ? Math.ceil((new Date(jar[0].targetDate).getTime() - jar[0].launchedAt.getTime()) / 86_400_000)
47   : 0;
48
49 const result = await Promise.all(
50   allMembers.map(async (m) => {
51     const contribResult = await db
52       .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
53       .from(contributions)
54       .where(
55         and(
56           eq(contributions.jarId, jarId),
57           eq(contributions.memberId, m.id),
58           inArray(contributions.status, ["completed", "simulated"]),
59         ),
60       );
61     const contributedAmountCents = Number(contribResult[0]?.total ?? 0);
62     const percentComplete =
63       m.contributionTargetCents > 0
64       ? Math.min(100, Math.round((contributedAmountCents / m.contributionTargetCents) * 1000) / 10)
65       : 0;
66
67     const healthStatus = calculateMemberHealth(
68       m.contributionTargetCents,
69       contributedAmountCents,
70       daysElapsed,
71       totalDays,
72     );

```

```

73
74     const prof = profileMap.get(m.userId);
75
76     return {
77         id: m.id,
78         jarId: m.jarId,
79         userId: m.userId,
80         role: m.role,
81         contributionTargetCents: m.contributionTargetCents,
82         contributedAmountCents,
83         percentComplete,
84         status: m.status,
85         healthStatus,
86         joinedAt: m.joinedAt,
87         profile: prof
88     } {
89         id: prof.id,
90         userId: prof.userId,
91         firstName: prof.firstName,
92         lastName: prof.lastName,
93         displayName: prof.displayName,
94         avatarUrl: prof.avatarUrl,
95         phone: prof.phone,
96         timeZone: prof.timeZone,
97         defaultCurrency: prof.defaultCurrency,
98         createdAt: prof.createdAt,
99     }
100     : null,
101 };
102 }},
103 );
104
105 res.json(result);
106 });
107
108 // PATCH /jars/:jarId/members/:memberId
109 router.patch("/jars/:jarId/members/:memberId", requireAuth, async (req, res) => {
110     const userId = (req as AuthenticatedRequest).userId;
111     const { jarId, memberId } = req.params as { jarId: string; memberId: string };
112
113     const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
114     if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
115     if (jar[0].organizerId !== userId) {
116         res.status(403).json({ error: "Forbidden", message: "Only organizer can update members" });
117         return;
118     }
119
120     const { contributionTargetCents, role, status } = req.body as {
121         contributionTargetCents?: number;
122         role?: string;
123         status?: string;
124     };
125
126     const updates: Partial<typeof jarMembers.$inferSelect> = {};
127     if (contributionTargetCents !== undefined) updates.contributionTargetCents = contributionTargetCents;
128     if (role !== undefined && role !== "organizer") updates.role = role;
129     if (status !== undefined) updates.status = status;
130
131     const [updated] = await db

```

artifacts/api-server/src/routes/members.ts (continued)

```
132     .update(jarMembers)
133     .set(updates)
134     .where(and(eq(jarMembers.id, memberId), eq(jarMembers.jarId, jarId)))
135     .returning();
136
137     if (!updated) { res.status(404).json({ error: "NotFound", message: "Member not found" }); return; }
138
139     res.json({ id: updated.id, jarId: updated.jarId, userId: updated.userId, role: updated.role,
140     contributionTargetCe...
141     });
142     export default router;
143
```

artifacts/api-server/src/routes/invitations.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, invitations, profiles, users } from "@workspace/db";
4  import { eq, and, desc } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7  import { createNotification } from "../lib/notifications.js";
8  import crypto from "node:crypto";
9  import { sendInvitationEmail } from "../lib/email.js";
10
11  const router = Router();
12
13  // POST /jars/:jarId/invitations
14  router.post("/jars/:jarId/invitations", requireAuth, async (req, res) => {
15     const userId = (req as AuthenticatedRequest).userId;
16     const userEmail = (req as AuthenticatedRequest).userEmail;
17     const { jarId } = req.params as { jarId: string };
18
19     const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
20     if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
21     if (jar[0].organizerId !== userId) {
22         res.status(403).json({ error: "Forbidden", message: "Only organizer can send invitations" });
23         return;
24     }
25
26     const { email, role = "member", contributionTargetCents } = req.body as {
27         email?: string;
28         role?: string;
29         contributionTargetCents?: number;
30     };
31
32     if (!email) { res.status(400).json({ error: "BadRequest", message: "Email is required" }); return; }
33     if (email.toLowerCase() === userEmail?.toLowerCase()) {
34         res.status(400).json({ error: "BadRequest", message: "You cannot invite yourself" });
35         return;
36     }
37
38     // Check for existing active invitation
39     const existing = await db
40         .select()
41         .from(invitations)
42         .where(and(eq(invitations.jarId, jarId), eq(invitations.email, email.toLowerCase()),
43         // ...
44         .limit(1);
```

```

45   if (existing[0]) {
46     res.status(409).json({ error: "Conflict", message: "A pending invitation already exists for this
47     });
48   }
49
50   const token = crypto.randomBytes(32).toString("hex");
51   const expiresAt = new Date(Date.now() + 7 * 24 * 3600 * 1000); // 7 days
52
53   const [invitation] = await db.insert(invitations).values({
54     jarId,
55     email: email.toLowerCase(),
56     role,
57     contributionTargetCents: contributionTargetCents ?? null,
58     status: "pending",
59     token,
60     sentAt: new Date(),
61     expiresAt,
62     invitedByUserId: userId,
63   }).returning();
64
65   if (!invitation) { res.status(500).json({ error: "InternalError", message: "Failed to create
66   invitation" }); return; }
67
68   // Send invitation email (non-blocking – errors are logged but don't fail the request)
69   const inviterProfile = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
70   const inviterName = inviterProfile[0]?.displayName ?? "A TripJar member";
71   sendInvitationEmail({
72     toEmail: email.toLowerCase(),
73     jarName: jar[0].name,
74     inviterName,
75     token,
76   }).catch(() => { /* already logged inside sendInvitationEmail */ });
77
78   await logActivity({
79     jarId,
80     userId,
81     eventType: "invitation_sent",
82     description: `Invitation sent to ${email}`,
83   });
84
85   // If the invited email belongs to an existing user, notify them
86   const invitedUser = await db.select().from(users).where(eq(users.email, email.toLowerCase())).limit(1);
87   if (invitedUser[0]) {
88     await createNotification({
89       userId: invitedUser[0].id,
90       type: "invitation_received",
91       title: `You're invited to ${jar[0].name}!`,
92       message: `Join the ${jar[0].name} savings jar and start saving together.`,
93       relatedJarId: jarId,
94     });
95   }
96
97   res.status(201).json(invitation);
98
99   // GET /invitations
100  router.get("/invitations", requireAuth, async (req, res) => {
101    const userEmail = (req as AuthenticatedRequest).userEmail;
102
103    const myInvitations = await db

```



```

104     .select()
105     .from(invitations)
106     .where(eq(invitations.email, userEmail.toLowerCase()))
107     .orderBy(desc(invitations.sentAt));
108
109     // Enrich with jar summaries
110     const result = await Promise.all(
111       myInvitations.map(async (inv) => {
112         const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
113         const memberCount = jar[0]
114           ? await db.select({ c: eq(jarMembers.status,
115             "active") }).from(jarMembers).where(eq(jarMembers.jarId, jar[0].id)).limit(1))
116           : [];
117         return {
118           ...inv,
119           jar: jar[0]
120             ? {
121                 id: jar[0].id,
122                 organizerId: jar[0].organizerId,
123                 name: jar[0].name,
124                 category: jar[0].category,
125                 destination: jar[0].destination,
126                 coverImageUrl: jar[0].coverImageUrl,
127                 targetDate: jar[0].targetDate,
128                 goalAmountCents: jar[0].goalAmountCents,
129                 currency: jar[0].currency,
130                 status: jar[0].status,
131                 memberCount: memberCount.length,
132                 totalSavedCents: 0,
133                 percentFunded: 0,
134                 daysRemaining: Math.max(0, Math.ceil((new Date(jar[0].targetDate).getTime() -
135                   Date.now()) / 864001000 - useRate)),
136                 health: null,
137               } : null,
138         };
139       }),
140     );
141
142     res.json(result);
143   });
144
145   // GET /invitations/token/:token
146   router.get("/invitations/token/:token", async (req, res) => {
147     const { token } = req.params as { token: string };
148
149     const [inv] = await db.select().from(invitations).where(eq(invitations.token, token)).limit(1);
150     if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found or expired" }); }
151
152     if (inv.status !== "pending" || inv.expiresAt < new Date()) {
153       res.status(404).json({ error: "NotFound", message: "Invitation is no longer valid" });
154       return;
155     }
156
157     const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
158
159     res.json({
160       ...inv,
161       jar: jar[0]
162         ? {

```

```

163         id: jar[0].id,
164         organizerId: jar[0].organizerId,
165         name: jar[0].name,
166         category: jar[0].category,
167         destination: jar[0].destination,
168         coverImageUrl: jar[0].coverImageUrl,
169         targetDate: jar[0].targetDate,
170         goalAmountCents: jar[0].goalAmountCents,
171         currency: jar[0].currency,
172         status: jar[0].status,
173         memberCount: 0,
174         totalSavedCents: 0,
175         percentFunded: 0,
176         daysRemaining: Math.max(0, Math.ceil((new Date(jar[0].targetDate).getTime() - Date.now()) /
177         // 24 * 60 * 60 * 1000)),
178         userRole: null,
179         health: null,
180     }
181     });
182 });
183
184 // POST /invitations/:invitationId/accept
185 router.post("/invitations/:invitationId/accept", requireAuth, async (req, res) => {
186     const userId = (req as AuthenticatedRequest).userId;
187     const userEmail = (req as AuthenticatedRequest).userEmail;
188     const { invitationId } = req.params as { invitationId: string };
189
190     const [inv] = await db.select().from(invitations).where(eq(invitations.id, invitationId)).limit(1);
191     if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found" }); return; }
192     if (inv.email !== userEmail.toLowerCase()) {
193         res.status(403).json({ error: "Forbidden", message: "This invitation is not for your email"
194         address" });
195         return;
196     }
197     if (inv.status !== "pending") {
198         res.status(400).json({ error: "BadRequest", message: `Invitation is already ${inv.status}` });
199         return;
200     }
201     if (inv.expiresAt < new Date()) {
202         await db.update(invitations).set({ status: "expired" }).where(eq(invitations.id, invitationId));
203         res.status(400).json({ error: "BadRequest", message: "Invitation has expired" });
204         return;
205     }
206
207     // Check if already a member
208     const existing = await db
209         .select()
210         .from(jarMembers)
211         .where(and(eq(jarMembers.jarId, inv.jarId), eq(jarMembers.userId, userId)))
212         .limit(1);
213
214     if (existing[0]) {
215         res.status(409).json({ error: "Conflict", message: "You are already a member of this jar" });
216         return;
217     }
218
219     const [member] = await db.insert(jarMembers).values({
220         jarId: inv.jarId,
221         userId,
222         role: inv.role,

```

```

222     contributionTargetCents: inv.contributionTargetCents ?? 0,
223     status: "active",
224     joinedAt: new Date(),
225   }).returning();
226
227   if (!member) { res.status(500).json({ error: "InternalError", message: "Failed to join jar" });
228   return; }
229
230   await db.update(invitations)
231     .set({ status: "accepted", acceptedAt: new Date() })
232     .where(eq(invitations.id, invitationId));
233
234   await logActivity({
235     jarId: inv.jarId,
236     userId,
237     eventType: "member_joined",
238     description: `A new member joined the jar`,
239   });
240
241   // Notify organizer
242   const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
243   if (jar[0]) {
244     const prof = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
245     await createNotification({
246       userId: jar[0].organizerId,
247       type: "member_joined",
248       title: "New member joined!",
249       message: `${prof[0]?.displayName ?? "Someone"} accepted your invitation to ${jar[0].name}.`,
250       relatedJarId: inv.jarId,
251     });
252   }
253
254   res.json({
255     id: member.id,
256     jarId: member.jarId,
257     userId: member.userId,
258     role: member.role,
259     contributionTargetCents: member.contributionTargetCents,
260     contributedAmountCents: 0,
261     percentComplete: 0,
262     status: member.status,
263     healthStatus: "NotStarted",
264     joinedAt: member.joinedAt,
265     profile: null,
266   });
267
268   // POST /invitations/:invitationId/decline
269   router.post("/invitations/:invitationId/decline", requireAuth, async (req, res) => {
270     const userEmail = (req as AuthenticatedRequest).userEmail;
271     const { invitationId } = req.params as { invitationId: string };
272
273     const [inv] = await db.select().from(invitations).where(eq(invitations.id, invitationId)).limit(1);
274     if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found" }); return; }
275     if (inv.email !== userEmail.toLowerCase()) {
276       res.status(403).json({ error: "Forbidden", message: "This invitation is not for your email" });
277       return;
278     }
279
280     await db.update(invitations).set({ status: "declined" }).where(eq(invitations.id, invitationId));

```

artifacts/api-server/src/routes/invitations.ts (continued)

```
281
282   res.json({ message: "Invitation declined" });
283 });
284
285 export default router;
286
```

artifacts/api-server/src/routes/contributions.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, contributions, profiles, milestones } from "@workspace/db";
4  import { eq, and, desc, sql, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7  import { createNotification, notifyAllMembers } from "../lib/notifications.js";
8
9  const router = Router();
10
11  // GET /jars/:jarId/contributions
12  router.get("/jars/:jarId/contributions", requireAuth, async (req, res) => {
13    const userId = (req as AuthenticatedRequest).userId;
14    const { jarId } = req.params as { jarId: string };
15    const { memberId } = req.query as { memberId?: string };
16
17    const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
18    if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
19
20    const userMember = await db
21      .select()
22      .from(jarMembers)
23      .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status,
24        "active"))).limit(1);
25
26    if (jar[0].organizerId !== userId && !userMember[0]) {
27      res.status(403).json({ error: "Forbidden", message: "Access denied" });
28      return;
29    }
30
31    let query = db
32      .select()
33      .from(contributions)
34      .where(
35        and(
36          eq(contributions.jarId, jarId),
37          memberId ? eq(contributions.memberId, memberId) : undefined,
38          inArray(contributions.status, ["completed", "simulated", "reversed"]),
39        ),
40      )
41      .orderBy(desc(contributions.createdAt)) as any;
42
43    const contribs = await query;
44
45    // Enrich with member names
46    const memberIds = [...new Set(contribs.map((c: any) => c.memberId))];
47    const members = memberIds.length > 0
48      ? await db.select().from(jarMembers).where(inArray(jarMembers.id, memberIds as string[]))
49      : [];
50    const memberUserIds = [...new Set(members.map((m) => m.userId))];
```

```

51   const memberProfiles = memberUserIds.length > 0
52     ? await db.select().from(profiles).where(inArray(profiles.userId, memberUserIds))
53     : [];
54
55   const profileByMemberId = new Map(
56     members.map((m) => [m.id, memberProfiles.find((p) => p.userId === m.userId)]),
57   );
58
59   const result = contribs.map((c: any) => ({
60     id: c.id,
61     jarId: c.jarId,
62     memberId: c.memberId,
63     amountCents: c.amountCents,
64     contributionDate: c.contributionDate,
65     status: c.status,
66     sourceType: c.sourceType,
67     milestoneId: c.milestoneId,
68     note: c.note,
69     memberName: profileByMemberId.get(c.memberId)?.displayName ?? null,
70     createdAt: c.createdAt,
71   }));
72
73   res.json(result);
74 });
75
76 // POST /jars/:jarId/contributions
77 router.post("/jars/:jarId/contributions", requireAuth, async (req, res) => {
78   const userId = (req as AuthenticatedRequest).userId;
79   const { jarId } = req.params as { jarId: string };
80
81   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
82   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
83
84   if (!["Saving", "CommitmentPending"].includes(jar[0].status)) {
85     res.status(400).json({ error: "BadRequest", message: "Contributions can only be added to active
86 jars" });
87   }
88
89   const member = await db
90     .select()
91     .from(jarMembers)
92     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status,
93 "Active")))
94     .limit(1);
95
96   if (!member[0]) {
97     res.status(403).json({ error: "Forbidden", message: "You are not an active member of this jar" });
98     return;
99   }
100   const { amountCents, contributionDate, milestoneId, note } = req.body as {
101     amountCents?: number;
102     contributionDate?: string;
103     milestoneId?: string;
104     note?: string;
105   };
106
107   if (!amountCents || amountCents < 1) {
108     res.status(400).json({ error: "BadRequest", message: "Amount must be at least $0.01" });
109     return;

```

```

110   }
111
112   // Validate milestone belongs to jar
113   if (milestoneId) {
114     const ms = await db.select().from(milestones).where(and(eq(milestones.id, milestoneId),
115     if (!ms[0]) {
116       res.status(400).json({ error: "BadRequest", message: "Milestone not found in this jar" });
117       return;
118     }
119   }
120
121   const today = new Date().toISOString().split("T")[0];
122   const [contribution] = await db.insert(contributions).values({
123     jarId,
124     memberId: member[0].id,
125     amountCents,
126     contributionDate: contributionDate ?? today,
127     status: "simulated",
128     sourceType: "manual",
129     milestoneId: milestoneId ?? null,
130     note: note ?? null,
131   }).returning();
132
133   if (!contribution) { res.status(500).json({ error: "InternalError", message: "Failed to record
134   contribution" }); ...
135   await logActivity({
136     jarId,
137     userId,
138     eventType: "contribution_added",
139     description: `A contribution of ${$(amountCents / 100).toFixed(2)} was added`,
140     amountCents,
141   });
142
143   // Check if jar is now fully funded
144   const totalResult = await db
145     .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
146     .from(contributions)
147     .where(and(eq(contributions.jarId, jarId), inArray(contributions.status, ["completed",
148     "simulated"])));
149   const totalSaved = Number(totalResult[0]?.total ?? 0);
150   if (totalSaved >= jar[0].goalAmountCents) {
151     await db.update(jars).set({ status: "FullyFunded", updatedAt: new Date() }).where(eq(jars.id,
152     jarId));
153     const allMembers = await db.select({ userId: jarMembers.userId }).from(jarMembers)
154       .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
155     await notifyAllMembers({
156       jarId,
157       memberUserIds: allMembers.map((m) => m.userId),
158       type: "goal_fully_funded",
159       title: `${jar[0].name} is fully funded!`,
160       message: "Congratulations! Your group has reached the savings goal.",
161     });
162   }
163   // Notify jar members
164   const allMembers = await db.select({ userId: jarMembers.userId }).from(jarMembers)
165     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
166   const prof = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
167
168   await notifyAllMembers({

```

```

169     jarId,
170     memberUserIds: allMembers.map((m) => m.userId),
171     type: "contribution_recorded",
172     title: "New contribution!",
173     message: `${prof[0]?.displayName ?? "A member"} added $$${(amountCents / 100).toFixed(2)} to
174     ${jar[0].name} ${jar[0].id}
175     excludeUserId: userId,
176   });
177   res.status(201).json({
178     id: contribution.id,
179     jarId: contribution.jarId,
180     memberId: contribution.memberId,
181     amountCents: contribution.amountCents,
182     contributionDate: contribution.contributionDate,
183     status: contribution.status,
184     sourceType: contribution.sourceType,
185     milestoneId: contribution.milestoneId,
186     note: contribution.note,
187     memberName: prof[0]?.displayName ?? null,
188     createdAt: contribution.createdAt,
189   });
190 });
191
192 // POST /jars/:jarId/contributions/:contributionId/reverse
193 router.post("/jars/:jarId/contributions/:contributionId/reverse", requireAuth, async (req, res) => {
194   const userId = (req as AuthenticatedRequest).userId;
195   const { jarId, contributionId } = req.params as { jarId: string; contributionId: string };
196
197   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
198   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
199
200   const [contrib] = await db.select().from(contributions).where(and(eq(contributions.id,
201   if (!contrib) { res.status(404).json({ error: "NotFound", message: "Contribution not found" });
202   return; }
203
204   // Only organizer or the contributing member can reverse
205   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.id, contrib.memberId),
206   if (jar[0].organizerId !== userId && !member[0]) {
207     res.status(403).json({ error: "Forbidden", message: "Only the organizer or contributing member can
208     return;
209   }
210
211   if (contrib.status === "reversed") {
212     res.status(400).json({ error: "BadRequest", message: "Contribution is already reversed" });
213     return;
214   }
215
216   const { reason } = req.body as { reason?: string };
217   const [updated] = await db.update(contributions)
218     .set({ status: "reversed", reversedAt: new Date(), reversalNote: reason ?? null })
219     .where(eq(contributions.id, contributionId))
220     .returning();
221
222   if (!updated) { res.status(500).json({ error: "InternalServerError", message: "Failed to reverse
223   contribution" }); return;
224
225   await logActivity({
226     jarId,
227     userId,
228     eventType: "contribution_reversed",
229     description: `A contribution of $$${(contrib.amountCents / 100).toFixed(2)} was reversed`,

```

artifacts/api-server/src/routes/contributions.ts (continued)

```
228     amountCents: contrib.amountCents,
229   });
230
231   res.json({ ...updated, memberName: null });
232 });
233
234 export default router;
235
```

artifacts/api-server/src/routes/milestones.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, milestones, contributions } from "@workspace/db";
4  import { eq, and, sql, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7
8  const router = Router();
9
10 async function checkJarAccess(jarId: string, userId: string) {
11   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
12   if (!jar[0]) return { jar: null, isOrganizer: false, isMember: false };
13   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
14     eq(jarMembers.userId, userId), isOrganizer: jar[0].organizerId === userId, isMember: !!member[0] });
15 }
16
17 // GET /jars/:jarId/milestones
18 router.get("/jars/:jarId/milestones", requireAuth, async (req, res) => {
19   const userId = (req as AuthenticatedRequest).userId;
20   const { jarId } = req.params as { jarId: string };
21
22   const { jar, isOrganizer, isMember } = await checkJarAccess(jarId, userId);
23   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
24   if (!isOrganizer && !isMember) { res.status(403).json({ error: "Forbidden", message: "Access
25     forbidden" }); return; }
26
27   const allMilestones = await db.select().from(milestones).where(eq(milestones.jarId, jarId));
28
29   // Calculate allocated amounts from contributions
30   const result = await Promise.all(
31     allMilestones.map(async (ms) => {
32       const allocated = await db
33         .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
34         .from(contributions)
35         .where(
36           and(
37             eq(contributions.milestoneId, ms.id),
38             inArray(contributions.status, ["completed", "simulated"]),
39           );
40       const allocatedAmountCents = Number(allocated[0]?.total ?? 0);
41       const percentFunded = ms.targetAmountCents > 0
42         ? Math.min(100, Math.round((allocatedAmountCents / ms.targetAmountCents) * 1000) / 10)
43         : 0;
44
45       return {
46         id: ms.id,
47         jarId: ms.jarId,
48         name: ms.name,
```



```

49         description: ms.description,
50         targetAmountCents: ms.targetAmountCents,
51         allocatedAmountCents,
52         percentFunded,
53         dueDate: ms.dueDate,
54         priority: ms.priority,
55         status: ms.status,
56         createdAt: ms.createdAt,
57     };
58     }),
59 );
60
61 res.json(result.sort((a, b) => a.priority - b.priority));
62 });
63
64 // POST /jars/:jarId/milestones
65 router.post("/jars/:jarId/milestones", requireAuth, async (req, res) => {
66     const userId = (req as AuthenticatedRequest).userId;
67     const { jarId } = req.params as { jarId: string };
68
69     const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
70     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
71     if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can manage
72 milestones" });...
73     const { name, description, targetAmountCents, dueDate, priority = 0 } = req.body as {
74         name?: string;
75         description?: string;
76         targetAmountCents?: number;
77         dueDate?: string;
78         priority?: number;
79     };
80
81     if (!name || !targetAmountCents) {
82         res.status(400).json({ error: "BadRequest", message: "name and targetAmountCents are required" });
83         return;
84     }
85
86     const [milestone] = await db.insert(milestones).values({
87         jarId,
88         name,
89         description: description ?? null,
90         targetAmountCents,
91         dueDate: dueDate ?? null,
92         priority,
93         status: "pending",
94     }).returning();
95
96     if (!milestone) { res.status(500).json({ error: "InternalServerError", message: "Failed to create
97 ..... " });
98     await logActivity({ jarId, userId, eventType: "milestone_created", description: `Milestone "${name}"
99 .....`
100     res.status(201).json({ ...milestone, allocatedAmountCents: 0, percentFunded: 0 });
101 });
102
103 // PATCH /jars/:jarId/milestones/:milestoneId
104 router.patch("/jars/:jarId/milestones/:milestoneId", requireAuth, async (req, res) => {
105     const userId = (req as AuthenticatedRequest).userId;
106     const { jarId, milestoneId } = req.params as { jarId: string; milestoneId: string };
107

```

artifacts/api-server/src/routes/milestones.ts (continued)

```
108   const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
109   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
110   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can manage
111   ...}); return; }
112   const { name, description, targetAmountCents, dueDate, priority, status } = req.body as {
113     name?: string; description?: string; targetAmountCents?: number; dueDate?: string; priority?:
114     number; status?: ...
115   };
116   const updates: Partial<typeof milestones.$inferSelect> = { updatedAt: new Date() };
117   if (name !== undefined) updates.name = name;
118   if (description !== undefined) updates.description = description;
119   if (targetAmountCents !== undefined) updates.targetAmountCents = targetAmountCents;
120   if (dueDate !== undefined) updates.dueDate = dueDate;
121   if (priority !== undefined) updates.priority = priority;
122   if (status !== undefined) updates.status = status;
123
124   const [updated] = await db.update(milestones).set(updates).where(and(eq(milestones.id, milestoneId),
125   ...));
126   if (!updated) { res.status(404).json({ error: "NotFound", message: "Milestone not found" }); return; }
127   res.json({ ...updated, allocatedAmountCents: 0, percentFunded: 0 });
128 }));
129
130 // DELETE /jars/:jarId/milestones/:milestoneId
131 router.delete("/jars/:jarId/milestones/:milestoneId", requireAuth, async (req, res) => {
132   const userId = (req as AuthenticatedRequest).userId;
133   const { jarId, milestoneId } = req.params as { jarId: string; milestoneId: string };
134
135   const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
136   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
137   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can delete
138   milestones" });...
139   await db.delete(milestones).where(and(eq(milestones.id, milestoneId), eq(milestones.jarId, jarId)));
140   res.json({ message: "Milestone deleted" });
141 });
142
143 export default router;
144
```

artifacts/api-server/src/routes/schedules.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, contributionSchedules } from "@workspace/db";
4  import { eq, and } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7  const router = Router();
8
9  async function getMyMembership(jarId: string, userId: string) {
10   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
11   if (!jar[0]) return { jar: null, member: null };
12   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
13   ...));
14   return { jar: jar[0], member: member[0] ?? null };
15 }
16
17 // GET /jars/:jarId/schedule
18 router.get("/jars/:jarId/schedule", requireAuth, async (req, res) => {
19   const userId = (req as AuthenticatedRequest).userId;
20   const { jarId } = req.params as { jarId: string };

```

```

20
21   const { jar, member } = await getMyMembership(jarId, userId);
22   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
23   if (!member && jar.organizerId !== userId) { res.status(403).json({ error: "Forbidden", message:
24     "Access denied" ...
25   if (!member) { res.json(null); return; }
26
27   const schedule = await db.select().from(contributionSchedules)
28     .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id),
29     .limit(1);
30
31   if (!schedule[0]) { res.json(null); return; }
32
33   res.json({
34     id: schedule[0].id,
35     jarId: schedule[0].jarId,
36     memberId: schedule[0].memberId,
37     frequency: schedule[0].frequency,
38     amountCents: schedule[0].amountCents,
39     startDate: schedule[0].startDate,
40     preferredDay: schedule[0].preferredDay,
41     endCondition: schedule[0].endCondition,
42     isPaused: schedule[0].isPaused,
43     isActive: schedule[0].isActive,
44     projectedTotalCents: null,
45     estimatedCompletionDate: null,
46     createdAt: schedule[0].createdAt,
47   });
48   });
49
50   // POST /jars/:jarId/schedule
51   router.post("/jars/:jarId/schedule", requireAuth, async (req, res) => {
52     const userId = (req as AuthenticatedRequest).userId;
53     const { jarId } = req.params as { jarId: string };
54
55     const { jar, member } = await getMyMembership(jarId, userId);
56     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
57     if (!member) { res.status(403).json({ error: "Forbidden", message: "You are not a member of this
58     jar" }); return; ...
59     const { frequency, amountCents, startDate, preferredDay, endCondition = "targetDate" } = req.body as {
60       frequency?: string; amountCents?: number; startDate?: string; preferredDay?: number; endCondition?:
61     };
62
63     if (!frequency || !amountCents || !startDate) {
64       res.status(400).json({ error: "BadRequest", message: "frequency, amountCents, and startDate are
65       return;
66     }
67
68     // Deactivate existing schedules
69     await db.update(contributionSchedules)
70       .set({ isActive: false })
71       .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id)));
72
73     const [schedule] = await db.insert(contributionSchedules).values({
74       jarId,
75       memberId: member.id,
76       frequency,
77       amountCents,
78       startDate,

```

artifacts/api-server/src/routes/schedules.ts (continued)

```
79     preferredDay: preferredDay ?? null,
80     endCondition,
81     isPaused: false,
82     isActive: true,
83   }).returning();
84
85   if (!schedule) { res.status(500).json({ error: "InternalServerError", message: "Failed to create
86     schedule" }); return; ...
87   res.status(201).json({ ...schedule, projectedTotalCents: null, estimatedCompletionDate: null });
88   });
89
90   // PATCH /jars/:jarId/schedule
91   router.patch("/jars/:jarId/schedule", requireAuth, async (req, res) => {
92     const userId = (req as AuthenticatedRequest).userId;
93     const { jarId } = req.params as { jarId: string };
94
95     const { jar, member } = await getMyMembership(jarId, userId);
96     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
97     if (!member) { res.status(403).json({ error: "Forbidden", message: "Access denied" }); return; }
98
99     const { frequency, amountCents, preferredDay, isPaused } = req.body as {
100       frequency?: string; amountCents?: number; preferredDay?: number; isPaused?: boolean;
101     };
102
103     const updates: Partial<typeof contributionSchedules.$inferSelect> = { updatedAt: new Date() };
104     if (frequency !== undefined) updates.frequency = frequency;
105     if (amountCents !== undefined) updates.amountCents = amountCents;
106     if (preferredDay !== undefined) updates.preferredDay = preferredDay;
107     if (isPaused !== undefined) updates.isPaused = isPaused;
108
109     const [updated] = await db.update(contributionSchedules)
110       .set(updates)
111       .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id),
112         eq(contributionSchedules.isActive, true)));
113
114     if (!updated) { res.status(404).json({ error: "NotFound", message: "No active schedule found" });
115       ...
116     res.json({ ...updated, projectedTotalCents: null, estimatedCompletionDate: null });
117   });
118
119   export default router;
120
```

artifacts/api-server/src/routes/reminders.ts

```
1  /**
2   * POST /internal/process-reminders
3   *
4   * Scans all active contribution schedules and creates in-app notifications for:
5   * - Contributions due today
6   * - Contributions that were due yesterday (missed)
7   *
8   * This endpoint is meant to be called by a scheduled job (cron / Replit scheduled deployment).
9   * It is protected by a shared secret via the X-Internal-Token header.
10  */
11
12  import { Router } from "express";
13  import { db } from "@workspace/db";
14  import {
```

```

15   contributionSchedules,
16   jarMembers,
17   jars,
18   users,
19   contributions,
20   notifications,
21 } from "@workspace/db";
22 import { eq, and, gte, lte, inArray } from "drizzle-orm";
23 import { computeNextDueDate, computePreviousDueDate, toISODate } from "../lib/schedule-utils.js";
24
25 const router = Router();
26
27 function requireInternalToken(req: import("express").Request, res: import("express").Response, next:
28 import("express").NextFunction) {
29   const configuredToken = process.env["INTERNAL_REMINDER_TOKEN"];
30   if (!configuredToken) {
31     // Fail closed: if the secret is not configured, refuse all requests
32     res.status(503).json({ error: "ServiceUnavailable", message: "INTERNAL_REMINDER_TOKEN is not
33     configured" });
34   }
35   const token = req.headers["x-internal-token"];
36   if (token !== configuredToken) {
37     res.status(403).json({ error: "Forbidden", message: "Invalid internal token" });
38     return;
39   }
40   next();
41 }
42
43 // POST /internal/process-reminders
44 router.post("/internal/process-reminders", requireInternalToken, async (_req, res) => {
45   const today = new Date();
46   const todayStr = toISODate(today);
47
48   const yesterday = new Date(today);
49   yesterday.setDate(yesterday.getDate() - 1);
50   const yesterdayStr = toISODate(yesterday);
51
52   // Load all active, non-paused schedules
53   const activeSchedules = await db
54     .select()
55     .from(contributionSchedules)
56     .where(and(eq(contributionSchedules.isActive, true), eq(contributionSchedules.isPaused, false)));
57
58   let notificationsCreated = 0;
59   let missedCount = 0;
60
61   for (const schedule of activeSchedules) {
62     const nextDue = computeNextDueDate(schedule, today);
63     const prevDue = computePreviousDueDate(schedule, today);
64
65     // Resolve jar member !' user
66     const [member] = await db
67       .select()
68       .from(jarMembers)
69       .where(eq(jarMembers.id, schedule.memberId))
70       .limit(1);
71
72     if (!member) continue;
73
74     const [jar] = await db

```

artifacts/api-server/src/routes/reminders.ts (continued)

```

133      // Check if a contribution was made on or within 1 day of the due date
134      const windowStart = new Date(prevDue);
135      windowStart.setDate(windowStart.getDate() - 1);
136      const windowEnd = new Date(prevDue);
137      windowEnd.setDate(windowEnd.getDate() + 1);
138
139      const windowStartStr = toISODate(windowStart);
140      const windowEndStr = toISODate(windowEnd);
141
142      // Count contributions in the window
143      const contribsInWindow = await db
144        .select({ id: contributions.id })
145        .from(contributions)
146        .where(
147          and(
148            eq(contributions.memberId, schedule.memberId),
149            eq(contributions.jarId, schedule.jarId),
150            gte(contributions.contributionDate, windowStartStr),
151            lte(contributions.contributionDate, windowEndStr),
152            inArray(contributions.status, ["completed", "simulated"]),
153          ),
154        )
155        .limit(1);
156
157      if (!contribsInWindow[0]) {
158        // Missed! Create notification (deduplicate)
159        const cutoff = new Date(today);
160        cutoff.setHours(cutoff.getHours() - 20);
161
162        const recentMissed = await db
163          .select({ id: notifications.id })
164          .from(notifications)
165          .where(
166            and(
167              eq(notifications.userId, member.userId),
168              eq(notifications.type, "contribution_missed"),
169              eq(notifications.relatedJarId, jar.id),
170              gte(notifications.createdAt, cutoff),
171            ),
172          )
173          .limit(1);
174
175        if (!recentMissed[0]) {
176          await db.insert(notifications).values({
177            userId: member.userId,
178            type: "contribution_missed",
179            title: "Missed Contribution",
180            message: `You missed your ${formattedAmount} contribution to ${jar.name} yesterday. Add
181 funds to stay on track`,
182            isRead: false,
183            relatedJarId: jar.id,
184            actionUrl: `/jar/${jar.id}`,
185          });
186          notificationsCreated++;
187          missedCount++;
188        }
189      }
190    }
191  }

```

artifacts/api-server/src/routes/reminders.ts (continued)

```
192   res.json({
193     processed: activeSchedules.length,
194     notificationsCreated,
195     missedCount,
196     runAt: today.toISOString(),
197   });
198 });
199
200 export default router;
201
```

artifacts/api-server/src/routes/commitments.ts

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, commitmentRequests, commitmentVotes, profiles } from "@workspace/db";
4  import { eq, and, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7  import { notifyAllMembers } from "../lib/notifications.js";
8
9  const router = Router();
10
11  async function buildCommitmentWithVotes(cr: typeof commitmentRequests.$inferSelect) {
12    const votes = await db.select().from(commitmentVotes).where(eq(commitmentVotes.commitmentRequestId,
13    cr.id));
14    const memberIds = [...new Set(votes.map((v) => v.memberId))];
15    const members = memberIds.length > 0
16      ? await db.select().from(jarMembers).where(inArray(jarMembers.id, memberIds))
17      : [];
18    const profileMap = new Map<string, string>();
19    if (members.length > 0) {
20      const profs = await db.select().from(profiles).where(inArray(profiles.userId, members.map((m) =>
21      m.userId)));
22      for (const p of profs) {
23        const m = members.find((mem) => mem.userId === p.userId);
24        if (m) profileMap.set(m.id, p.displayName);
25      }
26    }
27
28    const approvedCount = votes.filter((v) => v.vote === "approve").length;
29    const rejectedCount = votes.filter((v) => v.vote === "reject").length;
30    const pendingCount = votes.filter((v) => v.vote === "clarification").length;
31    const totalVotes = votes.length;
32    const approvalPercentage = totalVotes > 0 ? (approvedCount / totalVotes) * 100 : 0;
33
34    return {
35      id: cr.id,
36      jarId: cr.jarId,
37      milestoneId: cr.milestoneId,
38      amountCents: cr.amountCents,
39      purpose: cr.purpose,
40      vendorName: cr.vendorName,
41      paymentDeadline: cr.paymentDeadline,
42      requiredThreshold: Number(cr.requiredThreshold),
43      status: cr.status,
44      approvedCount,
45      rejectedCount,
46      pendingCount,
47      approvalPercentage,
48      votes: votes.map((v) => ({

```



```

47     id: v.id,
48     commitmentRequestId: v.commitmentRequestId,
49     memberId: v.memberId,
50     vote: v.vote,
51     note: v.note,
52     memberName: profileMap.get(v.memberId) ?? null,
53     createdAt: v.createdAt,
54   })),
55   createdAt: cr.createdAt,
56 };
57 }
58
59 // GET /jars/:jarId/commitments
60 router.get("/jars/:jarId/commitments", requireAuth, async (req, res) => {
61   const userId = (req as AuthenticatedRequest).userId;
62   const { jarId } = req.params as { jarId: string };
63
64   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
65   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
66
67   const member = await db.select().from(jarMembers).where(
68     eq(jarMembers.jarId, jarId) && eq(jarMembers.userId, userId) && !member[0]
69   );
70   if (!member[0]) { res.status(403).json({ error: "Forbidden", message: "Only the jar organizer can view commitments" }); return; }
71   const crs = await db.select().from(commitmentRequests).where(eq(commitmentRequests.jarId, jarId));
72   const result = await Promise.all(crs.map(buildCommitmentWithVotes));
73   res.json(result);
74 }
75
76 // POST /jars/:jarId/commitments
77 router.post("/jars/:jarId/commitments", requireAuth, async (req, res) => {
78   const userId = (req as AuthenticatedRequest).userId;
79   const { jarId } = req.params as { jarId: string };
80
81   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
82   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
83   if (jar[0].organizerId !== userId) { res.status(403).json({ error: "Forbidden", message: "Only the jar organizer can create commitments" }); return; }
84   const { milestoneId, amountCents, purpose, vendorName, paymentDeadline, requiredThreshold = 0.67, ...rest } = req.body as {
85     milestoneId?: string; amountCents?: number; purpose?: string; vendorName?: string; paymentDeadline?: string; requiredThreshold?: number;
86     [key: string]: any;
87   };
88   if (!amountCents || !purpose) {
89     res.status(400).json({ error: "BadRequest", message: "amountCents and purpose are required" });
90     return;
91   }
92
93   const [cr] = await db.insert(commitmentRequests).values({
94     jarId,
95     milestoneId: milestoneId ?? null,
96     amountCents,
97     purpose,
98     vendorName: vendorName ?? null,
99     paymentDeadline: paymentDeadline ?? null,
100     requiredThreshold: String(requiredThreshold),
101     status: "pending",
102     note: rest.note ?? null,
103     createdByUserId: userId,
104   }).returning();
105

```

```

106   if (!cr) { res.status(500).json({ error: "InternalError", message: "Failed to create commitment
107   ...
108   await db.update(jars).set({ status: "CommitmentPending", updatedAt: new Date() }).where(eq(jars.id,
109   await logActivity({ jarId, userId, eventType: "commitment_requested", description: `A commitment
110   request was subm...
111   const members = await db.select({ userId:
112   jarMembers.userId}).from(jarMembers).where(and(eq(jarMembers.jarId, ja...
113   jarId,
114   memberUserIds: members.map((m) => m.userId),
115   type: "commitment_requested",
116   title: "Commitment approval needed",
117   message: `Review and approve a ${$(amountCents / 100).toFixed(2)} commitment request for
118   ${jar[0].name},
119   ${jar[0].ownerId: userId,
120   });
121   res.status(201).json(await buildCommitmentWithVotes(cr));
122   });
123
124   // POST /jars/:jarId/commitments/:commitmentId/vote
125   router.post("/jars/:jarId/commitments/:commitmentId/vote", requireAuth, async (req, res) => {
126     const userId = (req as AuthenticatedRequest).userId;
127     const { jarId, commitmentId } = req.params as { jarId: string; commitmentId: string };
128
129     const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
130     if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
131
132     const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
133     if (!member[0]) { res.status(403).json({ error: "Forbidden", message: "Only members can vote" });
134     return; }
135     const [cr] = await db.select().from(commitmentRequests).where(eq(commitmentRequests.id,
136     commitmentId)).limit(1);
137     if (!cr || cr.jarId !== jarId) { res.status(404).json({ error: "NotFound", message: "Commitment
138     not found" });
139     if (cr.status !== "pending") { res.status(400).json({ error: "BadRequest", message: "This commitment
140     request is n...
141     const { vote, note } = req.body as { vote?: string; note?: string };
142     if (!vote || !["approve", "reject", "clarification"].includes(vote)) {
143       res.status(400).json({ error: "BadRequest", message: "vote must be approve, reject, or
144       clarification" });
145     }
146     // Upsert vote (replace existing)
147     const existingVote = await
148     if (existingVote[0]) {
149       await db.update(commitmentVotes).set({ vote, note: note ?? null }).where(eq(commitmentVotes.id,
150     }) else {
151       await db.insert(commitmentVotes).values({ commitmentRequestId: commitmentId, memberId: member[0].id,
152     })
153     await logActivity({ jarId, userId, eventType: "approval_submitted", description: `A member ${vote}d
154     the commitmen...
155     const updatedCr = await db.select().from(commitmentRequests).where(eq(commitmentRequests.id,
156     commitmentId)).limit(1);
157     res.json(await buildCommitmentWithVotes(updatedCr[0]));
158   });
159   export default router;
160

```

artifacts/api-server/src/routes/agreements.ts

```

1  import { Router } from "express";

```

```

2  import { db } from "@workspace/db";
3  import { jars, jarMembers, agreements, agreementAcceptances, profiles } from "@workspace/db";
4  import { eq, and, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7
8  const router = Router();
9
10 // GET /jars/:jarId/agreements
11 router.get("/jars/:jarId/agreements", requireAuth, async (req, res) => {
12   const userId = (req as AuthenticatedRequest).userId;
13   const { jarId } = req.params as { jarId: string };
14
15   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
16   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
17
18   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
19     if (jar[0].organizerId !== userId && !member[0]) { res.status(403).json({ error: "Forbidden", message:
20       "Access de...
21     const allAgreements = await db.select().from(agreements).where(eq(agreements.jarId, jarId));
22
23     const result = await Promise.all(
24       allAgreements.map(async (ag) => {
25         const acceptances = await
26         db.select().from(agreementAcceptances).where(eq(agreementAcceptances.agreementId, a...
27         const memberProfiles = userIds.length > 0
28           ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
29           : [];
30         const profileMap = new Map(memberProfiles.map((p) => [p.userId, p.displayName]));
31
32         const myAcceptance = acceptances.find((a) => a.userId === userId);
33
34         return {
35           id: ag.id,
36           jarId: ag.jarId,
37           version: ag.version,
38           content: ag.content,
39           createdAt: ag.createdAt,
40           effectiveDate: ag.effectiveDate,
41           acceptances: acceptances.map((a) => ({
42             id: a.id,
43             agreementId: a.agreementId,
44             userId: a.userId,
45             acceptedAt: a.acceptedAt,
46             memberName: profileMap.get(a.userId) ?? null,
47           })),
48           myAcceptance: myAcceptance
49           ? { id: myAcceptance.id, agreementId: myAcceptance.agreementId, userId: myAcceptance.userId,
50             acceptedAt: null,
51           };
52         },
53       );
54
55   res.json(result);
56 });
57
58 // POST /jars/:jarId/agreements/:agreementId/accept
59 router.post("/jars/:jarId/agreements/:agreementId/accept", requireAuth, async (req, res) => {
60   const userId = (req as AuthenticatedRequest).userId;

```

artifacts/api-server/src/routes/agreements.ts (continued)

```

61   const { jarId, agreementId } = req.params as { jarId: string; agreementId: string };
62
63   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
64   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
65
66   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
67   if (jar[0].organizerId !== userId && !member[0]) { res.status(403).json({ error: "Forbidden", message:
68   "Access de...
69   const ag = await db.select().from(agreements).where(and(eq(agreements.id, agreementId),
70   eq(agreements.jarId, jarId)); if (!ag[0]) { res.status(404).json({ error: "NotFound", message: "Agreement not found" }); return; }
71
72   // Check if already accepted
73   const existing = await
74   db.select().from(agreementAcceptances).where(and(eq(agreementAcceptances.agreementId, agre...
75   if (existing[0]) {
76     res.json({ id: existing[0].id, agreementId: existing[0].agreementId, userId: existing[0].userId,
77     acceptedAt: existing[0].acceptedAt; ex...
78   }
79
80   const [acceptance] = await db.insert(agreementAcceptances).values({
81     agreementId,
82     userId,
83     acceptedAt: new Date(),
84   }).returning();
85
86   if (!acceptance) { res.status(500).json({ error: "InternalError", message: "Failed to record
87   acceptance" }); retu...
88   await logActivity({ jarId, userId, eventType: "agreement_accepted", description: "A member accepted
89   the savings a...
90   res.json({ id: acceptance.id, agreementId: acceptance.agreementId, userId: acceptance.userId,
91   acceptedAt: accepta...
92
93   export default router;

```

artifacts/api-server/src/routes/notifications.ts

```

1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { notifications, jars } from "@workspace/db";
4  import { eq, and, desc } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7  const router = Router();
8
9  // GET /notifications
10 router.get("/notifications", requireAuth, async (req, res) => {
11   const userId = (req as AuthenticatedRequest).userId;
12   const { unreadOnly } = req.query as { unreadOnly?: string };
13
14   const whereClause = unreadOnly === "true"
15     ? and(eq(notifications.userId, userId), eq(notifications.isRead, false))
16     : eq(notifications.userId, userId);
17
18   const allNotifications = await db
19     .select()
20     .from(notifications)
21     .where(whereClause)
22     .orderBy(desc(notifications.createdAt))
23     .limit(50);

```

artifacts/api-server/src/routes/notifications.ts (continued)

```
24
25 // Enrich with jar names
26 const result = await Promise.all(
27   allNotifications.map(async (n) => {
28     let relatedJarName: string | null = null;
29     if (n.relatedJarId) {
30       const jar = await db.select({ name: jars.name }).from(jars).where(eq(jars.id,
31         n.relatedJarId));
32       relatedJarName = jar[0]?.name ?? null;
33     }
34     return {
35       id: n.id,
36       userId: n.userId,
37       type: n.type,
38       title: n.title,
39       message: n.message,
40       isRead: n.isRead,
41       relatedJarId: n.relatedJarId,
42       relatedJarName,
43       actionUrl: n.actionUrl,
44       createdAt: n.createdAt,
45     };
46   }),
47 );
48 res.json(result);
49 });
50
51 // PATCH /notifications/:notificationId/read
52 router.patch("/notifications/:notificationId/read", requireAuth, async (req, res) => {
53   const userId = (req as AuthenticatedRequest).userId;
54   const { notificationId } = req.params as { notificationId: string };
55
56   const [updated] = await db
57     .update(notifications)
58     .set({ isRead: true })
59     .where(and(eq(notifications.id, notificationId), eq(notifications.userId, userId)))
60     .returning();
61
62   if (!updated) { res.status(404).json({ error: "NotFound", message: "Notification not found" }); }
63   res.json({ ...updated, relatedJarName: null });
64 });
65
66 // POST /notifications/read-all
67 router.post("/notifications/read-all", requireAuth, async (req, res) => {
68   const userId = (req as AuthenticatedRequest).userId;
69   await db.update(notifications).set({ isRead: true }).where(eq(notifications.userId, userId));
70   res.json({ message: "All notifications marked as read" });
71 });
72
73 export default router;
74
```

artifacts/api-server/src/routes/activity.ts

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { activityEvents, jars, jarMembers, profiles } from "@workspace/db";
4 import { eq, and, desc, inArray } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
```

```

6
7   const router = Router();
8
9   // GET /jars/:jarId/activity
10  router.get("/jars/:jarId/activity", requireAuth, async (req, res) => {
11    const userId = (req as AuthenticatedRequest).userId;
12    const { jarId } = req.params as { jarId: string };
13    const limit = Math.min(50, parseInt((req.query.limit as string) ?? "20", 10));
14    const offset = parseInt((req.query.offset as string) ?? "0", 10);
15
16    const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
17    if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
18
19    const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
20    eq(jarMembers.userId, userId)));
21    if (!member[0]) { res.status(403).json({ error: "Forbidden", message: "Access denied" }); return; }
22    return;
23  }
24
25  const events = await db
26    .select()
27    .from(activityEvents)
28    .where(eq(activityEvents.jarId, jarId))
29    .orderBy(desc(activityEvents.createdAt))
30    .limit(limit)
31    .offset(offset);
32
33  const userIds = [...new Set(events.filter((e) => e.userId).map((e) => e.userId!))];
34  const actorProfiles = userIds.length > 0
35    ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
36    : [];
37  const profileMap = new Map(actorProfiles.map((p) => [p.userId, p]));
38
39  const result = events.map((e) => {
40    const prof = e.userId ? profileMap.get(e.userId) : null;
41    return {
42      id: e.id,
43      jarId: e.jarId,
44      userId: e.userId,
45      eventType: e.eventType,
46      description: e.description,
47      actorName: prof?.displayName ?? null,
48      actorAvatarUrl: prof?.avatarUrl ?? null,
49      amountCents: e.amountCents,
50      metadata: e.metadata,
51      createdAt: e.createdAt,
52    };
53  });
54
55  res.json(result);
56  });
57
58  // GET /activity
59  router.get("/activity", requireAuth, async (req, res) => {
60    const userId = (req as AuthenticatedRequest).userId;
61
62    const memberships = await db
63      .select({ jarId: jarMembers.jarId })
64      .from(jarMembers)

```

artifacts/api-server/src/routes/activity.ts (continued)

```
65     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active")));
66
67     if (memberships.length === 0) {
68         res.json([]);
69         return;
70     }
71
72     const jarIds = memberships.map((m) => m.jarId);
73     const events = await db
74         .select()
75         .from(activityEvents)
76         .where(inArray(activityEvents.jarId, jarIds))
77         .orderBy(desc(activityEvents.createdAt))
78         .limit(30);
79
80     const userIds = [...new Set(events.filter((e) => e.userId).map((e) => e.userId!))];
81     const actorProfiles = userIds.length > 0
82         ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
83         : [];
84     const profileMap = new Map(actorProfiles.map((p) => [p.userId, p]));
85
86     // Get jar names for context
87     const allJars = await db.select({ id: jars.id, name: jars.name }).from(jars).where(inArray(jars.id,
88     jarIds));
89     const jarNameMap = new Map(allJars.map((j) => [j.id, j.name]));
90
91     const result = events.map((e) => {
92         const prof = e.userId ? profileMap.get(e.userId) : null;
93         return {
94             id: e.id,
95             jarId: e.jarId,
96             userId: e.userId,
97             eventType: e.eventType,
98             description: e.description,
99             actorName: prof?.displayName ?? null,
100             actorAvatarUrl: prof?.avatarUrl ?? null,
101             amountCents: e.amountCents,
102             metadata: { ...((e.metadata as Record<string, unknown>) ?? {}), jarName: jarNameMap.get(e.jarId) },
103             createdAt: e.createdAt,
104         };
105     });
106     res.json(result);
107 });
108
109 export default router;
110
```

artifacts/api-server/src/routes/health.ts

```
1  import { Router, type IRouter } from "express";
2
3  const router: IRouter = Router();
4
5  router.get("/healthz", (_req, res) => {
6      res.json({ status: "ok" });
7  });
8
9  export default router;
10
```

Mobile App — Contexts

artifacts/mobile/context/auth-context.tsx

```
1  import React, { createContext, useContext, useEffect, useState } from 'react';
2  import AsyncStorage from '@react-native-async-storage/async-storage';
3  import { setBaseUrl, setAuthTokenGetter } from '@workspace/api-client-react';
4  import { useGetMe, useLogin, useRegister, useLogout } from '@workspace/api-client-react';
5  import type { User, Profile, LoginRequest, RegisterRequest } from '@workspace/api-client-react/src/
6  generated/api.sc...
7  interface AuthContextType {
8    user: User | null;
9    profile: Profile | null;
10   token: string | null;
11   isAuthenticated: boolean;
12   isLoading: boolean;
13   login: (credentials: LoginRequest) => Promise<void>;
14   register: (data: RegisterRequest) => Promise<void>;
15   logout: () => Promise<void>;
16   refresh: () => Promise<void>;
17 }
18
19 const AuthContext = createContext<AuthContextType | null>(null);
20
21 export function AuthProvider({ children }: { children: React.ReactNode }) {
22   const [token, setToken] = useState<string | null>(null);
23   const [isInitializing, setIsInitializing] = useState(true);
24
25   const { data: meData, isLoading: isMeLoading, refetch } = useGetMe({
26     query: {
27       enabled: !!token,
28       retry: false,
29     }
30   });
31
32   const { mutateAsync: loginMutation } = useLogin();
33   const { mutateAsync: registerMutation } = useRegister();
34   const { mutateAsync: logoutMutation } = useLogout();
35
36   useEffect(() => {
37     async function loadToken() {
38       try {
39         const storedToken = await AsyncStorage.getItem('auth_token');
40         if (storedToken) {
41           setToken(storedToken);
42           setAuthTokenGetter(() => storedToken);
43         }
44       } catch (e) {
45         console.error('Failed to load token', e);
46       } finally {
47         setIsInitializing(false);
48       }
49     }
50     loadToken();
51   }, []);
52 }
```


artifacts/mobile/contexts/auth-context.tsx (continued)

```
53   const login = async (credentials: LoginRequest) => {
54     const data = await loginMutation({ data: credentials });
55     const newToken = data.token;
56     await AsyncStorage.setItem('auth_token', newToken);
57     setAuthTokenGetter(() => newToken);
58     setToken(newToken);
59   };
60
61   const register = async (data: RegisterRequest) => {
62     const responseData = await registerMutation({ data });
63     const newToken = responseData.token;
64     await AsyncStorage.setItem('auth_token', newToken);
65     setAuthTokenGetter(() => newToken);
66     setToken(newToken);
67   };
68
69   const logoutAction = async () => {
70     try {
71       await logoutMutation();
72     } catch (e) {
73       // Ignore
74     }
75     await AsyncStorage.removeItem('auth_token');
76     setAuthTokenGetter(() => null);
77     setToken(null);
78   };
79
80   const value = {
81     user: meData?.user || null,
82     profile: meData?.profile || null,
83     token,
84     isAuthenticated: !!token && !!meData?.user,
85     isLoading: isInitializing || (!!token && isMeLoading),
86     login,
87     register,
88     logout: logoutAction,
89     refresh: async () => { await refetch(); }
90   };
91
92   return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
93 }
94
95 export function useAuth() {
96   const context = useContext(AuthContext);
97   if (!context) {
98     throw new Error('useAuth must be used within an AuthProvider');
99   }
100   return context;
101 }
102
```

artifacts/mobile/contexts/create-jar-context.tsx

```
1  import React, { createContext, useContext, useState } from 'react';
2  import type { CreateJarRequest } from '@workspace/api-client-react/src/generated/api.schemas';
3
4  type CreateJarState = Partial<CreateJarRequest> & {
5    milestones?: { name: string; targetAmountCents: number }[];
6    invitees?: { email: string; contributionTargetCents?: number }[];
```

```
7   contributionPlan?: { frequency: any; amountCents: number };
8   };
9
10  interface CreateJarContextType {
11    state: CreateJarState;
12    updateState: (updates: Partial<CreateJarState>) => void;
13    resetState: () => void;
14  }
15
16  const CreateJarContext = createContext<CreateJarContextType | null>(null);
17
18  export function CreateJarProvider({ children }: { children: React.ReactNode }) {
19    const [state, setState] = useState<CreateJarState>({});
20
21    const updateState = (updates: Partial<CreateJarState>) => {
22      setState((prev) => ({ ...prev, ...updates }));
23    };
24
25    const resetState = () => setState({});
26
27    return (
28      <CreateJarContext.Provider value={{ state, updateState, resetState }}>
29        {children}
30      </CreateJarContext.Provider>
31    );
32  }
33
34  export function useCreateJarContext() {
35    const context = useContext(CreateJarContext);
36    if (!context) {
37      throw new Error('useCreateJarContext must be used within a CreateJarProvider');
38    }
39    return context;
40  }
41
```

Mobile App — Auth Screens

artifacts/mobile/app/(auth)/index.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { ImageBackground } from 'expo-image';
4  import { LinearGradient } from 'expo-linear-gradient';
5  import { useRouter } from 'expo-router';
6  import { useColors } from '@/hooks/useColors';
7  import { Feather } from '@expo/vector-icons';
8  import * as Haptics from 'expo-haptics';
9
10 export default function WelcomeScreen() {
11   const router = useRouter();
12   const colors = useColors();
13
14   return (
15     <View style={styles.container}>
16       <ImageBackground
17         source={require('../assets/images/travel-hero.jpg')}
18         style={StyleSheet.absoluteFill}
19         contentFit="cover"
20       >
21         <LinearGradient
22           colors={['transparent', 'rgba(0,0,0,0.8)']}
23           style={StyleSheet.absoluteFill}
24         />
25
26         <View style={styles.content}>
27           <View style={styles.logoContainer}>
28             <View style={styles.iconRow}>
29               <Feather name="archive" size={48} color="#FFFFFF" />
30               <Feather name="send" size={32} color="#FFFFFF" style={{ marginLeft: -12, marginTop:
31               ~~~~~ </View>
32             <Text style={styles.brandName}>TripJar</Text>
33             <Text style={styles.tagline}>Save Together. Travel Together.</Text>
34           </View>
35
36           <View style={styles.actionContainer}>
37             <Pressable
38               style={({ pressed }) => [
39               ~~~~~ { backgroundColor: colors.primary },
40               pressed && { opacity: 0.9, transform: [{ scale: 0.98 }] },
41             ]}
42             onPress={() => {
43               Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
44               router.push('/(auth)/register');
45             }}
46           >
47             <Text style={styles.primaryButtonText}>Get Started</Text>
48           </Pressable>
49
50           <Pressable
51             style={({ pressed }) => [
```

```

53         styles.secondaryButton,
54         pressed && { opacity: 0.7 },
55     ]}
56     onPress={() => router.push('/(auth)/login')}
57     >
58     <Text style={styles.secondaryButtonText}>Sign In</Text>
59   </Pressable>
60 </View>
61 </View>
62 </ImageBackground>
63 </View>
64 );
65 }
66
67 const styles = StyleSheet.create({
68   container: {
69     flex: 1,
70   },
71   content: {
72     flex: 1,
73     padding: 24,
74     justifyContent: 'space-between',
75     paddingBottom: 48,
76   },
77   logoContainer: {
78     flex: 1,
79     justifyContent: 'center',
80     alignItems: 'center',
81   },
82   iconRow: {
83     flexDirection: 'row',
84     alignItems: 'center',
85     marginBottom: 16,
86   },
87   brandName: {
88     fontSize: 48,
89     fontWeight: 'bold',
90     color: 'FFFFFF',
91     letterSpacing: -1,
92   },
93   tagline: {
94     fontSize: 18,
95     color: 'E8F6EF',
96     marginTop: 8,
97     fontWeight: '500',
98   },
99   actionContainer: {
100     width: '100%',
101     gap: 16,
102   },
103   primaryButton: {
104     width: '100%',
105     height: 56,
106     borderRadius: 12,
107     justifyContent: 'center',
108     alignItems: 'center',
109   },
110   primaryButtonText: {
111     color: 'FFFFFF',

```

artifacts/mobile/app/(auth)/index.tsx (continued)

```
112     fontSize: 18,  
113     fontWeight: 'bold',  
114   },  
115   secondaryButton: {  
116     width: '100%',  
117     height: 56,  
118     justifyContent: 'center',  
119     alignItems: 'center',  
120   },  
121   secondaryButtonText: {  
122     color: '#FFFFFF',  
123     fontSize: 16,  
124     fontWeight: '600',  
125   },  
126 });  
127
```

artifacts/mobile/app/(auth)/login.tsx

```
1  import React, { useState } from 'react';  
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform,  
3  import { useRouter } from 'expo-router';  
4  import { useColors } from '@/hooks/useColors';  
5  import { Feather } from '@expo/vector-icons';  
6  import { useAuth } from '@/contexts/auth-context';  
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';  
8  
9  export default function LoginScreen() {  
10    const colors = useColors();  
11    const router = useRouter();  
12    const insets = useSafeAreaInsets();  
13    const { login } = useAuth();  
14  
15    const [email, setEmail] = useState('');  
16    const [password, setPassword] = useState('');  
17    const [showPassword, setShowPassword] = useState(false);  
18    const [error, setError] = useState('');  
19    const [isLoading, setIsLoading] = useState(false);  
20  
21    const handleLogin = async () => {  
22      if (!email || !password) {  
23        setError('Please enter both email and password');  
24        return;  
25      }  
26      try {  
27        setIsLoading(true);  
28        setError('');  
29        await login({ email, password });  
30        router.replace('/(tabs)');  
31      } catch (e: any) {  
32        setError(e.message || 'Failed to sign in. Please check your credentials.');33      } finally {  
34        setIsLoading(false);  
35      }  
36    };  
37  
38    return (  
39      <KeyboardAvoidingView  
40        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}  

```

```

41     style={[styles.container, { backgroundColor: colors.background }]}
42   >
43     <View style={[styles.content, { paddingTop: insets.top + 40, paddingBottom: insets.bottom + 20 }]}>
44       <View style={styles.header}>
45         <View style={styles.logoRow}>
46           <Feather name="archive" size={32} color={colors.primary} />
47           <Text style={[styles.brandName, { color: colors.primary }]}>TripJar</Text>
48         </View>
49         <Text style={[styles.title, { color: colors.foreground }]}>Welcome back</Text>
50       </View>
51
52       <View style={styles.form}>
53         <View style={styles.inputGroup}>
54           <Text style={[styles.label, { color: colors.foreground }]}>Email</Text>
55           <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
56 colors.border }]}>
57             <TextInput
58               style={[styles.input, { color: colors.foreground }]}
59               placeholder="you@example.com"
60               placeholderTextColor={colors.mutedForeground}
61               autoCapitalize="none"
62               keyboardType="email-address"
63               value={email}
64               onChangeText={setEmail}
65             />
66           </View>
67
68           <View style={styles.inputGroup}>
69             <Text style={[styles.label, { color: colors.foreground }]}>Password</Text>
70             <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
71 colors.border }]}>
72               <TextInput
73                 style={[styles.input, { color: colors.foreground }]}
74                 placeholder="....."
75                 placeholderTextColor={colors.mutedForeground}
76                 secureTextEntry={!showPassword}
77                 value={password}
78                 onChangeText={setPassword}
79               />
80               <Pressable onPress={() => setShowPassword(!showPassword)} style={styles.eyeIcon}>
81                 <Feather name={showPassword ? 'eye-off' : 'eye'} size={20} />
82               </Pressable>
83             </View>
84           </View>
85
86           {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
87
88           <Pressable
89             style={({ pressed }) => [
90               styles.button,
91               { backgroundColor: colors.primary },
92               pressed && { opacity: 0.9 },
93             ]}
94             onPress={handleLogin}
95             disabled={isLoading}
96           >
97             {isLoading ? (
98               <ActivityIndicator color={colors.primaryForeground} />
99             ) : (
100               <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Sign In</Text>

```

```

100         })
101     </Pressable>
102
103     <Pressable onPress={() => router.push('/(auth)/forgot-password')} style={styles.linkContainer}>
104         <Text style={[styles.linkText, { color: colors.primary }]}>Forgot password?</Text>
105     </Pressable>
106 </View>
107
108     <Pressable onPress={() => router.push('/(auth)/register')} style={styles.footerLink}>
109         <Text style={{ color: colors.mutedForeground }}>
110             Don't have an account? <Text style={{ color: colors.primary, fontWeight: 'bold' }}>Sign up</
111         </Text>
112     </Pressable>
113 </View>
114 </KeyboardAvoidingView>
115 );
116 }
117
118 const styles = StyleSheet.create({
119     container: {
120         flex: 1,
121     },
122     content: {
123         flex: 1,
124         padding: 24,
125     },
126     header: {
127         marginBottom: 40,
128     },
129     logoRow: {
130         flexDirection: 'row',
131         alignItems: 'center',
132         marginBottom: 16,
133     },
134     brandName: {
135         fontSize: 24,
136         fontWeight: 'bold',
137         marginLeft: 8,
138     },
139     title: {
140         fontSize: 32,
141         fontWeight: 'bold',
142     },
143     form: {
144         flex: 1,
145     },
146     inputGroup: {
147         marginBottom: 20,
148     },
149     label: {
150         fontSize: 14,
151         fontWeight: '600',
152         marginBottom: 8,
153     },
154     inputContainer: {
155         flexDirection: 'row',
156         alignItems: 'center',
157         borderWidth: 1,
158         borderRadius: 12,

```

artifacts/mobile/app/(auth)/login.tsx (continued)

```
159     height: 56,
160   },
161   input: {
162     flex: 1,
163     height: '100%',
164     paddingHorizontal: 16,
165     fontSize: 16,
166   },
167   eyeIcon: {
168     padding: 16,
169   },
170   errorText: {
171     fontSize: 14,
172     marginBottom: 16,
173   },
174   button: {
175     height: 56,
176     borderRadius: 12,
177     justifyContent: 'center',
178     alignItems: 'center',
179     marginTop: 8,
180   },
181   buttonText: {
182     fontSize: 16,
183     fontWeight: 'bold',
184   },
185   linkContainer: {
186     alignItems: 'center',
187     marginTop: 16,
188   },
189   linkText: {
190     fontSize: 14,
191     fontWeight: '600',
192   },
193   footerLink: {
194     alignItems: 'center',
195     marginTop: 'auto',
196   },
197 });
198
```

artifacts/mobile/app/(auth)/register.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform,
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { Feather } from '@expo/vector-icons';
6  import { useAuth } from '@/contexts/auth-context';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8
9  export default function RegisterScreen() {
10    const colors = useColors();
11    const router = useRouter();
12    const insets = useSafeAreaInsets();
13    const { register } = useAuth();
14
15    const [firstName, setFirstName] = useState('');
16    const [lastName, setLastName] = useState('');
```



```

17   const [email, setEmail] = useState('');
18   const [password, setPassword] = useState('');
19   const [showPassword, setShowPassword] = useState(false);
20   const [error, setError] = useState('');
21   const [isLoading, setIsLoading] = useState(false);
22
23   const getPasswordStrength = () => {
24     if (password.length === 0) return { width: '0%', color: colors.muted };
25     if (password.length < 6) return { width: '33%', color: colors.destructive };
26     if (password.length < 10) return { width: '66%', color: colors.warning };
27     return { width: '100%', color: colors.success };
28   };
29
30   const handleRegister = async () => {
31     if (!firstName || !lastName || !email || !password) {
32       setError('Please fill out all fields');
33       return;
34     }
35     try {
36       setIsLoading(true);
37       setError('');
38       await register({ firstName, lastName, email, password });
39       router.replace('/(auth)/profile-setup');
40     } catch (e: any) {
41       setError(e.message || 'Failed to create account. Please try again.');

```

```

76         <View style={{styles.inputGroup, { flex: 1, marginLeft: 8 }}}>
77             <Text style={{[styles.label, { color: colors.foreground }]}>Last Name</Text>
78             <View style={{[styles.inputContainer, { backgroundColor: colors.card, borderColor:
79                 colors.border }]}>
80                 <TextInput
81                     style={{[styles.input, { color: colors.foreground }]}
82                     placeholder="Doe"
83                     placeholderTextColor={colors.mutedForeground}
84                     value={lastName}
85                     onChangeText={setLastName}
86                 />
87             </View>
88         </View>
89
90         <View style={styles.inputGroup}>
91             <Text style={{[styles.label, { color: colors.foreground }]}>Email</Text>
92             <View style={{[styles.inputContainer, { backgroundColor: colors.card, borderColor:
93                 colors.border }]}>
94                 <TextInput
95                     style={{[styles.input, { color: colors.foreground }]}
96                     placeholder="you@example.com"
97                     placeholderTextColor={colors.mutedForeground}
98                     autoCapitalize="none"
99                     keyboardType="email-address"
100                    value={email}
101                    onChangeText={setEmail}
102                />
103            </View>
104
105            <View style={styles.inputGroup}>
106                <Text style={{[styles.label, { color: colors.foreground }]}>Password</Text>
107                <View style={{[styles.inputContainer, { backgroundColor: colors.card, borderColor:
108                    colors.border }]}>
109                    <TextInput
110                        style={{[styles.input, { color: colors.foreground }]}
111                        placeholder="....."
112                        placeholderTextColor={colors.mutedForeground}
113                        secureTextEntry={!showPassword}
114                        value={password}
115                        onChangeText={setPassword}
116                    />
117                    <Pressable onPress={() => setShowPassword(!showPassword)} style={styles.eyeIcon}>
118                        <Feather name={showPassword ? 'eye-off' : 'eye'} size={20}
119                        color={colors.mutedForeground} />
120                    </Pressable>
121                </View>
122                {password.length > 0 && (
123                    <View style={styles.strengthContainer}>
124                        <View style={{[styles.strengthBar, { backgroundColor: colors.muted }]}>
125                            <View style={{[styles.strengthFill, getPasswordStrength()]} />
126                        </View>
127                    </View>
128                )}
129            </View>
130
131            {error ? <Text style={{[styles.errorText, { color: colors.destructive }]}>{error}</Text> :
132            null}
133
134            <Pressable
135                style={{({ pressed }) => [
136                    styles.button,
137                    { backgroundColor: colors.primary },
138                ]}
139            />

```

```

135         pressed && { opacity: 0.9 },
136     ]}
137     onPress={handleRegister}
138     disabled={isLoading}
139 >
140     {isLoading ? (
141         <ActivityIndicator color={colors.primaryForeground} />
142     ) : (
143         <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Create Account</
144 Text>
145     )}
146     </Pressable>
147
148     <Text style={[styles.termsText, { color: colors.mutedForeground }]}>
149         By creating an account, you agree to the TripJar Terms of Service and Privacy Policy.
150     </Text>
151 </View>
152
153     <Pressable onPress={() => router.push('/(auth)/login')} style={styles.footerLink}>
154         <Text style={{ color: colors.mutedForeground }}>
155             Already have an account? <Text style={{ color: colors.primary, fontWeight: 'bold' }}>Sign
156         </Text>
157     </Pressable>
158 </View>
159 </ScrollView>
160 </KeyboardAvoidingView>
161 );
162 }
163
164 const styles = StyleSheet.create({
165     container: {
166         flex: 1,
167     },
168     content: {
169         flex: 1,
170         padding: 24,
171     },
172     header: {
173         marginBottom: 40,
174     },
175     logoRow: {
176         flexDirection: 'row',
177         alignItems: 'center',
178         marginBottom: 16,
179     },
180     brandName: {
181         fontSize: 24,
182         fontWeight: 'bold',
183         marginLeft: 8,
184     },
185     title: {
186         fontSize: 32,
187         fontWeight: 'bold',
188     },
189     form: {
190         flex: 1,
191     },
192     row: {
193         flexDirection: 'row',
194     },

```

```
194   inputGroup: {
195     marginBottom: 20,
196   },
197   label: {
198     fontSize: 14,
199     fontWeight: '600',
200     marginBottom: 8,
201   },
202   inputContainer: {
203     flexDirection: 'row',
204     alignItems: 'center',
205     borderWidth: 1,
206     borderRadius: 12,
207     height: 56,
208   },
209   input: {
210     flex: 1,
211     height: '100%',
212     paddingHorizontal: 16,
213     fontSize: 16,
214   },
215   eyeIcon: {
216     padding: 16,
217   },
218   strengthContainer: {
219     marginTop: 8,
220   },
221   strengthBar: {
222     height: 4,
223     borderRadius: 2,
224     overflow: 'hidden',
225   },
226   strengthFill: {
227     height: '100%',
228     borderRadius: 2,
229   },
230   errorText: {
231     fontSize: 14,
232     marginBottom: 16,
233   },
234   button: {
235     height: 56,
236     borderRadius: 12,
237     justifyContent: 'center',
238     alignItems: 'center',
239     marginTop: 8,
240   },
241   buttonText: {
242     fontSize: 16,
243     fontWeight: 'bold',
244   },
245   termsText: {
246     fontSize: 12,
247     textAlign: 'center',
248     marginTop: 16,
249     lineHeight: 18,
250   },
251   footerLink: {
252     alignItems: 'center',
```

```

253     marginTop: 32,
254   },
255   });
256

```

artifacts/mobile/app/(auth)/forgot-password.tsx

```

1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform,
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { Feather } from '@expo/vector-icons';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { useForgotPassword } from '@workspace/api-client-react';
8
9  export default function ForgotPasswordScreen() {
10   const colors = useColors();
11   const router = useRouter();
12   const insets = useSafeAreaInsets();
13
14   const [email, setEmail] = useState('');
15   const [isSuccess, setIsSuccess] = useState(false);
16   const [error, setError] = useState('');
17
18   const { mutateAsync: sendReset, isPending } = useForgotPassword();
19
20   const handleSendReset = async () => {
21     if (!email) {
22       setError('Please enter your email');
23       return;
24     }
25     try {
26       setError('');
27       await sendReset({ data: { email } });
28       setIsSuccess(true);
29     } catch (e: any) {
30       setError(e.message || 'Failed to send reset link.');

```

```

53         <Feather name="check-circle" size={24} color={colors.success} style={{ marginBottom: 12 }} />
54         <Text style={[styles.successText, { color: colors.foreground }]}>
55             Check your email for reset instructions.
56         </Text>
57         <Pressable
58             style={[styles.button, { backgroundColor: colors.primary, marginTop: 24, width: '100%' }]}
59             onPress={() => router.push('/(auth)/login')}
60         >
61             <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Return to Sign In</
62 Text>
63         </Pressable>
64     </View>
65 ) : (
66     <View style={styles.form}>
67         <View style={styles.inputGroup}>
68             <Text style={[styles.label, { color: colors.foreground }]}>Email</Text>
69             <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
70                 colors.border, borderWidth: 1, padding: 10 }]}>
71                 <TextInput
72                     style={[styles.input, { color: colors.foreground }]}
73                     placeholder="you@example.com"
74                     placeholderTextColor={colors.mutedForeground}
75                     autoCapitalize="none"
76                     keyboardType="email-address"
77                     value={email}
78                     onChangeText={setEmail}
79                 />
80             </View>
81             <View style={styles.errorText}>
82                 {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> :
83                 null}
84             </View>
85             <Pressable
86                 style={({ pressed }) => [
87                     styles.button,
88                     { backgroundColor: colors.primary },
89                     pressed && { opacity: 0.9 },
90                 ]}
91                 onPress={handleSendReset}
92                 disabled={isPending}
93             >
94                 {isPending ? (
95                     <ActivityIndicator color={colors.primaryForeground} />
96                 ) : (
97                     <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Send Reset Link</
98 Text>
99                 </Text>
100             </Pressable>
101         </View>
102     </View>
103 </KeyboardAvoidingView>
104 );
105 }
106
107 const styles = StyleSheet.create({
108     container: {
109         flex: 1,
110     },
111     content: {
112         flex: 1,
113         padding: 24,

```

```
112   },
113   backButton: {
114     width: 40,
115     height: 40,
116     justifyContent: 'center',
117     marginBottom: 24,
118   },
119   header: {
120     marginBottom: 40,
121   },
122   title: {
123     fontSize: 32,
124     fontWeight: 'bold',
125     marginBottom: 8,
126   },
127   subtitle: {
128     fontSize: 16,
129     lineHeight: 24,
130   },
131   form: {
132     flex: 1,
133   },
134   inputGroup: {
135     marginBottom: 24,
136   },
137   label: {
138     fontSize: 14,
139     fontWeight: '600',
140     marginBottom: 8,
141   },
142   inputContainer: {
143     flexDirection: 'row',
144     alignItems: 'center',
145     borderWidth: 1,
146     borderRadius: 12,
147     height: 56,
148   },
149   input: {
150     flex: 1,
151     height: '100%',
152     paddingHorizontal: 16,
153     fontSize: 16,
154   },
155   errorText: {
156     fontSize: 14,
157     marginBottom: 16,
158   },
159   button: {
160     height: 56,
161     borderRadius: 12,
162     justifyContent: 'center',
163     alignItems: 'center',
164   },
165   buttonText: {
166     fontSize: 16,
167     fontWeight: 'bold',
168   },
169   successBox: {
170     padding: 24,
```

artifacts/mobile/app/(auth)/forgot-password.tsx (continued)

```
171     borderRadius: 16,
172     borderWidth: 1,
173     alignItems: 'center',
174   },
175   successText: {
176     fontSize: 16,
177     textAlign: 'center',
178     lineHeight: 24,
179   },
180 });
181
```

artifacts/mobile/app/(auth)/profile-setup.tsx

```
1  import React, { useState, useEffect } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform,
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useAuth } from '@/contexts/auth-context';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { useUpdateProfile } from '@workspace/api-client-react';
8
9  export default function ProfileSetupScreen() {
10    const colors = useColors();
11    const router = useRouter();
12    const insets = useSafeAreaInsets();
13    const { profile, refresh } = useAuth();
14
15    const [displayName, setDisplayName] = useState('');
16    const [phone, setPhone] = useState('');
17    const [timeZone, setTimeZone] = useState('America/New_York');
18    const [error, setError] = useState('');
19
20    const { mutateAsync: updateProfile, isPending } = useUpdateProfile();
21
22    useEffect(() => {
23      if (profile) {
24        setDisplayName(profile.displayName || `${profile.firstName} ${profile.lastName}`.trim());
25        setPhone(profile.phone || '');
26        setTimeZone(profile.timeZone || 'America/New_York');
27      }
28    }, [profile]);
29
30    const handleComplete = async () => {
31      try {
32        setError('');
33        await updateProfile({
34          data: {
35            displayName,
36            phone: phone || null,
37            timeZone,
38          }
39        });
40        await refresh();
41        router.replace('/(tabs)');
42      } catch (e: any) {
43        setError(e.message || 'Failed to update profile.');
```



```

46
47   const handleSkip = () => {
48     router.replace('/(tabs)');
49   };
50
51   return (
52     <KeyboardAvoidingView
53       behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
54       style={[styles.container, { backgroundColor: colors.background }]}
55     >
56       <View style={[styles.content, { paddingTop: insets.top + 40, paddingBottom: insets.bottom + 20 }]}>
57         <View style={styles.header}>
58           <Text style={[styles.title, { color: colors.foreground }]}>One more thing</Text>
59           <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
60             Let's finish setting up your profile.
61           </Text>
62         </View>
63
64         <View style={styles.form}>
65           <View style={styles.inputGroup}>
66             <Text style={[styles.label, { color: colors.foreground }]}>Display Name</Text>
67             <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
68 colors.border }]}>
69               <TextInput
70                 style={[styles.input, { color: colors.foreground }]}
71                 placeholder="How should we call you?"
72                 placeholderTextColor={colors.mutedForeground}
73                 value={displayName}
74                 onChangeText={setDisplayName}
75               </>
76             </View>
77
78             <View style={styles.inputGroup}>
79               <Text style={[styles.label, { color: colors.foreground }]}>Phone Number (Optional)</Text>
80               <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
81 colors.border }]}>
82                 <TextInput
83                   style={[styles.input, { color: colors.foreground }]}
84                   placeholder="(555) 555-5555"
85                   placeholderTextColor={colors.mutedForeground}
86                   keyboardType="phone-pad"
87                   value={phone}
88                   onChangeText={setPhone}
89                 </>
90               </View>
91
92               <View style={styles.inputGroup}>
93                 <Text style={[styles.label, { color: colors.foreground }]}>Time Zone</Text>
94                 <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
95 colors.border }]}>
96                   <TextInput
97                     style={[styles.input, { color: colors.foreground }]}
98                     value={timeZone}
99                     onChangeText={setTimeZone}
100                   </>
101                 </View>
102
103               {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
104

```

```

105         <Pressable
106             style={({ pressed }) => [
107                 styles.button,
108                 { backgroundColor: colors.primary },
109                 pressed && { opacity: 0.9 },
110             ]}
111             onPress={handleComplete}
112             disabled={isPending}
113         >
114             {isPending ? (
115                 <ActivityIndicator color={colors.primaryForeground} />
116             ) : (
117                 <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Complete Setup</
118 Text>
119             )}
120         </Pressable>
121
122         <Pressable onPress={handleSkip} style={styles.skipButton}>
123             <Text style={[styles.skipText, { color: colors.mutedForeground }]}>Skip for now</Text>
124         </Pressable>
125     </View>
126 </KeyboardAvoidingView>
127 );
128 }
129
130 const styles = StyleSheet.create({
131     container: {
132         flex: 1,
133     },
134     content: {
135         flex: 1,
136         padding: 24,
137     },
138     header: {
139         marginBottom: 40,
140     },
141     title: {
142         fontSize: 32,
143         fontWeight: 'bold',
144         marginBottom: 8,
145     },
146     subtitle: {
147         fontSize: 16,
148         lineHeight: 24,
149     },
150     form: {
151         flex: 1,
152     },
153     inputGroup: {
154         marginBottom: 20,
155     },
156     label: {
157         fontSize: 14,
158         fontWeight: '600',
159         marginBottom: 8,
160     },
161     inputContainer: {
162         flexDirection: 'row',
163         alignItems: 'center',

```

```
164     borderWidth: 1,
165     borderRadius: 12,
166     height: 56,
167   },
168   input: {
169     flex: 1,
170     height: '100%',
171     paddingHorizontal: 16,
172     fontSize: 16,
173   },
174   errorText: {
175     fontSize: 14,
176     marginBottom: 16,
177   },
178   button: {
179     height: 56,
180     borderRadius: 12,
181     justifyContent: 'center',
182     alignItems: 'center',
183     marginTop: 8,
184   },
185   buttonText: {
186     fontSize: 16,
187     fontWeight: 'bold',
188   },
189   skipButton: {
190     alignItems: 'center',
191     marginTop: 24,
192     padding: 8,
193   },
194   skipText: {
195     fontSize: 16,
196     fontWeight: '500',
197   },
198 });
199
```

Mobile App — Tab Screens

artifacts/mobile/app/(tabs)/index.tsx

```
1  import React, { useCallback, useState } from 'react';
2  import { View, Text, StyleSheet, ScrollView, RefreshControl, Pressable, Platform } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useGetDashboard } from '@workspace/api-client-react';
7  import { useAuth } from '@contexts/auth-context';
8  import { MemberAvatar } from '@components/MemberAvatar';
9  import { ProgressBar } from '@components/ProgressBar';
10 import { EmptyState } from '@components/EmptyState';
11 import { SkeletonLoader } from '@components/SkeletonLoader';
12 import { CircularProgress } from '@components/CircularProgress';
13 import { Feather } from '@expo/vector-icons';
14 import { ImageBackground } from 'expo-image';
15 import { LinearGradient } from 'expo-linear-gradient';
16 import { JarHealthBadge } from '@components/JarHealthBadge';
17 import * as Haptics from 'expo-haptics';
18
19 export default function HomeScreen() {
20   const colors = useColors();
21   const router = useRouter();
22   const insets = useSafeAreaInsets();
23   const { profile } = useAuth();
24   const [refreshing, setRefreshing] = useState(false);
25
26   const { data: dashboard, isLoading, refetch } = useGetDashboard();
27
28   const onRefresh = useCallback(async () => {
29     setRefreshing(true);
30     await refetch();
31     setRefreshing(false);
32   }, [refetch]);
33
34   const formatCurrency = (cents: number) => {
35     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
36   };
37
38   const getTimeOfDay = () => {
39     const hour = new Date().getHours();
40     if (hour < 12) return 'Good morning';
41     if (hour < 17) return 'Good afternoon';
42     return 'Good evening';
43   };
44
45   const getTodayDate = () => {
46     return new Date().toLocaleDateString('en-US', { weekday: 'long', month: 'long', day: 'numeric' });
47   };
48
49   const renderSkeleton = () => (
50     <View style={[styles.container, { paddingTop: insets.top + 20, paddingHorizontal: 16 }]}>
51       <SkeletonLoader width={200} height={32} style={{ marginBottom: 8 }} />
52       <SkeletonLoader width={150} height={20} style={{ marginBottom: 32 }} />
```

```

53     <SkeletonLoader width="100%" height={240} borderRadius={16} style={{ marginBottom: 24 }} />
54     <SkeletonLoader width="100%" height={120} borderRadius={16} />
55   </View>
56 );
57
58 if (isLoading && !dashboard) {
59   return renderSkeleton();
60 }
61
62 const hasJars = dashboard && dashboard.totalJars > 0;
63
64 return (
65   <ScrollView
66     style={[styles.container, { backgroundColor: colors.background }]}
67     contentContainerStyle=[
68       styles.contentContainer,
69       { paddingTop: insets.top + 20, paddingBottom: insets.bottom + 80 },
70     ]
71     refreshControl={
72       <RefreshControl refreshing={refreshing} onRefresh={onRefresh}
73       tint={colors.primary} />
74     <View style={styles.header}>
75       <Text style={[styles.greeting, { color: colors.foreground }]}>
76         {getTimeOfDay()}, {profile?.firstName || 'Traveler'}!
77       </Text>
78       <Text style={[styles.dateText, { color: colors.mutedForeground }]}>{getTodayDate()}</Text>
79     </View>
80     {!hasJars ? (
81       <EmptyState
82         icon="archive"
83         title="You don't have any Jars yet"
84         description="Create your first goal and start saving together."
85         action={{
86           label: 'Create a Jar',
87           onPress: () => router.push('/create-jar'),
88         }}
89       />
90     ) : (
91       <>
92         {dashboard?.featuredJar && (
93           <Pressable
94             onPress={() => router.push(`/jar/${dashboard.featuredJar!.id}`)}
95             style={({ pressed }) => [
96               styles.heroCard,
97               { backgroundColor: colors.card, borderColor: colors.border },
98               pressed && { opacity: 0.95, transform: [{ scale: 0.98 }] },
99             ]}
100           >
101             <ImageBackground
102               source={
103                 dashboard.featuredJar.coverImageUrl
104                 ? { uri: dashboard.featuredJar.coverImageUrl }
105                 : require('../../assets/images/hawaii-cover.jpg')
106               }
107               style={styles.heroImage}
108             >
109               <LinearGradient colors={['rgba(0,0,0,0.2)', 'rgba(0,0,0,0.8)']}
110               style={Style.absoluteFill} />
111               <View style={styles.heroContent}>
112                 <View style={styles.heroTopRow}>

```

```

112             <View>
113                 <Text style={styles.heroJarName}>{dashboard.featuredJar.name}</Text>
114                 {dashboard.featuredJar.destination && (
115                     <Text style={styles.heroDestination}>{dashboard.featuredJar.destination}</Text>
116                 )}
117             </View>
118             {dashboard.featuredJar.daysRemaining !== null &&
119                 <View style={styles.daysChip}>
120                     <Text style={styles.daysText}>{dashboard.featuredJar.daysRemaining} days left</
121                 </View>
122             )}
123         </View>
124
125         <View style={styles.heroBottomRow}>
126             <CircularProgress size={80} progress={dashboard.featuredJar.percentFunded}>
127                 <Text style={styles.heroPercentText}>{dashboard.featuredJar.percentFunded}%</Text>
128             </CircularProgress>
129             <View style={styles.heroStats}>
130                 <Text style={styles.heroAmountText}>
131                     <Text style={{ fontWeight: 'bold', color: '#fff' }}
132 >{formatCurrency(dashboard.featuredJar.healthStatus, { color: 'rgba(255,255,255,0.7)' }}> of
133                     </Text>
134                     {dashboard.featuredJar.health && (
135                         <View style={{ marginTop: 8 }}>
136                             <JarHealthBadge status={dashboard.featuredJar.health.status} />
137                         </View>
138                     )}
139                 </View>
140             </View>
141         </View>
142     </ImageBackground>
143 </Pressable>
144 )}
145
146     {dashboard?.personalProgress && (
147         <View style={[styles.sectionCard, { backgroundColor: colors.card, borderColor:
148 colors.border }]}>
149             <Text style={[styles.sectionTitle, { color: colors.foreground }]}>Your Contribution</Text>
150             <ProgressBar progress={dashboard.personalProgress.percentComplete} height={8} />
151             <View style={styles.progressRow}>
152                 <Text style={[styles.progressText, { color: colors.mutedForeground }]}>
153                     Contributed: {formatCurrency(dashboard.personalProgress.contributedAmountCents)} /
154                 </Text>
155                 <Text style={[styles.progressText, { color: colors.foreground, fontWeight: '600' }]}>
156                     Remaining: {formatCurrency(dashboard.personalProgress.remainingAmountCents)}
157                 </Text>
158             </View>
159             {dashboard.personalProgress.nextScheduledDate && (
160                 <View style={[styles.scheduledInfo, { backgroundColor: colors.secondary }]}>
161                     <Feather name="calendar" size={14} color={colors.primary} />
162                     <Text style={[styles.scheduledText, { color: colors.secondaryForeground }]}>
163                         Next: {formatCurrency(dashboard.personalProgress.nextScheduledAmountCents || 0)} on
164                     </Text>
165                 </View>
166             )}
167         </View>
168     )}
169
170     {dashboard?.memberProgress && dashboard.memberProgress.length > 0 && (
171         <View style={styles.section}>

```

```

171         <Text style={[styles.sectionHeaderTitle, { color: colors.foreground }]}>Group Progress</
172 Text>
173         <ScrollView horizontal showsHorizontalScrollIndicator={false}
174         <View key={member.memberId} style={[styles.memberCard, { backgroundColor: colors.card,
175         <MemberAvatar
176             displayName={member.displayName}
177             avatarUrl={member.avatarUrl}
178             size={48}
179             showStatus
180             status={member.status}
181         />
182         <Text style={[styles.memberName, { color: colors.foreground }]} numberOfLines={1}>
183             {member.displayName.split(' ')[0]}
184         </Text>
185         <Text style={[styles.memberPercent, { color: colors.primary }]}
186 >{member.percentComplete}%</Text>
187         )}
188     </ScrollView>
189 </View>
190 )}
191
192 <View style={styles.quickActionsContainer}>
193     <Pressable
194         style={({ pressed }) => [
195             styles.actionButton,
196             { backgroundColor: colors.primary },
197             pressed && { opacity: 0.9 },
198         ]}
199         onPress={() => {
200             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
201             if (dashboard?.featuredJar) {
202                 router.push(`/contribution/${dashboard.featuredJar.id}`);
203             }
204         }}
205     >
206         <Feather name="plus" size={20} color={colors.primaryForeground} />
207         <Text style={[styles.actionButtonText, { color: colors.primaryForeground }]}>Add Funds</
208 Text>
209     </Pressable>
210     <Pressable
211         style={({ pressed }) => [
212             styles.actionButton,
213             { backgroundColor: colors.secondary },
214             pressed && { opacity: 0.8 },
215         ]}
216         onPress={() => {
217             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
218             router.push('/create-jar');
219         }}
220     >
221         <Feather name="archive" size={20} color={colors.primary} />
222         <Text style={[styles.actionButtonText, { color: colors.primary }]}>New Jar</Text>
223     </Pressable>
224 </View>
225 </>
226 )}
227 </ScrollView>
228 );
229 }

```

```
230
231 const styles = StyleSheet.create({
232   container: {
233     flex: 1,
234   },
235   contentContainer: {
236     paddingHorizontal: 16,
237   },
238   header: {
239     marginBottom: 24,
240   },
241   greeting: {
242     fontSize: 28,
243     fontWeight: 'bold',
244     marginBottom: 4,
245     letterSpacing: -0.5,
246   },
247   dateText: {
248     fontSize: 16,
249     fontWeight: '500',
250   },
251   heroCard: {
252     borderRadius: 24,
253     overflow: 'hidden',
254     borderWidth: 1,
255     height: 280,
256     marginBottom: 24,
257   },
258   heroImage: {
259     width: '100%',
260     height: '100%',
261   },
262   heroContent: {
263     flex: 1,
264     padding: 24,
265     justifyContent: 'space-between',
266   },
267   heroTopRow: {
268     flexDirection: 'row',
269     justifyContent: 'space-between',
270     alignItems: 'flex-start',
271   },
272   heroJarName: {
273     fontSize: 28,
274     fontWeight: 'bold',
275     color: '#fff',
276     marginBottom: 4,
277   },
278   heroDestination: {
279     fontSize: 16,
280     color: '#E8F6EF',
281     fontWeight: '500',
282   },
283   daysChip: {
284     backgroundColor: 'rgba(0,0,0,0.5)',
285     paddingHorizontal: 12,
286     paddingVertical: 6,
287     borderRadius: 16,
288     backdropFilter: 'blur(4px)', // Web
```



```
289   },
290   daysText: {
291     color: '#fff',
292     fontSize: 14,
293     fontWeight: 'bold',
294   },
295   heroBottomRow: {
296     flexDirection: 'row',
297     alignItems: 'center',
298     gap: 20,
299   },
300   heroPercentText: {
301     color: '#fff',
302     fontSize: 20,
303     fontWeight: 'bold',
304   },
305   heroStats: {
306     flex: 1,
307   },
308   heroAmountText: {
309     fontSize: 16,
310   },
311   sectionCard: {
312     padding: 20,
313     borderRadius: 20,
314     borderWidth: 1,
315     marginBottom: 24,
316   },
317   sectionTitle: {
318     fontSize: 18,
319     fontWeight: 'bold',
320     marginBottom: 16,
321   },
322   progressRow: {
323     flexDirection: 'row',
324     justifyContent: 'space-between',
325     marginTop: 12,
326   },
327   progressText: {
328     fontSize: 13,
329   },
330   scheduledInfo: {
331     flexDirection: 'row',
332     alignItems: 'center',
333     padding: 12,
334     borderRadius: 12,
335     marginTop: 16,
336     gap: 8,
337   },
338   scheduledText: {
339     fontSize: 14,
340     fontWeight: '500',
341   },
342   section: {
343     marginBottom: 24,
344   },
345   sectionHeaderTitle: {
346     fontSize: 20,
347     fontWeight: 'bold',
```

artifacts/mobile/app/(tabs)/index.tsx (continued)

```
348     marginBottom: 16,
349   },
350   horizontalScroll: {
351     paddingRight: 16,
352     gap: 12,
353   },
354   memberCard: {
355     padding: 16,
356     borderRadius: 16,
357     borderWidth: 1,
358     alignItems: 'center',
359     width: 100,
360   },
361   memberName: {
362     fontSize: 14,
363     fontWeight: '600',
364     marginTop: 8,
365     marginBottom: 4,
366   },
367   memberPercent: {
368     fontSize: 14,
369     fontWeight: 'bold',
370   },
371   quickActionsContainer: {
372     flexDirection: 'row',
373     gap: 12,
374     marginBottom: 24,
375   },
376   actionButton: {
377     flex: 1,
378     flexDirection: 'row',
379     alignItems: 'center',
380     justifyContent: 'center',
381     height: 56,
382     borderRadius: 16,
383     gap: 8,
384   },
385   actionButtonText: {
386     fontSize: 16,
387     fontWeight: 'bold',
388   },
389 });
390
```

artifacts/mobile/app/(tabs)/jars.tsx

```
1  import React, { useState, useCallback } from 'react';
2  import { View, Text, StyleSheet, FlatList, Pressable } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useListJars } from '@workspace/api-client-react';
7  import { JarCard } from '@components/JarCard';
8  import { EmptyState } from '@components/EmptyState';
9  import { SkeletonLoader } from '@components/SkeletonLoader';
10 import { Feather } from '@expo/vector-icons';
11
12 type TabType = 'Active' | 'Invited' | 'Completed' | 'Archived';
13
```

```

14 export default function JarsScreen() {
15   const colors = useColors();
16   const router = useRouter();
17   const insets = useSafeAreaInsets();
18   const [activeTab, setActiveTab] = useState<TabType>('Active');
19
20   // Map local tab state to API status parameter
21   const getStatusParam = () => {
22     switch (activeTab) {
23       case 'Active': return 'Saving, FullyFunded';
24       case 'Invited': return 'Inviting';
25       case 'Completed': return 'Completed';
26       case 'Archived': return 'Cancelled';
27       default: return undefined;
28     }
29   };
30
31   const { data: jars, isLoading, refetch, isRefetching } = useListJars({ status: getStatusParam() });
32
33   const tabs: TabType[] = ['Active', 'Invited', 'Completed', 'Archived'];
34
35   const renderHeader = () => (
36     <View style={[styles.header, { paddingTop: insets.top + 16, backgroundColor: colors.background }]}>
37       <View style={styles.headerRow}>
38         <Text style={[styles.title, { color: colors.foreground }]}>My Jars</Text>
39         <Pressable
40           style={[styles.fab, { backgroundColor: colors.primary }]}
41           onPress={() => router.push('/create-jar')}
42         >
43           <Feather name="plus" size={24} color={colors.primaryForeground} />
44         </Pressable>
45       </View>
46
47       <View style={styles.tabsContainer}>
48         <FlatList
49           horizontal
50           showsHorizontalScrollIndicator={false}
51           data={tabs}
52           keyExtractor={(item) => item}
53           renderItem={({ item }) => (
54             <Pressable
55               style={[
56                 styles.tab,
57                 activeTab === item ? { backgroundColor: colors.foreground } : { backgroundColor:
58 colors.secondary }...
59               ]}
60               onPress={() => setActiveTab(item)}
61             >
62               <Text
63                 style={[
64                   styles.tabText,
65                   activeTab === item ? { color: colors.background } : { color: colors.foreground },
66                 ]}
67               >
68                 {item}
69               </Text>
70             </Pressable>
71           )}
72           contentContainerStyle={styles.tabsList}
73         </FlatList>
74       </View>

```

```

73     </View>
74   </View>
75 );
76
77 const renderEmpty = () => {
78   if (isLoading) {
79     return (
80       <View style={styles.emptyContainer}>
81         <SkeletonLoader width="100%" height={200} borderRadius={16} style={{ marginBottom: 16 }} />
82         <SkeletonLoader width="100%" height={200} borderRadius={16} />
83       </View>
84     );
85   }
86
87   let message = "You don't have any jars in this category.";
88   if (activeTab === 'Active') message = "You don't have any active jars right now.";
89   else if (activeTab === 'Invited') message = "You have no pending invitations.";
90
91   return (
92     <EmptyState
93       icon="archive"
94       title="No Jars Found"
95       description={message}
96       action={activeTab === 'Active' ? {
97         label: 'Create a Jar',
98         onPress: () => router.push('/create-jar'),
99       } : undefined}
100     />
101   );
102 };
103
104 return (
105   <View style={[styles.container, { backgroundColor: colors.background }]}>
106     {renderHeader()}
107     <FlatList
108       data={jars || []}
109       keyExtractor={({item}) => item.id}
110       renderItem={({item}) => <JarCard jar={item} />}
111       contentContainerStyle={[
112         styles.listContent,
113         { paddingBottom: insets.bottom + 100 },
114       ]}
115       ListEmptyComponent={renderEmpty}
116       refreshing={isRefetching}
117       onRefresh={refetch}
118     />
119   </View>
120 );
121 }
122
123 const styles = StyleSheet.create({
124   container: {
125     flex: 1,
126   },
127   header: {
128     paddingHorizontal: 16,
129     paddingBottom: 16,
130     borderBottomWidth: 1,
131     borderBottomColor: 'rgba(0,0,0,0.05)',

```

artifacts/mobile/app/(tabs)/jars.tsx (continued)

```
132   },
133   headerRow: {
134     flexDirection: 'row',
135     justifyContent: 'space-between',
136     alignItems: 'center',
137     marginBottom: 20,
138   },
139   title: {
140     fontSize: 32,
141     fontWeight: 'bold',
142   },
143   fab: {
144     width: 44,
145     height: 44,
146     borderRadius: 22,
147     alignItems: 'center',
148     justifyContent: 'center',
149   },
150   tabsContainer: {
151     marginTop: 8,
152   },
153   tabsList: {
154     gap: 8,
155   },
156   tab: {
157     paddingHorizontal: 16,
158     paddingVertical: 8,
159     borderRadius: 20,
160   },
161   tabText: {
162     fontSize: 14,
163     fontWeight: '600',
164   },
165   listContent: {
166     padding: 16,
167   },
168   emptyContainer: {
169     marginTop: 20,
170   },
171 });
172
```

artifacts/mobile/app/(tabs)/activity.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, FlatList, Platform } from 'react-native';
3  import { useColors } from '@/hooks/useColors';
4  import { useSafeAreaInsets } from 'react-native-safe-area-context';
5  import { useListJarActivity } from '@workspace/api-client-react';
6  import { MemberAvatar } from '@components/MemberAvatar';
7  import { EmptyState } from '@components/EmptyState';
8  import { SkeletonLoader } from '@components/SkeletonLoader';
9  import type { ActivityEvent } from '@workspace/api-client-react/src/generated/api.schemas';
10 import { Feather } from '@expo/vector-icons';
11
12 export default function ActivityScreen() {
13   const colors = useColors();
14   const insets = useSafeAreaInsets();
15
```

```

16   const { data: activityItems, isLoading, refetch, isRefetching } = useListJarActivity();
17
18   const getRelativeTime = (dateStr: string) => {
19     const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
20     const date = new Date(dateStr);
21     const now = new Date();
22     const diffHours = (date.getTime() - now.getTime()) / (1000 * 60 * 60);
23
24     if (Math.abs(diffHours) < 1) {
25       const diffMins = Math.round(diffHours * 60);
26       return rtf.format(diffMins, 'minute');
27     }
28     if (Math.abs(diffHours) < 24) {
29       return rtf.format(Math.round(diffHours), 'hour');
30     }
31     const diffDays = Math.round(diffHours / 24);
32     return rtf.format(diffDays, 'day');
33   };
34
35   const renderItem = ({ item }: { item: ActivityEvent }) => {
36     const isContribution = item.eventType === 'contribution_added';
37
38     return (
39       <View style={[styles.activityItem, { borderBottomColor: colors.border }]}>
40         <View style={styles.avatarContainer}>
41           {item.actorName ? (
42             <MemberAvatar displayName={item.actorName} avatarUrl={item.actorAvatarUrl} size={40} />
43           ) : (
44             <View style={[styles.systemAvatar, { backgroundColor: colors.secondary }]}>
45               <Feather name="bell" size={20} color={colors.primary} />
46             </View>
47           )}
48         </View>
49         <View style={styles.activityContent}>
50           <Text style={[styles.description, { color: colors.foreground }]}>
51             {item.description}
52           </Text>
53           <Text style={[styles.timeText, { color: colors.mutedForeground }]}>
54             {getRelativeTime(item.createdAt)}
55           </Text>
56         </View>
57         {isContribution && item.amountCents ? (
58           <View style={[styles.amountChip, { backgroundColor: colors.lightGreen }]}>
59             <Text style={[styles.amountText, { color: colors.darkGreen }]}>
60               +${(item.amountCents / 100).toLocaleString()}
61             </Text>
62           </View>
63         ) : null}
64       </View>
65     );
66   };
67
68   return (
69     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
70       <View style={styles.header}>
71         <Text style={[styles.title, { color: colors.foreground }]}>Activity</Text>
72       </View>
73
74       <FlatList

```

```

75     data={activityItems || []}
76     keyExtractor={({item}) => item.id}
77     renderItem={renderItem}
78     contentContainerStyle=[styles.listContent, { paddingBottom: insets.bottom + 100 }]
79     refreshing={isRefetching}
80     onRefresh={refetch}
81     ListEmptyComponent={
82       isLoading ? (
83         <View style={styles.skeletonContainer}>
84           <SkeletonLoader height={60} style={{ marginBottom: 16 }} />
85           <SkeletonLoader height={60} style={{ marginBottom: 16 }} />
86           <SkeletonLoader height={60} />
87         </View>
88       ) : (
89         <EmptyState
90           icon="activity"
91           title="No recent activity"
92           description="When members contribute or jars are updated, you'll see it here."
93         />
94       )
95     }
96   />
97 </View>
98 );
99 }
100
101 const styles = StyleSheet.create({
102   container: {
103     flex: 1,
104   },
105   header: {
106     paddingHorizontal: 16,
107     paddingVertical: 16,
108     borderBottomWidth: 1,
109     borderBottomColor: 'rgba(0,0,0,0.05)',
110   },
111   title: {
112     fontSize: 32,
113     fontWeight: 'bold',
114   },
115   listContent: {
116     flexGrow: 1,
117   },
118   activityItem: {
119     flexDirection: 'row',
120     padding: 16,
121     borderBottomWidth: 1,
122     alignItems: 'center',
123   },
124   avatarContainer: {
125     marginRight: 12,
126   },
127   systemAvatar: {
128     width: 40,
129     height: 40,
130     borderRadius: 20,
131     alignItems: 'center',
132     justifyContent: 'center',
133   },

```

artifacts/mobile/app/(tabs)/activity.tsx (continued)

```
134   activityContent: {
135     flex: 1,
136     marginRight: 12,
137   },
138   description: {
139     fontSize: 15,
140     lineHeight: 20,
141     marginBottom: 4,
142   },
143   timeText: {
144     fontSize: 13,
145   },
146   amountChip: {
147     paddingHorizontal: 10,
148     paddingVertical: 4,
149     borderRadius: 12,
150   },
151   amountText: {
152     fontWeight: 'bold',
153     fontSize: 14,
154   },
155   skeletonContainer: {
156     padding: 16,
157   },
158 });
159
```

artifacts/mobile/app/(tabs)/notifications.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, FlatList, Pressable } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useListNotifications } from '@workspace/api-client-react';
7  import { EmptyState } from '@components/EmptyState';
8  import { SkeletonLoader } from '@components/SkeletonLoader';
9  import { Feather } from '@expo/vector-icons';
10 import type { Notification } from '@workspace/api-client-react/src/generated/api.schemas';
11
12 export default function NotificationsScreen() {
13   const colors = useColors();
14   const router = useRouter();
15   const insets = useSafeAreaInsets();
16
17   const { data: notifications, isLoading, refetch, isRefetching } = useListNotifications();
18
19   // Mock mark as read (actual API might have a mutation for this)
20   const markAsRead = (id: string) => {
21     // API call would go here
22   };
23
24   const getIconForType = (type: string) => {
25     switch (type) {
26       case 'contribution_recorded': return 'dollar-sign';
27       case 'invitation_received': return 'mail';
28       case 'member_joined': return 'user-plus';
29       case 'goal_fully_funded': return 'check-circle';
30       case 'jar_halfway_funded': return 'trending-up';
```



```

31     default: return 'bell';
32   }
33 };
34
35 const renderItem = ({ item }: { item: Notification }) => {
36   return (
37     <Pressable
38       style={({ pressed }) => [
39         styles.notificationItem,
40         { borderBottomColor: colors.border },
41         !item.isRead && { backgroundColor: colors.secondary },
42         pressed && { opacity: 0.7 },
43       ]
44       onPress={() => {
45         if (!item.isRead) markAsRead(item.id);
46         if (item.relatedJarId) {
47           router.push(`/jar/${item.relatedJarId}`);
48         } else if (item.actionUrl) {
49           // handle other URLs if needed
50         }
51       }}
52     >
53       <View style={[styles.iconContainer, { backgroundColor: item.isRead ? colors.card :
54 colors.background }]}>
55         <Image alt="notification icon" name={getIconForType(item.type)} size={20} color={colors.primary} />
56       </View>
57       <View style={styles.content}>
58         <Text style={[styles.title, { color: colors.foreground }]}>{item.title}</Text>
59         <Text style={[styles.message, { color: colors.mutedForeground }]}>{item.message}</Text>
60         <Text style={[styles.time, { color: colors.mutedForeground }]}>
61           {new Date(item.createdAt).toLocaleDateString()}
62         </Text>
63       </View>
64       {!item.isRead && <View style={[styles.unreadDot, { backgroundColor: colors.primary }]} />}
65     </Pressable>
66   );
67 };
68
69 return (
70   <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
71     <View style={styles.header}>
72       <Text style={[styles.headerTitle, { color: colors.foreground }]}>Notifications</Text>
73     </View>
74     <FlatList
75       data={notifications || []}
76       keyExtractor={({ item }) => item.id}
77       renderItem={renderItem}
78       contentContainerStyle={[styles.listContent, { paddingBottom: insets.bottom + 100 }]}
79       refreshing={isRefetching}
80       onRefresh={refetch}
81       ListEmptyComponent={
82         isLoading ? (
83           <View style={{ padding: 16, gap: 16 }}>
84             <SkeletonLoader height={80} />
85             <SkeletonLoader height={80} />
86           </View>
87         ) : (
88           <EmptyState
89             icon="bell"

```

```
90         title="You're all caught up"
91         description="We'll notify you when there's activity in your jars."
92     />
93 )
94 }
95 />
96 </View>
97 );
98 }
99
100 const styles = StyleSheet.create({
101     container: {
102         flex: 1,
103     },
104     header: {
105         paddingHorizontal: 16,
106         paddingVertical: 16,
107         borderBottomWidth: 1,
108         borderBottomColor: 'rgba(0,0,0,0.05)',
109     },
110     headerTitle: {
111         fontSize: 32,
112         fontWeight: 'bold',
113     },
114     listContent: {
115         flexGrow: 1,
116     },
117     notificationItem: {
118         flexDirection: 'row',
119         padding: 16,
120         borderBottomWidth: 1,
121     },
122     iconContainer: {
123         width: 48,
124         height: 48,
125         borderRadius: 24,
126         alignItems: 'center',
127         justifyContent: 'center',
128         marginRight: 16,
129         shadowColor: '#000',
130         shadowOpacity: 0.05,
131         shadowOffset: { width: 0, height: 2 },
132         shadowRadius: 4,
133         elevation: 2,
134     },
135     content: {
136         flex: 1,
137     },
138     title: {
139         fontSize: 16,
140         fontWeight: 'bold',
141         marginBottom: 4,
142     },
143     message: {
144         fontSize: 14,
145         lineHeight: 20,
146         marginBottom: 8,
147     },
148     time: {
```

artifacts/mobile/app/(tabs)/notifications.tsx (continued)

```
149     fontSize: 12,
150   },
151   unreadDot: {
152     width: 8,
153     height: 8,
154     borderRadius: 4,
155     marginLeft: 8,
156     marginTop: 8,
157   },
158 });
159
```

artifacts/mobile/app/(tabs)/profile.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, ScrollView, Pressable, Alert, Platform } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useAuth } from '@/contexts/auth-context';
7  import { useGetDashboard } from '@workspace/api-client-react';
8  import { MemberAvatar } from '@components/MemberAvatar';
9  import { Feather } from '@expo/vector-icons';
10
11  export default function ProfileScreen() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { profile, user, logout } = useAuth();
16
17    const { data: dashboard } = useGetDashboard();
18
19    const handleLogout = () => {
20      Alert.alert(
21        "Sign Out",
22        "Are you sure you want to sign out?",
23        [
24          { text: "Cancel", style: "cancel" },
25          {
26            text: "Sign Out",
27            style: "destructive",
28            onPress: async () => {
29              await logout();
30              router.replace('/(auth)');
31            }
32          }
33        ],
34      );
35    };
36
37    const formatCurrency = (cents: number) => {
38      return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
39    };
40
41    const SettingRow = ({ icon, title, onPress, destructive = false, placeholder = false }: any) => (
42      <Pressable
43        style={({ pressed }) => [
44          styles.settingRow,
45          { borderBottomColor: colors.border },

```

```

46         pressed && { backgroundColor: colors.secondary },
47     ]}
48     onPress={placeholder ? undefined : onPress}
49 >
50     <View style={styles.settingIconContainer}>
51         <Feather name={icon} size={20} color={destructive ? colors.destructive : colors.foreground} />
52     </View>
53     <Text style={[
54         styles.settingTitle,
55         { color: destructive ? colors.destructive : colors.foreground },
56         placeholder && { opacity: 0.5 }
57     ]}>
58         {title}
59     </Text>
60     {placeholder ? (
61         <View style={[styles.comingSoonChip, { backgroundColor: colors.muted }]}>
62             <Text style={[styles.comingSoonText, { color: colors.mutedForeground }]}>Soon</Text>
63         </View>
64     ) : (
65         <Feather name="chevron-right" size={20} color={colors.mutedForeground} />
66     )}
67 </Pressable>
68 );
69
70 return (
71     <ScrollView
72         style={[styles.container, { backgroundColor: colors.background }]}
73         contentContainerStyle={{ paddingTop: insets.top + 20, paddingBottom: insets.bottom + 100 }}
74     >
75         <View style={styles.header}>
76             <Pressable
77                 onPress={() => {
78                     if (Platform.OS === 'web') {
79                         window.alert('Photo upload is coming soon!');
80                     } else {
81                         Alert.alert('Coming Soon', 'Photo upload will be available in a future update.');

```

```

105         <Text style={[styles.statValue, { color: colors.foreground }]}>
106             {dashboard?.totalJars || 0}
107         </Text>
108         <Text style={[styles.statLabel, { color: colors.mutedForeground }]}>Jars</Text>
109     </View>
110     <View style={[styles.statDivider, { backgroundColor: colors.border }] } />
111     <View style={styles.statItem}>
112         <Text style={[styles.statValue, { color: colors.primary }]}>
113             {formatCurrency(dashboard?.personalProgress?.contributedAmountCents || 0)}
114         </Text>
115         <Text style={[styles.statLabel, { color: colors.mutedForeground }]}>Contributed</Text>
116     </View>
117 </View>
118
119     <View style={styles.section}>
120         <Text style={[styles.sectionTitle, { color: colors.mutedForeground }]}>Account</Text>
121         <View style={[styles.settingsGroup, { backgroundColor: colors.card, borderColor:
122 colors.border }]}>
123             <SettingRow icon="user" title="Edit Profile" onPress={() => router.push('/(auth)/profile-
124             <SettingRow icon="credit-card" title="Payment Methods" placeholder />
125             <SettingRow icon="bell" title="Notifications" placeholder />
126         </View>
127     </View>
128
129     <View style={styles.section}>
130         <Text style={[styles.sectionTitle, { color: colors.mutedForeground }]}>Legal & Support</Text>
131         <View style={[styles.settingsGroup, { backgroundColor: colors.card, borderColor:
132         <SettingRow icon="help-circle" title="Help & Support" placeholder />
133         <SettingRow icon="file-text" title="Terms of Service" placeholder />
134         <SettingRow icon="shield" title="Privacy Policy" placeholder />
135     </View>
136 </View>
137
138     <View style={styles.section}>
139         <View style={[styles.settingsGroup, { backgroundColor: colors.card, borderColor:
140         <SettingRow icon="log-out" title="Sign Out" onPress={handleLogout} destructive />
141     </View>
142 </View>
143 </ScrollView>
144 );
145 }
146
147 const styles = StyleSheet.create({
148     container: {
149         flex: 1,
150     },
151     header: {
152         alignItems: 'center',
153         marginBottom: 32,
154     },
155     avatarWrapper: {
156         position: 'relative',
157     },
158     cameraOverlay: {
159         position: 'absolute',
160         bottom: 0,
161         right: 0,
162         width: 28,
163         height: 28,
164         borderRadius: 14,

```

```
164     alignItems: 'center',
165     justifyContent: 'center',
166     borderWidth: 2,
167     borderColor: '#fff',
168   },
169   name: {
170     fontSize: 24,
171     fontWeight: 'bold',
172     marginTop: 16,
173     marginBottom: 4,
174   },
175   email: {
176     fontSize: 16,
177   },
178   statsContainer: {
179     flexDirection: 'row',
180     marginHorizontal: 16,
181     borderRadius: 16,
182     borderWidth: 1,
183     paddingVertical: 20,
184     marginBottom: 32,
185   },
186   statItem: {
187     flex: 1,
188     alignItems: 'center',
189   },
190   statDivider: {
191     width: 1,
192     height: '100%',
193   },
194   statValue: {
195     fontSize: 24,
196     fontWeight: 'bold',
197     marginBottom: 4,
198   },
199   statLabel: {
200     fontSize: 14,
201   },
202   section: {
203     marginBottom: 24,
204     paddingHorizontal: 16,
205   },
206   sectionTitle: {
207     fontSize: 14,
208     fontWeight: '600',
209     marginBottom: 8,
210     textTransform: 'uppercase',
211     letterSpacing: 0.5,
212   },
213   settingsGroup: {
214     borderRadius: 16,
215     borderWidth: 1,
216     overflow: 'hidden',
217   },
218   settingRow: {
219     flexDirection: 'row',
220     alignItems: 'center',
221     padding: 16,
222     borderBottomWidth: StyleSheet.hairlineWidth,
```

```
223   },
224   settingIconContainer: {
225     width: 32,
226     alignItems: 'flex-start',
227   },
228   settingTitle: {
229     flex: 1,
230     fontSize: 16,
231     fontWeight: '500',
232   },
233   comingSoonChip: {
234     paddingHorizontal: 8,
235     paddingVertical: 4,
236     borderRadius: 8,
237   },
238   comingSoonText: {
239     fontSize: 12,
240     fontWeight: '600',
241   },
242   });
243
```

Mobile App — Detail Screens

artifacts/mobile/app/jar/[id].tsx

```
1  import React, { useState } from 'react';
2  import {
3    View,
4    Text,
5    StyleSheet,
6    ScrollView,
7    Pressable,
8    RefreshControl,
9    Platform,
10   Modal,
11   TextInput,
12   Alert,
13   KeyboardAvoidingView,
14 } from 'react-native';
15 import { useLocalSearchParams, useRouter } from 'expo-router';
16 import { useColors } from '@/hooks/useColors';
17 import { useSafeAreaInsets } from 'react-native-safe-area-context';
18 import {
19   useGetJar,
20   useGetJarHealth,
21   useListJarMembers,
22   useListMilestones,
23   useListJarActivity,
24   useListAgreements,
25   useGetContributionSchedule,
26   useCreateInvitation,
27 } from '@workspace/api-client-react';
28 import { ImageBackground } from 'expo-image';
29 import { LinearGradient } from 'expo-linear-gradient';
30 import { Feather } from '@expo/vector-icons';
31 import { SkeletonLoader } from '@components/SkeletonLoader';
32 import { CircularProgress } from '@components/CircularProgress';
33 import { JarHealthBadge } from '@components/JarHealthBadge';
34 import { ProgressBar } from '@components/ProgressBar';
35 import { MemberAvatar } from '@components/MemberAvatar';
36 import { EmptyState } from '@components/EmptyState';
37 import { ScheduleSetupSheet } from '@components/ScheduleSetupSheet';
38 import { useAuth } from '@/contexts/auth-context';
39
40 type Tab = 'Overview' | 'Members' | 'Milestones' | 'Activity' | 'Agreements' | 'Settings';
41 const TABS: Tab[] = ['Overview', 'Members', 'Milestones', 'Activity', 'Agreements', 'Settings'];
42
43 export default function JarDetailScreen() {
44   const { id } = useLocalSearchParams<{ id: string }>();
45   const router = useRouter();
46   const colors = useColors();
47   const insets = useSafeAreaInsets();
48   const { user } = useAuth();
49
50   const [activeTab, setActiveTab] = useState<Tab>('Overview');
51   const [refreshing, setRefreshing] = useState(false);
52   const [scheduleSheetVisible, setScheduleSheetVisible] = useState(false);
```



```

53   const [inviteModalVisible, setInviteModalVisible] = useState(false);
54   const [inviteEmail, setInviteEmail] = useState('');
55   const [inviteSending, setInviteSending] = useState(false);
56   const [inviteError, setInviteError] = useState('');
57
58   const { data: jar, isLoading: jarLoading, refetch: refetchJar } = useGetJar(id!, { query: { enabled: !!
59   const { data: health, refetch: refetchHealth } = useGetJarHealth(id!, { query: { enabled: !!id } });
60   const { data: members, refetch: refetchMembers } = useListJarMembers(id!, { query: { enabled: !!id &&
61   const { data: milestones, refetch: refetchMilestones } = useListMilestones(id!, { query: { enabled: !!
62   const { data: activity, refetch: refetchActivity } = useListJarActivity(id!, undefined, { query:
63   const { data: agreements, refetch: refetchAgreements } = useListAgreements(id!, { query: { enabled: !!
64   const { data: mySchedule, refetch: refetchSchedule } = useGetContributionSchedule(id!, { query:
65
66   const createInvitationMutation = useCreateInvitation();
67
68   const onRefresh = async () => {
69     setRefreshing(true);
70     await refetchJar();
71     await refetchHealth();
72     if (activeTab === 'Overview') await refetchSchedule();
73     if (activeTab === 'Members') await refetchMembers();
74     if (activeTab === 'Milestones') await refetchMilestones();
75     if (activeTab === 'Activity') await refetchActivity();
76     if (activeTab === 'Agreements') await refetchAgreements();
77     setRefreshing(false);
78   };
79
80   const formatCurrency = (cents: number) => {
81     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
82   };
83
84   const renderScheduleCard = () => {
85     const hasSchedule = mySchedule && mySchedule.isActive;
86     const freqLabel = hasSchedule
87       ? mySchedule.frequency === 'weekly'
88         ? 'Weekly'
89         : mySchedule.frequency === 'biweekly'
90         ? 'Every 2 weeks'
91         : mySchedule.frequency === 'monthly'
92         ? 'Monthly'
93         : mySchedule.frequency
94       : null;
95
96     return (
97       <View style={[styles.scheduleCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
98         <View style={styles.scheduleCardHeader}>
99           <View style={styles.scheduleCardLeft}>
100             <Feather name="repeat" size={16} color={colors.primary} style={{ marginRight: 8 }} />
101             <Text style={[styles.scheduleCardTitle, { color: colors.foreground }]}>
102               {hasSchedule ? 'Your Schedule' : 'Contribution Schedule'}
103             </Text>
104           </View>
105           <Pressable
106             onPress={() => setScheduleSheetVisible(true)}
107             style={[styles.scheduleEditBtn, { backgroundColor: colors.secondary }]}
108           >
109             <Feather name={hasSchedule ? 'edit-2' : 'plus'} size={14} color={colors.primary} />
110             <Text style={styles.scheduleEditBtnText, { color: colors.primary }}>
111               {hasSchedule ? 'Edit' : 'Set Up'}

```

```

112         </Text>
113     </Pressable>
114 </View>
115
116     {hasSchedule ? (
117         <View style={styles.scheduleDetails}>
118             {mySchedule.isPaused && (
119                 <View style={[[styles.pausedBadge, { backgroundColor: colors.muted }]}>
120                     <Feather name="pause" size={12} color={colors.mutedForeground} />
121                     <Text style={[[styles.pausedText, { color: colors.mutedForeground }]}>Paused</Text>
122                 </View>
123             )}
124             <View style={styles.scheduleDetailRow}>
125                 <Text style={[[styles.scheduleDetailLabel, { color: colors.mutedForeground }]}>Frequency</
126 Text>
127                 <Text style={[[styles.scheduleDetailValue, { color: colors.foreground }]}>{freqLabel}</Text>
128             </View>
129             <View style={styles.scheduleDetailRow}>
130                 <Text style={[[styles.scheduleDetailLabel, { color: colors.mutedForeground }]}>Amount</Text>
131                 <Text style={[[styles.scheduleDetailValue, { color: colors.foreground }]}>
132                     {formatCurrency(mySchedule.amountCents)}
133                 </Text>
134             </View>
135         ) : (
136             <Text style={[[styles.scheduleEmptyText, { color: colors.mutedForeground }]}>
137                 Set up a recurring schedule so you never miss a contribution.
138             </Text>
139         )}
140     </View>
141 );
142 };
143
144 const renderOverview = () => {
145     if (!jar) return null;
146     return (
147         <View style={styles.tabContent}>
148             <View style={styles.overviewTopRow}>
149                 <CircularProgress
150                     size={120}
151                     progress={jar.percentFunded}
152                     strokeWidth={12}
153                     color={colors.primary}
154                     trackColor={colors.muted}
155                 >
156                     <Text style={[[styles.centerPercent, { color: colors.foreground }]}>{jar.percentFunded}%</
157 Text>
158                     <Text style={[[styles.centerSub, { color: colors.mutedForeground }]}>funded</Text>
159                 </CircularProgress>
160             <View style={styles.overviewStats}>
161                 <Text style={[[styles.overviewAmount, { color: colors.foreground }]}>
162                     {formatCurrency(jar.totalSavedCents)}
163                 </Text>
164                 <Text style={[[styles.overviewTarget, { color: colors.mutedForeground }]}>
165                     of {formatCurrency(jar.goalAmountCents)}
166                 </Text>
167                 {jar.daysRemaining !== null && jar.daysRemaining !== undefined && (
168                     <View style={[[styles.daysLeftChip, { backgroundColor: colors.secondary }]}>
169                         <Text style={[[styles.daysLeftText, { color: colors.primary }]}>{jar.daysRemaining} days
170 left</Text>
171                     </View>

```

```

171         )}
172     </View>
173 </View>
174
175     {health && (
176         <View style={[styles.healthCard, { backgroundColor: colors.card, borderColor: colors.border }]}
177             <JarHealthBadge status={health.status} />
178             <Text style={[styles.healthMessage, { color: colors.foreground }]}>{health.message}</Text>
179         </View>
180     )}
181
182     {renderScheduleCard()}
183
184     <View style={[styles.actionRow, { marginTop: 24 }]}>
185         <Pressable
186             style={[styles.actionButton, { backgroundColor: colors.primary }]}
187             onPress={() => router.push(`/contribution/${jar.id}`)}
188             >
189             <Feather name="plus" size={20} color={colors.primaryForeground} />
190             <Text style={[styles.actionButtonText, { color: colors.primaryForeground }]}>Add Funds</Text>
191         </Pressable>
192     </View>
193 </View>
194 );
195 };
196
197 const handleSendInvite = async () => {
198     if (!inviteEmail.trim()) {
199         setInviteError('Please enter an email address');
200         return;
201     }
202     const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
203     if (!emailRegex.test(inviteEmail.trim())) {
204         setInviteError('Please enter a valid email address');
205         return;
206     }
207     setInviteError('');
208     setInviteSending(true);
209     try {
210         await createInvitationMutation.mutateAsync({
211             jarId: id!,
212             data: { email: inviteEmail.trim().toLowerCase() },
213         });
214         setInviteModalVisible(false);
215         setInviteEmail('');
216         Alert.alert('Invitation sent!', `An invitation has been sent to ${inviteEmail.trim()}.`);
217     } catch (e: any) {
218         const msg = e?.message ?? 'Failed to send invitation. Please try again.';
219         setInviteError(msg);
220     } finally {
221         setInviteSending(false);
222     }
223 };
224
225 const renderMembers = () => {
226     if (!members) return <SkeletonLoader height={200} />;
227     return (
228         <View style={styles.tabContent}>
229             {isOrganizer && (

```

```

230         <Pressable
231             style={[styles.inviteButton, { backgroundColor: colors.primary }]}
232             onPress={() => { setInviteEmail(''); setInviteError(''); setInviteModalVisible(true); }}
233         >
234             <Feather name="user-plus" size={18} color="#fff" />
235             <Text style={styles.inviteButtonText}>Invite Member</Text>
236         </Pressable>
237     )}
238     {members.map(member => (
239         <View key={member.id} style={[styles.memberRow, { backgroundColor: colors.card, borderColor:
240 colors.borderColor,
241             displayName={member.profile?.displayName || 'Unknown'}
242             avatarUrl={member.profile?.avatarUrl}
243             size={48}
244             showStatus
245             status={member.healthStatus}
246         />
247         <View style={styles.memberInfo}>
248             <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 4 }}>
249                 <Text style={styles.memberName, { color: colors.foreground }}>
250 >{member.profile?.displayName}</Text>
251                 <View style={styles.roleChip, { backgroundColor: '#FEF3C7' }}>
252                     <Feather name="award" size={12} color="#92400E" />
253                     <Text style={styles.roleText, { color: '#92400E' }}>Organizer</Text>
254                 </View>
255             </View>
256             <ProgressBar progress={member.percentComplete} height={6} />
257             <View style={{ flexDirection: 'row', justifyContent: 'space-between', marginTop: 8 }}>
258                 <Text style={styles.memberAmount, { color: colors.mutedForeground }}>
259                     {formatCurrency(member.contributedAmountCents)} /
260                     {formatCurrency(member.amountCents)}
261                 </Text>
262                 <Text style={styles.memberPercent, { color: colors.primary }}>
263                     {member.percentComplete}%
264                 </Text>
265             </View>
266         </View>
267     ))}
268     {members.length === 0 && !isOrganizer && <EmptyState icon="users" title="No members yet" />}
269 </View>
270 );
271 };
272
273 const renderMilestones = () => {
274     if (!milestones) return <SkeletonLoader height={200} />;
275     return (
276         <View style={styles.tabContent}>
277             {milestones.map(milestone => (
278                 <View key={milestone.id} style={[styles.milestoneCard, { backgroundColor: colors.card,
279                 borderColor: colors.borderColor,
280                 <View style={styles.milestoneHeader}>
281                     <Text style={styles.milestoneName, { color: colors.foreground }}>{milestone.name}</Text>
282                     <View style={styles.statusChip, { backgroundColor: milestone.status === 'funded' ?
283                     <Text style={styles.statusText, { color: milestone.status === 'funded' ? '#fff' :
284                     colors.foreground... {milestone.status.toUpperCase()}
285                     </Text>
286                 </View>
287                 </View>
288                 <ProgressBar progress={milestone.percentFunded} height={8} />
289                 <View style={styles.milestoneFooter}>
290                     <Text style={styles.milestoneAmount, { color: colors.mutedForeground }}>

```

```

289         {formatCurrency(milestone.allocatedAmountCents)} of
290         {formatCurrency(milestone.targetAmountCents)}
291         {milestone.dueDate && (
292             <Text style={[styles.milestoneDate, { color: colors.mutedForeground }]}>
293                 Due: {new Date(milestone.dueDate).toLocaleDateString()}
294             </Text>
295         )}
296     </View>
297 </View>
298 ))}
299 {milestones.length === 0 && <EmptyState icon="target" title="No milestones yet" />}
300 </View>
301 );
302 };
303
304 const renderActivity = () => {
305     if (!activity) return <SkeletonLoader height={200} />;
306     return (
307         <View style={styles.tabContent}>
308             {activity.map(item => (
309                 <View key={item.id} style={[styles.activityItem, { borderBottomColor: colors.border }]}>
310                     <View style={styles.activityIcon}>
311                         {item.actorAvatarUrl || item.actorName ? (
312                             <MemberAvatar displayName={item.actorName || ''} avatarUrl={item.actorAvatarUrl}
313                             </MemberAvatar>
314                         ) : (
315                             <View style={[styles.systemAvatar, { backgroundColor: colors.muted }]}>
316                                 <Feather name="bell" size={16} color={colors.foreground} />
317                             </View>
318                         )}
319                     </View>
320                     <View style={styles.activityContent}>
321                         <Text style={[styles.activityDesc, { color: colors.foreground }]}>{item.description}</
322                         <Text style={[styles.activityTime, { color: colors.mutedForeground }]}>
323                             {new Date(item.createdAt).toLocaleDateString()}
324                         </Text>
325                     </View>
326                 </View>
327             )}
328             {activity.length === 0 && <EmptyState icon="activity" title="No activity yet" />}
329         </View>
330     );
331 };
332
333 const renderAgreements = () => {
334     if (!agreements || agreements.length === 0) return (
335         <View style={styles.tabContent}>
336             <EmptyState icon="file-text" title="No agreements" />
337         </View>
338     );
339     const agreement = agreements[0]; // Just show the latest
340     return (
341         <View style={styles.tabContent}>
342             <View style={[styles.agreementCard, { backgroundColor: colors.card, borderColor:
343             colors.border }]}>
344                 <Text style={[styles.agreementContent, { color: colors.foreground }]}>{agreement.content}</
345                 <View style={[styles.divider, { backgroundColor: colors.border }]} />
346                 <Text style={[styles.agreementStatus, { color: agreement.myAcceptance ? colors.success :
347                 colors.mutedForeground }]}>
348                     {agreement.myAcceptance ? "You have accepted this agreement." : "You have not accepted this
349                     agreement"}
350                 </Text>
351             </View>
352         </View>
353     );
354 };

```

```

348     </View>
349   );
350 };
351
352 const renderSettings = () => {
353   return (
354     <View style={styles.tabContent}>
355       <Text style={{ color: colors.foreground, fontSize: 16 }}>Settings coming soon</Text>
356     </View>
357   );
358 };
359
360 if (jarLoading && !jar) {
361   return <SkeletonLoader height="100%" />;
362 }
363
364 if (!jar) return <EmptyState icon="alert-circle" title="Jar not found" />;
365
366 const isOrganizer = jar.organizerId === user?.id; // This might be available on members list instead
367
368 const imageSource = jar.coverImageUrl
369   ? { uri: jar.coverImageUrl }
370   : require('../../assets/images/hawaii-cover.jpg');
371
372 return (
373   <View style={[styles.container, { backgroundColor: colors.background }]}>
374     </* Invite Member Modal */>
375     <Modal
376       visible={inviteModalVisible}
377       transparent
378       animationType="slide"
379       onRequestClose={() => setInviteModalVisible(false)}
380     >
381       <KeyboardAvoidingView
382         behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
383         style={styles.modalOverlay}
384       >
385         <Pressable style={styles.modalBackdrop} onPress={() => setInviteModalVisible(false)} />
386         <View style={[styles.modalSheet, { backgroundColor: colors.card }]}>
387           <View style={styles.modalHeader}>
388             <Text style={[styles.modalTitle, { color: colors.foreground }]}>Invite Member</Text>
389             <Pressable onPress={() => setInviteModalVisible(false)}>
390               <Feather name="x" size={24} color={colors.mutedForeground} />
391             </Pressable>
392           </View>
393           <Text style={[styles.modalSubtitle, { color: colors.mutedForeground }]}>
394             Enter the email address of the person you'd like to invite to {jar?.name}.
395           </Text>
396           <View style={[styles.modalInputContainer, { backgroundColor: colors.background, borderColor:
397             colors.mutedForeground, borderStyle: 'solid', borderWidth: 1 }]}>
398             <Text input
399               style={[styles.modalInput, { color: colors.foreground }]}
400               placeholder="friend@example.com"
401               placeholderTextColor={colors.mutedForeground}
402               autoCapitalize="none"
403               keyboardType="email-address"
404               autoFocus
405               value={inviteEmail}
406               onChangeText={text => { setInviteEmail(text); setInviteError(''); }}

```

```

407         onSubmitEditing={handleSendInvite}
408         returnKeyType="send"
409     />
410 </View>
411     {inviteError ? (
412         <Text style={[styles.modalError, { color: '#EF4444' }]}>{inviteError}</Text>
413     ) : null}
414     <Pressable
415         style={[styles.modalSendButton, { backgroundColor: inviteSending ? colors.muted :
416 colors.primary }]}
417         onPress={handleSendInvite}
418         disabled={inviteSending}
419     >
420         {inviteSending ? (
421             <Text style={[styles.modalSendText, { color: colors.mutedForeground }]}>Sending...</Text>
422         ) : (
423             <Feather name="send" size={18} color="#fff" />
424             <Text style={styles.modalSendText}>Send Invitation</Text>
425         )}
426     </Pressable>
427 </View>
428 </KeyboardAvoidingView>
429 </Modal>
430
431 <ScheduleSetupSheet
432     visible={scheduleSheetVisible}
433     jarId={id!}
434     onClose={() => {
435         setScheduleSheetVisible(false);
436         refetchSchedule();
437     }}
438 />
439
440 <View style={[styles.headerImageContainer, { height: insets.top + 160 }]}>
441     <ImageBackground source={imageSource} style={StyleSheet.absoluteFill} contentFit="cover">
442         <LinearGradient colors={['rgba(0,0,0,0.4)', 'rgba(0,0,0,0.8)']}
443         <View style={[styles.headerContent, { paddingTop: insets.top }]}>
444             <View style={styles.navRow}>
445                 <Pressable onPress={() => router.back()} style={styles.backButton}>
446                     <Feather name="arrow-left" size={24} color="#fff" />
447                 </Pressable>
448                 <Pressable style={styles.shareButton}>
449                     <Feather name="share" size={24} color="#fff" />
450                 </Pressable>
451             </View>
452             <View style={styles.titleArea}>
453                 <Text style={styles.headerTitle}>{jar.name}</Text>
454                 {jar.destination && (
455                     <Text style={styles.headerSubtitle}><Feather name="map-pin" size={14} />
456                     {jar.destination}</Text>
457                 )}
458             </View>
459         </ImageBackground>
460     </View>
461
462     <View style={[styles.tabBar, { borderBottomColor: colors.border }]}>
463         <ScrollView horizontal showsHorizontalScrollIndicator={false}
464         contentContainerStyle={styles.tabBarScroll}>
465             {ABS.map(tab => {
466                 if (tab === 'Settings' && !isOrganizer) return null; // Simple check

```

```

466         const isActive = activeTab === tab;
467         return (
468             <Pressable
469                 key={tab}
470                 onPress={() => setActiveTab(tab)}
471                 style={[styles.tabButton, isActive && { borderBottomColor: colors.primary,
472                 borderBottomWidth: 2 }]}
473                 <Text style={[styles.tabText, { color: isActive ? colors.primary :
474                 colors.mutedForeground, fontWeig...
475                 </Text>
476             </Pressable>
477         );
478     }]}
479 </ScrollView>
480 </View>
481
482 <ScrollView
483     style={styles.mainScroll}
484     contentContainerStyle={{ paddingBottom: insets.bottom + 40 }}
485     refreshControl={<RefreshControl refreshing={refreshing} onRefresh={onRefresh}
486     tintColor={colors.primary} />...
487     {activeTab === 'Overview' && renderOverview()}
488     {activeTab === 'Members' && renderMembers()}
489     {activeTab === 'Milestones' && renderMilestones()}
490     {activeTab === 'Activity' && renderActivity()}
491     {activeTab === 'Agreements' && renderAgreements()}
492     {activeTab === 'Settings' && renderSettings()}
493 </ScrollView>
494 </View>
495 );
496 }
497
498 const styles = StyleSheet.create({
499     container: { flex: 1 },
500     headerImageContainer: {
501         width: '100%',
502     },
503     headerContent: {
504         flex: 1,
505         paddingHorizontal: 16,
506         paddingBottom: 16,
507         justifyContent: 'space-between',
508     },
509     navRow: {
510         flexDirection: 'row',
511         justifyContent: 'space-between',
512         alignItems: 'center',
513         paddingVertical: 8,
514     },
515     backButton: { padding: 8, marginLeft: -8 },
516     shareButton: { padding: 8, marginRight: -8 },
517     titleArea: {
518         justifyContent: 'flex-end',
519     },
520     headerTitle: {
521         color: '#fff',
522         fontSize: 32,
523         fontWeight: 'bold',
524         marginBottom: 4,

```



```
525   },
526   headerSubtitle: {
527     color: '#E8F6EF',
528     fontSize: 16,
529     fontWeight: '500',
530   },
531   tabBar: {
532     borderBottomWidth: 1,
533   },
534   tabBarScroll: {
535     paddingHorizontal: 8,
536   },
537   tabButton: {
538     paddingHorizontal: 16,
539     paddingVertical: 14,
540   },
541   tabText: {
542     fontSize: 15,
543   },
544   mainScroll: {
545     flex: 1,
546   },
547   tabContent: {
548     padding: 16,
549   },
550   overviewTopRow: {
551     flexDirection: 'row',
552     alignItems: 'center',
553     marginBottom: 24,
554   },
555   centerPercent: {
556     fontSize: 24,
557     fontWeight: 'bold',
558   },
559   centerSub: {
560     fontSize: 12,
561   },
562   overviewStats: {
563     flex: 1,
564     marginLeft: 24,
565   },
566   overviewAmount: {
567     fontSize: 28,
568     fontWeight: 'bold',
569     marginBottom: 4,
570   },
571   overviewTarget: {
572     fontSize: 16,
573     marginBottom: 12,
574   },
575   daysLeftChip: {
576     alignSelf: 'flex-start',
577     paddingHorizontal: 12,
578     paddingVertical: 6,
579     borderRadius: 16,
580   },
581   daysLeftText: {
582     fontWeight: 'bold',
583     fontSize: 14,
```

```
584   },
585   healthCard: {
586     padding: 16,
587     borderRadius: 16,
588     borderWidth: 1,
589   },
590   healthMessage: {
591     marginTop: 12,
592     fontSize: 15,
593     lineHeight: 22,
594   },
595   // Schedule card
596   scheduleCard: {
597     padding: 16,
598     borderRadius: 16,
599     borderWidth: 1,
600     marginTop: 16,
601   },
602   scheduleCardHeader: {
603     flexDirection: 'row',
604     justifyContent: 'space-between',
605     alignItems: 'center',
606     marginBottom: 12,
607   },
608   scheduleCardLeft: {
609     flexDirection: 'row',
610     alignItems: 'center',
611   },
612   scheduleCardTitle: {
613     fontSize: 15,
614     fontWeight: '700',
615   },
616   scheduleEditBtn: {
617     flexDirection: 'row',
618     alignItems: 'center',
619     gap: 4,
620     paddingHorizontal: 10,
621     paddingVertical: 6,
622     borderRadius: 10,
623   },
624   scheduleEditBtnText: {
625     fontSize: 13,
626     fontWeight: '600',
627   },
628   scheduleDetails: {
629     gap: 8,
630   },
631   scheduleDetailRow: {
632     flexDirection: 'row',
633     justifyContent: 'space-between',
634   },
635   scheduleDetailLabel: {
636     fontSize: 14,
637   },
638   scheduleDetailValue: {
639     fontSize: 14,
640     fontWeight: '600',
641   },
642   pausedBadge: {
```

```
643     flexDirection: 'row',
644     alignItems: 'center',
645     gap: 4,
646     alignSelf: 'flex-start',
647     paddingHorizontal: 8,
648     paddingVertical: 4,
649     borderRadius: 8,
650     marginBottom: 4,
651   },
652   pausedText: {
653     fontSize: 12,
654     fontWeight: '600',
655   },
656   scheduleEmptyText: {
657     fontSize: 14,
658     lineHeight: 20,
659   },
660   actionRow: {
661     flexDirection: 'row',
662     gap: 12,
663   },
664   actionButton: {
665     flex: 1,
666     flexDirection: 'row',
667     height: 56,
668     borderRadius: 12,
669     alignItems: 'center',
670     justifyContent: 'center',
671     gap: 8,
672   },
673   actionButtonText: {
674     fontSize: 16,
675     fontWeight: 'bold',
676   },
677   memberRow: {
678     flexDirection: 'row',
679     padding: 16,
680     borderRadius: 16,
681     borderWidth: 1,
682     marginBottom: 12,
683     alignItems: 'center',
684   },
685   memberInfo: {
686     flex: 1,
687     marginLeft: 16,
688   },
689   memberName: {
690     fontSize: 16,
691     fontWeight: 'bold',
692   },
693   roleChip: {
694     flexDirection: 'row',
695     alignItems: 'center',
696     paddingHorizontal: 6,
697     paddingVertical: 2,
698     borderRadius: 8,
699     marginLeft: 8,
700     gap: 4,
701   },
```

```
702     roleText: {
703       fontSize: 10,
704       fontWeight: 'bold',
705     },
706     memberAmount: {
707       fontSize: 13,
708     },
709     memberPercent: {
710       fontSize: 13,
711       fontWeight: 'bold',
712     },
713     milestoneCard: {
714       padding: 16,
715       borderRadius: 16,
716       borderWidth: 1,
717       marginBottom: 12,
718     },
719     milestoneHeader: {
720       flexDirection: 'row',
721       justifyContent: 'space-between',
722       alignItems: 'center',
723       marginBottom: 12,
724     },
725     milestoneName: {
726       fontSize: 18,
727       fontWeight: 'bold',
728     },
729     statusChip: {
730       paddingHorizontal: 8,
731       paddingVertical: 4,
732       borderRadius: 8,
733     },
734     statusText: {
735       fontSize: 10,
736       fontWeight: 'bold',
737     },
738     milestoneFooter: {
739       flexDirection: 'row',
740       justifyContent: 'space-between',
741       marginTop: 12,
742     },
743     milestoneAmount: {
744       fontSize: 14,
745       fontWeight: '500',
746     },
747     milestoneDate: {
748       fontSize: 14,
749     },
750     activityItem: {
751       flexDirection: 'row',
752       paddingVertical: 12,
753       borderBottomWidth: StyleSheet.hairlineWidth,
754       alignItems: 'center',
755     },
756     activityIcon: {
757       marginRight: 12,
758     },
759     systemAvatar: {
760       width: 32,
```

```
761     height: 32,
762     borderRadius: 16,
763     alignItems: 'center',
764     justifyContent: 'center',
765   },
766   activityContent: {
767     flex: 1,
768   },
769   activityDesc: {
770     fontSize: 14,
771     marginBottom: 4,
772     lineHeight: 20,
773   },
774   activityTime: {
775     fontSize: 12,
776   },
777   agreementCard: {
778     padding: 16,
779     borderRadius: 16,
780     borderWidth: 1,
781   },
782   agreementContent: {
783     fontSize: 15,
784     lineHeight: 24,
785   },
786   divider: {
787     height: 1,
788     marginVertical: 16,
789   },
790   agreementStatus: {
791     fontSize: 14,
792     fontWeight: '600',
793   },
794   // Invite button (Members tab)
795   inviteButton: {
796     flexDirection: 'row',
797     alignItems: 'center',
798     justifyContent: 'center',
799     gap: 8,
800     height: 48,
801     borderRadius: 12,
802     marginBottom: 16,
803   },
804   inviteButtonText: {
805     color: '#fff',
806     fontSize: 16,
807     fontWeight: 'bold',
808   },
809   // Invite modal
810   modalOverlay: {
811     flex: 1,
812     justifyContent: 'flex-end',
813   },
814   modalBackdrop: {
815     ...StyleSheet.absoluteFillObject,
816     backgroundColor: 'rgba(0,0,0,0.45)',
817   },
818   modalSheet: {
819     borderTopLeftRadius: 24,
```

artifacts/mobile/app/jar/[id].tsx (continued)

```
820     borderTopRightRadius: 24,  
821     padding: 24,  
822     paddingBottom: 40,  
823   },  
824   modalHeader: {  
825     flexDirection: 'row',  
826     justifyContent: 'space-between',  
827     alignItems: 'center',  
828     marginBottom: 12,  
829   },  
830   modalTitle: {  
831     fontSize: 22,  
832     fontWeight: 'bold',  
833   },  
834   modalSubtitle: {  
835     fontSize: 14,  
836     lineHeight: 20,  
837     marginBottom: 20,  
838   },  
839   modalInputContainer: {  
840     flexDirection: 'row',  
841     alignItems: 'center',  
842     borderWidth: 1,  
843     borderRadius: 12,  
844     paddingHorizontal: 14,  
845     height: 52,  
846     marginBottom: 8,  
847   },  
848   modalInput: {  
849     flex: 1,  
850     fontSize: 16,  
851   },  
852   modalError: {  
853     fontSize: 13,  
854     marginBottom: 16,  
855   },  
856   modalSendButton: {  
857     flexDirection: 'row',  
858     alignItems: 'center',  
859     justifyContent: 'center',  
860     gap: 8,  
861     height: 52,  
862     borderRadius: 12,  
863     marginTop: 8,  
864   },  
865   modalSendText: {  
866     color: '#fff',  
867     fontSize: 17,  
868     fontWeight: 'bold',  
869   },  
870 });  
871
```

artifacts/mobile/app/contribution/[jarId].tsx

```
1  import React, { useState } from 'react';  
2  import { View, Text, StyleSheet, Pressable, KeyboardAvoidingView, Platform, ScrollView,  
3  import { useLocalSearchParams, useRouter } from 'expo-router';  
4  import { useColors } from '@/hooks/useColors';
```

```

5  import { Feather } from '@expo/vector-icons';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { CurrencyInput } from '@components/CurrencyInput';
8  import { useCreateContribution, useGetJar, useQueryClient } from '@workspace/api-client-react';
9
10 export default function AddContributionScreen() {
11   const { jarId } = useLocalSearchParams<{ jarId: string }>();
12   const router = useRouter();
13   const colors = useColors();
14   const insets = useSafeAreaInsets();
15   const queryClient = useQueryClient();
16
17   const [amountCents, setAmountCents] = useState(0);
18   const [success, setSuccess] = useState(false);
19   const [error, setError] = useState('');
20
21   const { data: jar } = useGetJar(jarId!);
22   const { mutateAsync: createContribution, isPending } = useCreateContribution();
23
24   const handleSubmit = async () => {
25     if (amountCents <= 0) return;
26     try {
27       setError('');
28       await createContribution({
29         data: {
30           amountCents,
31           contributionDate: new Date().toISOString(),
32           // milestoneId and note could be added here
33         }
34       });
35
36       queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
37       queryClient.invalidateQueries({ queryKey: ['/api/jars'] });
38       queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}`] });
39       queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}/activity`] });
40
41       setSuccess(true);
42     } catch (e: any) {
43       setError(e.message || 'Failed to add contribution.');

```

```

64         </Pressable>
65     </View>
66 </View>
67 );
68 }
69
70 return (
71     <KeyboardAvoidingView
72         behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
73         style={[styles.container, { backgroundColor: colors.background }]}
74     >
75         <View style={[styles.inner, { paddingTop: insets.top }]}>
76             <View style={styles.header}>
77                 <Pressable onPress={() => router.back()} style={styles.closeButton}>
78                     <Feather name="x" size={24} color={colors.foreground} />
79                 </Pressable>
80                 <Text style={[styles.headerTitle, { color: colors.foreground }]}>Add Funds</Text>
81                 <View style={{ width: 40 }} />
82             </View>
83
84             <ScrollView contentContainerStyle={styles.content}>
85                 <Text style={[styles.title, { color: colors.foreground }]}>How much are you adding?</Text>
86                 <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
87                     {jar?.name ? `Contributing to ${jar.name}` : ''}
88                 </Text>
89
90                 <View style={styles.inputWrapper}>
91                     <CurrencyInput
92                         value={amountCents}
93                         onChangeCents={setAmountCents}
94                         style={styles.largeInput}
95                         autoFocus
96                     />
97                 </View>
98
99                 {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
100
101             </ScrollView>
102
103             <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
104 colors.background }]}>
105                 <Pressable
106                     style={[
107                         styles.button,
108                         { backgroundColor: amountCents > 0 ? colors.primary : colors.muted },
109                     ]}
110                     onPress={handleSubmit}
111                     disabled={amountCents <= 0 || isPending}
112                 >
113                     {isPending ? (
114                         <ActivityIndicator color={colors.primaryForeground} />
115                     ) : (
116                         <Text style={[styles.buttonText, { color: amountCents > 0 ? colors.primaryForeground :
117 colors.mutedFo... Submit Contribution
118                     </Text>
119                     )}
120                 </Pressable>
121             </View>
122         </KeyboardAvoidingView>

```



```

123     });
124   }
125
126   const styles = StyleSheet.create({
127     container: { flex: 1 },
128     inner: { flex: 1 },
129     header: {
130       flexDirection: 'row',
131       alignItems: 'center',
132       justifyContent: 'space-between',
133       paddingHorizontal: 16,
134       paddingVertical: 12,
135     },
136     closeButton: { width: 40, height: 40, justifyContent: 'center' },
137     headerTitle: { fontSize: 18, fontWeight: '600' },
138     content: { padding: 24, paddingBottom: 40 },
139     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 8, textAlign: 'center' },
140     subtitle: { fontSize: 16, marginBottom: 40, textAlign: 'center' },
141     inputWrapper: {
142       alignItems: 'center',
143     },
144     largeInput: {
145       height: 80,
146     },
147     errorText: { fontSize: 14, textAlign: 'center', marginTop: 16 },
148     footer: {
149       paddingHorizontal: 24,
150       paddingTop: 16,
151       borderTopWidth: 1,
152     },
153     button: {
154       height: 56,
155       borderRadius: 12,
156       alignItems: 'center',
157       justifyContent: 'center',
158     },
159     buttonText: { fontSize: 18, fontWeight: 'bold' },
160     successContent: {
161       flex: 1,
162       alignItems: 'center',
163       justifyContent: 'center',
164       padding: 32,
165     },
166     successTitle: {
167       fontSize: 32,
168       fontWeight: 'bold',
169       color: '#fff',
170       marginBottom: 16,
171     },
172     successSubtitle: {
173       fontSize: 18,
174       color: '#fff',
175       textAlign: 'center',
176       lineHeight: 26,
177       opacity: 0.9,
178     },
179   });
180

```

```
1  import React, { useState } from 'react';
2  import {
3    View,
4    Text,
5    StyleSheet,
6    Pressable,
7    ActivityIndicator,
8    Alert,
9  } from 'react-native';
10 import { useLocalSearchParams, useRouter } from 'expo-router';
11 import { useColors } from '@/hooks/useColors';
12 import { useSafeAreaInsets } from 'react-native-safe-area-context';
13 import {
14   useGetInvitationByToken,
15   useAcceptInvitation,
16   useDeclineInvitation,
17 } from '@workspace/api-client-react';
18 import { useAuth } from '@/contexts/auth-context';
19
20 export default function InviteScreen() {
21   const { token } = useLocalSearchParams<{ token: string }>();
22   const router = useRouter();
23   const colors = useColors();
24   const insets = useSafeAreaInsets();
25   const { isAuthenticated } = useAuth();
26
27   const [accepting, setAccepting] = useState(false);
28   const [declining, setDeclining] = useState(false);
29
30   const {
31     data: invite,
32     isLoading,
33     isError,
34   } = useGetInvitationByToken(token!, { query: { enabled: !!token } });
35
36   const acceptMutation = useAcceptInvitation();
37   const declineMutation = useDeclineInvitation();
38
39   const handleAccept = async () => {
40     if (!invite) return;
41
42     if (!isAuthenticated) {
43       // Redirect to login; after auth they can come back and accept
44       router.push('/(auth)/login');
45       return;
46     }
47
48     setAccepting(true);
49     try {
50       const member = await acceptMutation.mutateAsync({ invitationId: invite.id });
51       router.replace(`/jar/${member.jarId}`);
52     } catch (e: any) {
53       Alert.alert(
54         'Could not accept',
55         e?.message ?? 'Something went wrong. Please try again.',
56       );
57     } finally {
58       setAccepting(false);
59     }
60   }
61 }
```

```

60   };
61
62   const handleDecline = async () => {
63     if (!invite) return;
64
65     if (!isAuthenticated) {
66       router.replace('/(tabs)/jars');
67       return;
68     }
69
70     setDeclining(true);
71     try {
72       await declineMutation.mutateAsync({ invitationId: invite.id });
73       router.replace('/(tabs)/jars');
74     } catch (e: any) {
75       Alert.alert(
76         'Could not decline',
77         e?.message ?? 'Something went wrong. Please try again.',
78       );
79     } finally {
80       setDeclining(false);
81     }
82   };
83
84   if (isLoading) {
85     return (
86       <View
87         style={[
88           styles.container,
89           {
90             backgroundColor: colors.background,
91             justifyContent: 'center',
92             alignItems: 'center',
93           },
94         ]>
95       <ActivityIndicator size="large" color={colors.primary} />
96     </View>
97   );
98 }
99
100
101 if (isError || !invite) {
102   return (
103     <View
104       style={[
105         styles.container,
106         {
107           backgroundColor: colors.background,
108           justifyContent: 'center',
109           alignItems: 'center',
110           padding: 32,
111         },
112       ]>
113     <Text style={[styles.title, { color: colors.foreground, marginBottom: 12 }]}>
114       Invitation not found
115     </Text>
116     <Text
117       style=[

```

```

119         styles.subtitle,
120         { color: colors.mutedForeground, marginBottom: 32 },
121     ]}
122     >
123     This invitation link may have expired or already been used.
124 </Text>
125 <Pressable
126     style={[styles.button, { backgroundColor: colors.primary }]}
127     onPress={() => router.replace('/(tabs)/jars')}
128 >
129     <Text style={styles.acceptText}>Go to My Jars</Text>
130 </Pressable>
131 </View>
132 );
133 }
134
135 return (
136     <View
137         style={[
138             styles.container,
139             { backgroundColor: colors.background, paddingTop: insets.top },
140         ]}
141     >
142         <View style={styles.content}>
143             <Text style={[styles.title, { color: colors.foreground }]}>
144                 You're invited!
145             </Text>
146             <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
147                 You have been invited to join a savings jar.
148             </Text>
149
150             <View
151                 style={[
152                     styles.card,
153                     { backgroundColor: colors.card, borderColor: colors.border },
154                 ]}
155             >
156                 <Text style={[styles.jarName, { color: colors.foreground }]}>
157                     {invite.jar?.name}
158                 </Text>
159                 {invite.jar?.destination ? (
160                     <Text style={[styles.destination, { color: colors.mutedForeground }]}>
161                         {invite.jar.destination}
162                     </Text>
163                 ) : null}
164                 <View style={[styles.divider, { backgroundColor: colors.border }]} />
165                 {invite.jar?.goalAmountCents != null ? (
166                     <Text style={[styles.goalText, { color: colors.foreground }]}>
167                         Total Goal: ${
168                             (invite.jar.goalAmountCents / 100).toLocaleString()
169                         }
170                     </Text>
171                 ) : null}
172                 {invite.contributionTargetCents ? (
173                     <Text
174                         style={[
175                             styles.goalText,
176                             { color: colors.primary, marginTop: 8 },
177                         ]}
178                     >
179                         Your Share: $

```

```

178         {(invite.contributionTargetCents / 100).toLocaleString()}
179     </Text>
180   ) : null}
181 </View>
182
183   {!isAuthenticated && (
184     <View
185       style={[
186         styles.authNotice,
187         { backgroundColor: colors.muted, borderColor: colors.border },
188       ]}
189     >
190       <Text style={[styles.authNoticeText, { color: colors.mutedForeground }]}>
191         You'll need to sign in or create an account before accepting.
192       </Text>
193     </View>
194   )}
195
196   <View style={styles.actions}>
197     <Pressable
198       style={[
199         styles.button,
200         styles.acceptButton,
201         { backgroundColor: colors.primary },
202       ]}
203       onPress={handleAccept}
204       disabled={accepting || declining}
205     >
206       {accepting ? (
207         <ActivityIndicator color="#fff" />
208       ) : (
209         <Text style={styles.acceptText}>
210           {isAuthenticated ? 'Accept Invitation' : 'Sign in to Accept'}
211         </Text>
212       )}
213     </Pressable>
214     <Pressable
215       style={[
216         styles.button,
217         styles.declineButton,
218         { borderColor: colors.border },
219       ]}
220       onPress={handleDecline}
221       disabled={accepting || declining}
222     >
223       {declining ? (
224         <ActivityIndicator color={colors.foreground} />
225       ) : (
226         <Text style={[styles.declineText, { color: colors.foreground }]}>
227           Decline
228         </Text>
229       )}
230     </Pressable>
231   </View>
232 </View>
233 </View>
234 );
235 }
236

```

```
237 const styles = StyleSheet.create({
238   container: { flex: 1 },
239   content: {
240     flex: 1,
241     padding: 24,
242     justifyContent: 'center',
243   },
244   title: {
245     fontSize: 32,
246     fontWeight: 'bold',
247     marginBottom: 12,
248     textAlign: 'center',
249   },
250   subtitle: {
251     fontSize: 16,
252     textAlign: 'center',
253     marginBottom: 40,
254     lineHeight: 24,
255   },
256   card: {
257     padding: 24,
258     borderRadius: 16,
259     borderWidth: 1,
260     marginBottom: 24,
261   },
262   jarName: {
263     fontSize: 24,
264     fontWeight: 'bold',
265     marginBottom: 4,
266     textAlign: 'center',
267   },
268   destination: {
269     fontSize: 16,
270     textAlign: 'center',
271   },
272   divider: {
273     height: 1,
274     marginVertical: 16,
275   },
276   goalText: {
277     fontSize: 18,
278     fontWeight: '600',
279     textAlign: 'center',
280   },
281   authNotice: {
282     borderRadius: 12,
283     borderWidth: 1,
284     padding: 16,
285     marginBottom: 24,
286   },
287   authNoticeText: {
288     fontSize: 14,
289     textAlign: 'center',
290     lineHeight: 20,
291   },
292   actions: {
293     gap: 16,
294   },
295   button: {
```

```
296     height: 56,  
297     borderRadius: 12,  
298     alignItems: 'center',  
299     justifyContent: 'center',  
300   },  
301   acceptButton: {},  
302   acceptText: {  
303     color: '#fff',  
304     fontSize: 18,  
305     fontWeight: 'bold',  
306   },  
307   declineButton: {  
308     borderWidth: 1,  
309     backgroundColor: 'transparent',  
310   },  
311   declineText: {  
312     fontSize: 18,  
313     fontWeight: '600',  
314   },  
315   });  
316
```

Mobile App — Create Jar Wizard

artifacts/mobile/app/create-jar/index.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, ScrollView, Platform } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9  import { CreateJarRequestCategory } from '@workspace/api-client-react/src/generated/api.schemas';
10
11  const CATEGORIES = [
12    { id: 'Vacation', icon: 'sun', label: 'Vacation' },
13    { id: 'Cruise', icon: 'anchor', label: 'Cruise' },
14    { id: 'Wedding', icon: 'heart', label: 'Wedding' },
15    { id: 'Honeymoon', icon: 'map', label: 'Honeymoon' },
16    { id: 'MissionTrip', icon: 'globe', label: 'Mission Trip' },
17    { id: 'Reunion', icon: 'users', label: 'Reunion' },
18    { id: 'Celebration', icon: 'award', label: 'Celebration' },
19    { id: 'Other', icon: 'star', label: 'Other' },
20  ] as const;
21
22  export default function CreateJarStep1() {
23    const colors = useColors();
24    const router = useRouter();
25    const insets = useSafeAreaInsets();
26    const { state, updateState } = useCreateJarContext();
27
28    const isFormValid = state.name && state.category;
29
30    const handleNext = () => {
31      if (isFormValid) {
32        router.push('/create-jar/dates');
33      }
34    };
35
36    return (
37      <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
38        <View style={styles.header}>
39          <Pressable onPress={() => router.back()} style={styles.backButton}>
40            <Feather name="x" size={24} color={colors.foreground} />
41          </Pressable>
42          <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 1 of 8</Text>
43          <View style={{ width: 40 }} />
44        </View>
45        <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
46          <ProgressBar progress={12.5} height={4} />
47        </View>
48
49        <ScrollView contentContainerStyle={styles.content} keyboardShouldPersistTaps="handled">
50          <Text style={[styles.title, { color: colors.foreground }]}>What are you saving for?</Text>
51
52          <View style={styles.inputGroup}>
```



```

53         <Text style={[styles.label, { color: colors.foreground }]}>Jar Name</Text>
54         <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
55             colors.border, borderWidth: 1, borderRadius: 10 }]}>
56             <TextInput
57                 style={[styles.input, { color: colors.foreground }]}
58                 placeholder="e.g., Hawaii 2027"
59                 placeholderTextColor={colors.mutedForeground}
60                 value={state.name || ''}
61                 onChangeText={({text}) => updateState({ name: text })}
62             />
63         </View>
64     </View>
65     <View style={styles.inputGroup}>
66         <Text style={[styles.label, { color: colors.foreground }]}>Category</Text>
67         <View style={styles.categoryGrid}>
68             {CATEGORIES.map((cat) => (
69                 <Pressable
70                     key={cat.id}
71                     style={[
72                         styles.categoryCard,
73                         { backgroundColor: colors.card, borderColor: colors.border },
74                         state.category === cat.id && { backgroundColor: colors.secondary, borderColor:
75                             colors.border }
76                     ]}
77                     onPress={() => updateState({ category: cat.id as CreateJarRequestCategory })}
78                     >
79                         <Feather
80                             name={cat.icon as any}
81                             size={24}
82                             color={state.category === cat.id ? colors.primary : colors.mutedForeground}
83                             style={{ marginBottom: 8 }}
84                         />
85                         <Text
86                             style={[
87                                 styles.categoryText,
88                                 { color: state.category === cat.id ? colors.primary : colors.foreground },
89                             ]}
90                         >
91                             {cat.label}
92                         </Text>
93                     </Pressable>
94                 )})
95         </View>
96     </View>
97     <View style={styles.inputGroup}>
98         <Text style={[styles.label, { color: colors.foreground }]}>Destination (Optional)</Text>
99         <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
100             colors.border, borderWidth: 1, borderRadius: 10 }]}>
101             <TextInput
102                 style={[styles.input, { color: colors.foreground }]}
103                 placeholder="e.g., Maui, Hawaii"
104                 placeholderTextColor={colors.mutedForeground}
105                 value={state.destination || ''}
106                 onChangeText={({text}) => updateState({ destination: text })}
107             />
108         </View>
109     </View>
110     <View style={styles.inputGroup}>
111         <Text style={[styles.label, { color: colors.foreground }]}>Description (Optional)</Text>

```

```

112         <View style={[styles.textAreaContainer, { backgroundColor: colors.card, borderColor:
113         <TextInput
114             style={[styles.textArea, { color: colors.foreground }]}
115             placeholder="What makes this trip special?"
116             placeholderTextColor={colors.mutedForeground}
117             multiline
118             numberOfLines={4}
119             value={state.description || ''}
120             onChangeText={({text}) => updateState({ description: text })}
121         />
122     </View>
123 </View>
124 </ScrollView>
125
126     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
127     <Pressable
128         style=[
129             styles.button,
130             { backgroundColor: isValid ? colors.primary : colors.muted },
131         ]
132         onPress={handleNext}
133         disabled={!isValid}
134     >
135         <Text style={[styles.buttonText, { color: isValid ? colors.primaryForeground :
136         colors.mutedForeground }]>Continue</Text>
137     </Pressable>
138 </View>
139 </View>
140 </View>
141 );
142 }
143
144 const styles = StyleSheet.create({
145     container: { flex: 1 },
146     header: {
147         flexDirection: 'row',
148         alignItems: 'center',
149         justifyContent: 'space-between',
150         paddingHorizontal: 16,
151         paddingVertical: 12,
152     },
153     backButton: { width: 40, height: 40, justifyContent: 'center' },
154     stepText: { fontSize: 14, fontWeight: '600' },
155     content: { padding: 24, paddingBottom: 40 },
156     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 32 },
157     inputGroup: { marginBottom: 24 },
158     label: { fontSize: 16, fontWeight: '600', marginBottom: 12 },
159     inputContainer: {
160         borderWidth: 1,
161         borderRadius: 12,
162         height: 56,
163         paddingHorizontal: 16,
164     },
165     input: { flex: 1, fontSize: 16 },
166     textAreaContainer: {
167         borderWidth: 1,
168         borderRadius: 12,
169         padding: 16,
170         minHeight: 120,

```

artifacts/mobile/app/create-jar/index.tsx (continued)

```
171   },
172   textArea: { flex: 1, fontSize: 16, textAlignVertical: 'top' },
173   categoryGrid: {
174     flexDirection: 'row',
175     flexWrap: 'wrap',
176     gap: 12,
177   },
178   categoryCard: {
179     width: '48%',
180     borderWidth: 1,
181     borderRadius: 12,
182     padding: 16,
183     alignItems: 'center',
184     justifyContent: 'center',
185   },
186   categoryText: { fontSize: 14, fontWeight: '600' },
187   footer: {
188     paddingHorizontal: 24,
189     paddingTop: 16,
190     borderTopWidth: 1,
191   },
192   button: {
193     height: 56,
194     borderRadius: 12,
195     alignItems: 'center',
196     justifyContent: 'center',
197   },
198   buttonText: { fontSize: 18, fontWeight: 'bold' },
199 });
200
```

artifacts/mobile/app/create-jar/dates.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9  import { DateInput } from '@components/DateInput';
10
11  export default function CreateJarStep2() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { state, updateState } = useCreateJarContext();
16
17    // Parse stored YYYY-MM-DD at noon local time to avoid UTC-shift off-by-one
18    const parseLocal = (s: string) => { const [y, m, d] = s.split('-').map(Number); return new Date(y, m -
19    // Store as YYYY-MM-DD using local calendar date (not UTC)
20    const toLocalISO = (d: Date) => `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-
21    #format(d, 'MM-DD')
22    const startDate = state.startDate ? parseLocal(state.startDate) : undefined;
23    const endDate = state.endDate ? parseLocal(state.endDate) : undefined;
24    const targetDate = state.targetDate ? parseLocal(state.targetDate) : undefined;
25
26    const isValid = !!targetDate && (!startDate || targetDate <= startDate);
```

```

27
28   const handleNext = () => {
29     if (isFormValid) {
30       router.push('/create-jar/goal');
31     }
32   };
33
34   return (
35     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
36       <View style={styles.header}>
37         <Pressable onPress={() => router.back()} style={styles.backButton}>
38           <Feather name="arrow-left" size={24} color={colors.foreground} />
39         </Pressable>
40         <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 2 of 8</Text>
41         <View style={{ width: 40 }} />
42       </View>
43       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
44         <ProgressBar progress={25} height={4} />
45       </View>
46
47       <ScrollView contentContainerStyle={styles.content}>
48         <Text style={[styles.title, { color: colors.foreground }]}>When is the trip?</Text>
49
50         <View style={styles.inputGroup}>
51           <Text style={[styles.label, { color: colors.foreground }]}>Trip Start Date (Optional)</Text>
52           <DateInput
53             value={startDate}
54             onChange={(d) => updateState({ startDate: toLocalISO(d) })}
55             placeholder="Select trip start date"
56           />
57         </View>
58
59         <View style={styles.inputGroup}>
60           <Text style={[styles.label, { color: colors.foreground }]}>Trip End Date (Optional)</Text>
61           <DateInput
62             value={endDate}
63             onChange={(d) => updateState({ endDate: toLocalISO(d) })}
64             placeholder="Select trip end date"
65           />
66         </View>
67
68         <View style={[styles.divider, { backgroundColor: colors.border }]} />
69
70         <View style={styles.inputGroup}>
71           <Text style={[styles.label, { color: colors.foreground }]}>Savings Target Date</Text>
72           <Text style={[styles.subLabel, { color: colors.mutedForeground }]}>
73             When do you want to have all the money saved by? We recommend 30-60 days before the trip.
74           </Text>
75           <DateInput
76             value={targetDate}
77             onChange={(d) => updateState({ targetDate: toLocalISO(d) })}
78             placeholder="Select target date"
79           />
80           {targetDate && startDate && targetDate > startDate && (
81             <Text style={[styles.errorText, { color: colors.destructive }]}>
82               Target date must be before the trip start date.
83             </Text>
84           )}
85         </View>

```

artifacts/mobile/app/create-jar/dates.tsx (continued)

```
86
87     </ScrollView>
88
89     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
90 colors.background,
91     style=[
92       styles.button,
93       { backgroundColor: isFormValid ? colors.primary : colors.muted },
94     ]}
95     onPress={handleNext}
96     disabled={!isFormValid}
97   >
98     <Text style={[styles.buttonText, { color: isFormValid ? colors.primaryForeground :
99     'Continue'
100   </Text>
101   </Pressable>
102   </View>
103 </View>
104 );
105 }
106
107 const styles = StyleSheet.create({
108   container: { flex: 1 },
109   header: {
110     flexDirection: 'row',
111     alignItems: 'center',
112     justifyContent: 'space-between',
113     paddingHorizontal: 16,
114     paddingVertical: 12,
115   },
116   backButton: { width: 40, height: 40, justifyContent: 'center' },
117   stepText: { fontSize: 14, fontWeight: '600' },
118   content: { padding: 24, paddingBottom: 40 },
119   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 32 },
120   inputGroup: { marginBottom: 24 },
121   label: { fontSize: 16, fontWeight: '600', marginBottom: 8 },
122   subLabel: { fontSize: 14, marginBottom: 12, lineHeight: 20 },
123   divider: { height: 1, marginVertical: 16 },
124   errorText: { marginTop: 8, fontSize: 14 },
125   footer: {
126     paddingHorizontal: 24,
127     paddingTop: 16,
128     borderTopWidth: 1,
129   },
130   button: {
131     height: 56,
132     borderRadius: 12,
133     alignItems: 'center',
134     justifyContent: 'center',
135   },
136   buttonText: { fontSize: 18, fontWeight: 'bold' },
137 });
138
```

artifacts/mobile/app/create-jar/goal.tsx

```
1 import React from 'react';
2 import { View, Text, StyleSheet, Pressable, KeyboardAvoidingView, Platform, ScrollView } from 'react-
3 import { useRouter } from 'expo-router';
```

```

4   import { useColors } from '@hooks/useColors';
5   import { useCreateJarContext } from '@contexts/create-jar-context';
6   import { Feather } from '@expo/vector-icons';
7   import { useSafeAreaInsets } from 'react-native-safe-area-context';
8   import { ProgressBar } from '@components/ProgressBar';
9   import { CurrencyInput } from '@components/CurrencyInput';
10
11  export default function CreateJarStep3() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { state, updateState } = useCreateJarContext();
16
17    const amountCents = state.goalAmountCents || 0;
18    const isFormValid = amountCents > 0;
19
20    const handleNext = () => {
21      if (isFormValid) {
22        router.push('/create-jar/milestones');
23      }
24    };
25
26    return (
27      <KeyboardAvoidingView
28        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
29        style={[styles.container, { backgroundColor: colors.background }]}
30      >
31        <View style={[styles.inner, { paddingTop: insets.top }]}>
32          <View style={styles.header}>
33            <Pressable onPress={() => router.back()} style={styles.backButton}>
34              <Feather name="arrow-left" size={24} color={colors.foreground} />
35            </Pressable>
36            <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 3 of 8</Text>
37            <View style={{ width: 40 }} />
38          </View>
39          <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
40            <ProgressBar progress={37.5} height={4} />
41          </View>
42
43          <ScrollView contentContainerStyle={styles.content}>
44            <Text style={[styles.title, { color: colors.foreground }]}>What's your total goal?</Text>
45            <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
46              Don't worry if you don't know the exact amount yet. You can always adjust this later or
47            </Text>
48
49            <View style={styles.inputWrapper}>
50              <CurrencyInput
51                value={amountCents}
52                onChangeCents={(cents) => updateState({ goalAmountCents: cents })}
53                style={styles.largeInput}
54                autoFocus
55              />
56            </View>
57          </ScrollView>
58
59          <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
60            colors.background,
61            style=[
62              styles.button,

```

artifacts/mobile/app/create-jar/goal.tsx (continued)

```
63         { backgroundColor: isValid ? colors.primary : colors.muted },
64       ]}
65       onPress={handleNext}
66       disabled={!isValid}
67     >
68       <Text style={[styles.buttonText, { color: isValid ? colors.primaryForeground :
69         colors.muted }]>Continue
70     </Text>
71   </Pressable>
72 </View>
73 </View>
74 </KeyboardAvoidingView>
75 );
76 }
77
78 const styles = StyleSheet.create({
79   container: { flex: 1 },
80   inner: { flex: 1 },
81   header: {
82     flexDirection: 'row',
83     alignItems: 'center',
84     justifyContent: 'space-between',
85     paddingHorizontal: 16,
86     paddingVertical: 12,
87   },
88   backButton: { width: 40, height: 40, justifyContent: 'center' },
89   stepText: { fontSize: 14, fontWeight: '600' },
90   content: { padding: 24, paddingBottom: 40 },
91   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
92   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 40 },
93   inputWrapper: {
94     alignItems: 'center',
95   },
96   largeInput: {
97     height: 80,
98   },
99   footer: {
100     paddingHorizontal: 24,
101     paddingTop: 16,
102     borderTopWidth: 1,
103   },
104   button: {
105     height: 56,
106     borderRadius: 12,
107     alignItems: 'center',
108     justifyContent: 'center',
109   },
110   buttonText: { fontSize: 18, fontWeight: 'bold' },
111 });
112
```

artifacts/mobile/app/create-jar/milestones.tsx

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, Pressable, ScrollView, Modal, TextInput, KeyboardAvoidingView,
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { useCreateJarContext } from '@contexts/create-jar-context';
6 import { Feather } from '@expo/vector-icons';
```

```

7   import { useSafeAreaInsets } from 'react-native-safe-area-context';
8   import { ProgressBar } from '@components/ProgressBar';
9   import { CurrencyInput } from '@components/CurrencyInput';
10
11   const SUGGESTIONS = ['Flights', 'Lodging', 'Activities', 'Food', 'Emergency Buffer'];
12
13   export default function CreateJarStep4() {
14     const colors = useColors();
15     const router = useRouter();
16     const insets = useSafeAreaInsets();
17     const { state, updateState } = useCreateJarContext();
18
19     const [milestones, setMilestones] = useState<{name: string, targetAmountCents: number}
20     []>(state.milestones || []);
21     const [modalVisible, setModalVisible] = useState(false);
22     const [newName, setNewName] = useState('');
23     const [newAmount, setNewAmount] = useState(0);
24
25     const goalAmount = state.goalAmountCents || 0;
26     const totalAllocated = milestones.reduce((sum, m) => sum + m.targetAmountCents, 0);
27
28     const handleAddMilestone = () => {
29       if (newName && newAmount > 0) {
30         setMilestones([...milestones, { name: newName, targetAmountCents: newAmount }]);
31         setNewName('');
32         setNewAmount(0);
33         setModalVisible(false);
34       }
35     };
36
37     const removeMilestone = (index: number) => {
38       const next = [...milestones];
39       next.splice(index, 1);
40       setMilestones(next);
41     };
42
43     const handleNext = () => {
44       updateState({ milestones });
45       router.push('/create-jar/members');
46     };
47
48     const formatCurrency = (cents: number) => {
49       return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
50     };
51
52     return (
53       <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
54         <View style={styles.header}>
55           <Pressable onPress={() => router.back()} style={styles.backButton}>
56             <Feather name="arrow-left" size={24} color={colors.foreground} />
57           </Pressable>
58           <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 4 of 8</Text>
59           <View style={{ width: 40 }} />
60         </View>
61         <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
62           <ProgressBar progress={50} height={4} />
63         </View>
64
65       <ScrollView contentContainerStyle={styles.content}>

```



```

66         <Text style={[styles.title, { color: colors.foreground }]}>Break it down</Text>
67         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
68             Milestones help track specific expenses like flights or lodging.
69         </Text>
70
71         <View style={[styles.summaryCard, { backgroundColor: colors.secondary }]}>
72             <View style={styles.summaryRow}>
73                 <Text style={[styles.summaryLabel, { color: colors.secondaryForeground }]}>Total Goal</Text>
74                 <Text style={[styles.summaryValue, { color: colors.secondaryForeground }]}
75                     {formatCurrency(goalAmount)}</Text>
76             </View>
77             <View style={styles.summaryRow}>
78                 <Text style={[styles.summaryLabel, { color: colors.secondaryForeground }]}>Allocated</Text>
79                 <Text style={[styles.summaryValue, { color: colors.secondaryForeground }]}
80                     {formatCurrency(totalAllocated)}</Text>
81             </View>
82             {totalAllocated > goalAmount && (
83                 <Text style={[styles.errorText, { color: colors.destructive, marginTop: 8 }]}>
84                     You've allocated more than your total goal.
85                 </Text>
86             )}
87         </View>
88
89         <View style={styles.suggestionsContainer}>
90             <Text style={[styles.sectionTitle, { color: colors.foreground }]}>Quick Add</Text>
91             <ScrollView horizontal showsHorizontalScrollIndicator={false}
92                 contentContainerStyle={styles.suggestionsContainer}>
93                 {suggestions.map((s) => (
94                     <Pressable
95                         key={s}
96                         style={[styles.suggestionChip, { backgroundColor: colors.card, borderColor:
97                             colors.border }]}
98                         onPress={() => {
99                             setNewName(s);
100                             setNewAmount(0);
101                             setModalVisible(true);
102                         }}
103                     >
104                         <Feather name="plus" size={16} color={colors.primary} />
105                         <Text style={[styles.suggestionText, { color: colors.foreground }]}>{s}</Text>
106                     </Pressable>
107                 ))}
108             </ScrollView>
109         </View>
110
111         <View style={styles.listContainer}>
112             {milestones.map((m, i) => (
113                 <View key={i} style={styles.milestoneRow, { backgroundColor: colors.card, borderColor:
114                     colors.border }}>
115                     <View style={styles.milestoneInfo}>
116                         <Text style={[styles.milestoneName, { color: colors.foreground }]}>{m.name}</Text>
117                         <Text style={[styles.milestoneAmount, { color: colors.mutedForeground }]}
118                             {formatCurrency(m.amount)}</Text>
119                     </View>
120                     <Pressable onPress={() => removeMilestone(i)} style={styles.removeButton}>
121                         <Feather name="trash-2" size={20} color={colors.destructive} />
122                     </Pressable>
123                 </View>
124             ))}
125
126         <Pressable
127             style={styles.addCustomButton, { borderColor: colors.border }}
128             onPress={() => {
129                 setNewName('');
130                 setNewAmount(0);

```

```

125         setModalVisible(true);
126     }}
127 >
128     <Feather name="plus" size={20} color={colors.primary} />
129     <Text style={[styles.addCustomText, { color: colors.primary }]}>Add Custom Milestone</Text>
130 </Pressable>
131 </View>
132 </ScrollView>
133
134 <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
135     <Pressable
136         style={[styles.button, { backgroundColor: colors.primary }]}
137         onPress={handleNext}
138     >
139         <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
140             Continue
141         </Text>
142     </Pressable>
143 </View>
144
145 <Modal visible={modalVisible} transparent animationType="slide">
146     <View style={[styles.modalOverlay, { backgroundColor: 'rgba(0,0,0,0.5)' }]}>
147         <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
148         style={styles.modalContent}>
149             <View style={[styles.modalInner, { backgroundColor: colors.background, paddingBottom:
150                 insets.bottom, paddingTop: 10}]>
151                 <View style={styles.modalHeader}>
152                     <Text style={[styles.modalTitle, { color: colors.foreground }]}>Add Milestone</Text>
153                     <Pressable onPress={() => setModalVisible(false)}>
154                         <Feather name="x" size={24} color={colors.foreground} />
155                     </Pressable>
156                 </View>
157                 <View style={styles.inputGroup}>
158                     <Text style={[styles.label, { color: colors.foreground }]}>Name</Text>
159                     <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
160                         colors.border, padding: 5}]>
161                         <TextInput
162                             style={[styles.input, { color: colors.foreground }]}
163                             value={newName}
164                             onChangeText={setNewName}
165                             placeholder="e.g., Flights"
166                         />
167                     </View>
168                 </View>
169                 <View style={styles.inputGroup}>
170                     <Text style={[styles.label, { color: colors.foreground }]}>Target Amount</Text>
171                     <CurrencyInput value={newAmount} onChangeCents={setNewAmount} />
172                 </View>
173                 <Pressable
174                     style={[
175                         styles.button,
176                         { backgroundColor: newName && newAmount > 0 ? colors.primary : colors.muted,
177                         }
178                     ]>
179                     <Text style={[styles.buttonText, { color: newName && newAmount > 0 ?
180                         colors.primaryForeground : colors.mutedForeground }]}>Add Milestone
181                 </Text>
182             </Text>
183

```

```

184         </Pressable>
185     </View>
186 </KeyboardAvoidingView>
187 </View>
188 </Modal>
189 </View>
190 );
191 }
192
193 const styles = StyleSheet.create({
194     container: { flex: 1 },
195     header: {
196         flexDirection: 'row',
197         alignItems: 'center',
198         justifyContent: 'space-between',
199         paddingHorizontal: 16,
200         paddingVertical: 12,
201     },
202     backButton: { width: 40, height: 40, justifyContent: 'center' },
203     stepText: { fontSize: 14, fontWeight: '600' },
204     content: { padding: 24, paddingBottom: 40 },
205     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
206     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 24 },
207     summaryCard: {
208         padding: 16,
209         borderRadius: 12,
210         marginBottom: 32,
211     },
212     summaryRow: {
213         flexDirection: 'row',
214         justifyContent: 'space-between',
215         marginBottom: 8,
216     },
217     summaryLabel: { fontSize: 16, fontWeight: '500' },
218     summaryValue: { fontSize: 16, fontWeight: 'bold' },
219     errorText: { fontSize: 14, fontWeight: '600' },
220     suggestionsContainer: { marginBottom: 24 },
221     sectionTitle: { fontSize: 16, fontWeight: '600', marginBottom: 12 },
222     suggestionsScroll: { gap: 12, paddingRight: 24 },
223     suggestionChip: {
224         flexDirection: 'row',
225         alignItems: 'center',
226         paddingHorizontal: 12,
227         paddingVertical: 8,
228         borderRadius: 20,
229         borderWidth: 1,
230         gap: 6,
231     },
232     suggestionText: { fontSize: 14, fontWeight: '500' },
233     listContainer: { gap: 12 },
234     milestoneRow: {
235         flexDirection: 'row',
236         justifyContent: 'space-between',
237         alignItems: 'center',
238         padding: 16,
239         borderWidth: 1,
240         borderRadius: 12,
241     },
242     milestoneInfo: { flex: 1 },

```

```
243 milestoneName: { fontSize: 16, fontWeight: '600', marginBottom: 4 },
244 milestoneAmount: { fontSize: 14 },
245 removeButton: { padding: 8 },
246 addCustomButton: {
247   flexDirection: 'row',
248   alignItems: 'center',
249   justifyContent: 'center',
250   padding: 16,
251   borderWidth: 1,
252   borderStyle: 'dashed',
253   borderRadius: 12,
254   gap: 8,
255   marginTop: 8,
256 },
257 addCustomText: { fontSize: 16, fontWeight: '600' },
258 footer: {
259   paddingHorizontal: 24,
260   paddingTop: 16,
261   borderTopWidth: 1,
262 },
263 button: {
264   height: 56,
265   borderRadius: 12,
266   alignItems: 'center',
267   justifyContent: 'center',
268 },
269 buttonText: { fontSize: 18, fontWeight: 'bold' },
270 modalOverlay: {
271   flex: 1,
272   justifyContent: 'flex-end',
273 },
274 modalContent: {
275   width: '100%',
276 },
277 modalInner: {
278   borderTopLeftRadius: 24,
279   borderTopRightRadius: 24,
280   padding: 24,
281 },
282 modalHeader: {
283   flexDirection: 'row',
284   justifyContent: 'space-between',
285   alignItems: 'center',
286   marginBottom: 24,
287 },
288 modalTitle: { fontSize: 20, fontWeight: 'bold' },
289 inputGroup: { marginBottom: 20 },
290 label: { fontSize: 14, fontWeight: '600', marginBottom: 8 },
291 inputContainer: {
292   borderWidth: 1,
293   borderRadius: 12,
294   height: 56,
295   paddingHorizontal: 16,
296   justifyContent: 'center',
297 },
298 input: { flex: 1, fontSize: 16 },
299 });
300
```

```

1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView, TextInput, Switch } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9
10 export default function CreateJarStep5() {
11   const colors = useColors();
12   const router = useRouter();
13   const insets = useSafeAreaInsets();
14   const { state, updateState } = useCreateJarContext();
15
16   const [invitees, setInvitees] = useState<{email: string}[]>(state.invitees || []);
17   const [newEmail, setNewEmail] = useState('');
18   const [equalSplit, setEqualSplit] = useState(true);
19
20   const goalAmount = state.goalAmountCents || 0;
21   // total people = creator + invitees
22   const totalPeople = invitees.length + 1;
23   const splitAmount = goalAmount / totalPeople;
24
25   const handleAddInvitee = () => {
26     if (newEmail && newEmail.includes('@')) {
27       setInvitees([...invitees, { email: newEmail }]);
28       setNewEmail('');
29     }
30   };
31
32   const removeInvitee = (index: number) => {
33     const next = [...invitees];
34     next.splice(index, 1);
35     setInvitees(next);
36   };
37
38   const handleNext = () => {
39     updateState({ invitees });
40     router.push('/create-jar/plan');
41   };
42
43   const formatCurrency = (cents: number) => {
44     return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
45   };
46
47   return (
48     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
49       <View style={styles.header}>
50         <Pressable onPress={() => router.back()} style={styles.backButton}>
51           <Feather name="arrow-left" size={24} color={colors.foreground} />
52         </Pressable>
53         <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 5 of 8</Text>
54         <View style={{ width: 40 }} />
55       </View>
56       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
57         <ProgressBar progress={62.5} height={4} />
58       </View>
59

```

```

60     <ScrollView contentContainerStyle={styles.content}>
61         <Text style={[styles.title, { color: colors.foreground }]}>Who's coming?</Text>
62         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
63             Invite friends or family to save together. You can always add more people later.
64         </Text>
65
66         <View style={styles.inputGroup}>
67             <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
68 colors.borderColor }]}>
69                 <Text style={styles.input, { color: colors.foreground }}>
70                     value={newEmail}
71                     onChangeText={setNewEmail}
72                     placeholder="Email address"
73                     placeholderTextColor={colors.mutedForeground}
74                     autoCapitalize="none"
75                     keyboardType="email-address"
76                 </Text>
77                 <Pressable
78                     style={[styles.addButton, { backgroundColor: newEmail.includes('@') ? colors.primary :
79 colors.borderColor }]}
80                     onPress={handleAddInvitee}
81                     disabled={!newEmail.includes('@')}
82                 >
83                     <Text style={[styles.addButtonText, { color: newEmail.includes('@') ?
84 colors.borderColor }]}>Add</Text>
85                 </Pressable>
86             </View>
87         </View>
88
89         {invitees.length > 0 && (
90             <View style={styles.splitToggleRow}>
91                 <View>
92                     <Text style={[styles.splitTitle, { color: colors.foreground }]}>Split equally?</Text>
93                     <Text style={[styles.splitSubtitle, { color: colors.mutedForeground }]}>
94                         Each person aims for {formatCurrency(splitAmount)}
95                     </Text>
96                 </View>
97                 <Switch
98                     value={equalSplit}
99                     onChange={setEqualSplit}
100                     trackColor={{ false: colors.muted, true: colors.primary }}
101                 </Switch>
102             </View>
103         )}
104
105         <View style={styles.listContainer}>
106             <Text style={[styles.listHeader, { color: colors.foreground }]}>Group Members ({totalPeople})</Text>
107
108             <View style={[styles.memberRow, { backgroundColor: colors.card, borderColor: colors.borderColor }]}>
109                 <View style={styles.avatarPlaceholder, { backgroundColor: colors.primary }}>
110                     <Text style={styles.avatarText}>You</Text>
111                 </View>
112                 <Text style={styles.memberEmail, { color: colors.foreground }}>You (Organizer)</Text>
113                 {equalSplit && goalAmount > 0 && (
114                     <Text style={[styles.splitAmount, { color: colors.mutedForeground }]}>
115                         {formatCurrency(splitAmount)}
116                     </Text>
117                 )}
118             </View>
119
120             {invitees.map((invitee, i) => (
121                 <View key={i} style={styles.memberRow, { backgroundColor: colors.card, borderColor:
122 colors.borderColor }]}>
123                     <View style={styles.avatarPlaceholder, { backgroundColor: colors.muted }}>

```

```

119         <Feather name="user" size={16} color={colors.mutedForeground} />
120     </View>
121     <Text style={[[styles.memberEmail, { color: colors.foreground }]]}>{invitee.email}</Text>
122     {equalSplit && goalAmount > 0 && (
123         <Text style={[[styles.splitAmount, { color: colors.mutedForeground }]]}
124     >{formatCurrency(splitAmount)}...
125         <Pressable onPress={() => removeInvitee(i)} style={styles.removeBtn}>
126             <Feather name="x" size={20} color={colors.mutedForeground} />
127         </Pressable>
128     </View>
129     )]}
130 </View>
131 </ScrollView>
132
133 <View style={[[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
134 colors.background}
135     style={[[styles.button, { backgroundColor: colors.primary }]]}
136     onPress={handleNext}
137     >
138         <Text style={[[styles.buttonText, { color: colors.primaryForeground }]]}>
139             Continue
140         </Text>
141     </Pressable>
142 </View>
143 </View>
144 );
145 }
146
147 const styles = StyleSheet.create({
148     container: { flex: 1 },
149     header: {
150         flexDirection: 'row',
151         alignItems: 'center',
152         justifyContent: 'space-between',
153         paddingHorizontal: 16,
154         paddingVertical: 12,
155     },
156     backButton: { width: 40, height: 40, justifyContent: 'center' },
157     stepText: { fontSize: 14, fontWeight: '600' },
158     content: { padding: 24, paddingBottom: 40 },
159     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
160     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
161     inputGroup: { marginBottom: 24 },
162     inputContainer: {
163         flexDirection: 'row',
164         alignItems: 'center',
165         borderWidth: 1,
166         borderRadius: 12,
167         height: 56,
168         paddingLeft: 16,
169         paddingRight: 8,
170     },
171     input: { flex: 1, fontSize: 16, height: '100%' },
172     addButton: {
173         paddingHorizontal: 16,
174         paddingVertical: 8,
175         borderRadius: 8,
176     },
177     addButtonText: { fontWeight: '600', fontSize: 14 },

```

artifacts/mobile/app/create-jar/members.tsx (continued)

```
178   splitToggleRow: {
179     flexDirection: 'row',
180     justifyContent: 'space-between',
181     alignItems: 'center',
182     marginBottom: 24,
183     padding: 16,
184     borderRadius: 12,
185     borderWidth: 1,
186     borderColor: 'transparent', // you can apply border color if needed
187   },
188   splitTitle: { fontSize: 16, fontWeight: 'bold', marginBottom: 4 },
189   splitSubtitle: { fontSize: 14 },
190   listContainer: { gap: 12 },
191   listHeader: { fontSize: 18, fontWeight: 'bold', marginBottom: 8 },
192   memberRow: {
193     flexDirection: 'row',
194     alignItems: 'center',
195     padding: 12,
196     borderWidth: 1,
197     borderRadius: 12,
198   },
199   avatarPlaceholder: {
200     width: 32,
201     height: 32,
202     borderRadius: 16,
203     alignItems: 'center',
204     justifyContent: 'center',
205     marginRight: 12,
206   },
207   avatarText: { fontSize: 10, color: '#fff', fontWeight: 'bold' },
208   memberEmail: { flex: 1, fontSize: 14, fontWeight: '500' },
209   splitAmount: { fontSize: 14, fontWeight: '600', marginRight: 8 },
210   removeBtn: { padding: 4 },
211   footer: {
212     paddingHorizontal: 24,
213     paddingTop: 16,
214     borderTopWidth: 1,
215   },
216   button: {
217     height: 56,
218     borderRadius: 12,
219     alignItems: 'center',
220     justifyContent: 'center',
221   },
222   buttonText: { fontSize: 18, fontWeight: 'bold' },
223 });
224
```

artifacts/mobile/app/create-jar/plan.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9
```



```

10 export default function CreateJarStep6() {
11   const colors = useColors();
12   const router = useRouter();
13   const insets = useSafeAreaInsets();
14   const { state, updateState } = useCreateJarContext();
15
16   const [frequency, setFrequency] = useState<'weekly' | 'biweekly' | 'monthly'>('monthly');
17
18   const goalAmount = state.goalAmountCents || 0;
19   const totalPeople = (state.invitees?.length || 0) + 1;
20   const splitAmount = goalAmount / totalPeople;
21
22   // Let's assume there's 6 months to target date for mockup calculation
23   const months = 6;
24   let paymentAmount = splitAmount / months;
25   if (frequency === 'weekly') paymentAmount = splitAmount / (months * 4);
26   if (frequency === 'biweekly') paymentAmount = splitAmount / (months * 2);
27
28   const handleNext = () => {
29     updateState({ contributionPlan: { frequency, amountCents: paymentAmount } });
30     router.push('/create-jar/rules');
31   };
32
33   const formatCurrency = (cents: number) => {
34     return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
35   };
36
37   const OptionCard = ({ type, title, subtitle }: { type: any, title: string, subtitle: string }) => {
38     const isSelected = frequency === type;
39     return (
40       <Pressable
41         style={[
42           styles.optionCard,
43           { backgroundColor: colors.card, borderColor: isSelected ? colors.primary : colors.border },
44           isSelected && { backgroundColor: colors.secondary }
45         ]}
46         onPress={() => setFrequency(type)}
47       >
48         <View style={styles.optionContent}>
49           <Text style={[styles.optionTitle, { color: isSelected ? colors.primary : colors.foreground }]}>
50             >{title}</Text>
51           <Text style={[styles.optionSubtitle, { color: colors.mutedForeground }]}>{subtitle}</Text>
52         </View>
53         <View style={[
54           styles.radio,
55           { borderColor: isSelected ? colors.primary : colors.mutedForeground },
56           isSelected && { backgroundColor: colors.primary }
57         ]]>
58           {isSelected && <Feather name="check" size={14} color="#fff" />}
59         </View>
60       </Pressable>
61     );
62   };
63
64   return (
65     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
66       <View style={styles.header}>
67         <Pressable onPress={() => router.back()} style={styles.backButton}>
68           <Feather name="arrow-left" size={24} color={colors.foreground} />
69         </Pressable>

```

```

69         <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 6 of 8</Text>
70         <View style={{ width: 40 }} />
71     </View>
72     <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
73         <ProgressBar progress={75} height={4} />
74     </View>
75
76     <ScrollView contentContainerStyle={styles.content}>
77         <Text style={[styles.title, { color: colors.foreground }]}>How will you save?</Text>
78         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
79             Based on your goal and target date, here's what each person needs to save.
80         </Text>
81
82         <View style={styles.optionsContainer}>
83             <OptionCard type="weekly" title="Weekly" subtitle={`$${formatCurrency(paymentAmount)} / week`} /
84         >
85             <OptionCard type="biweekly" title="Bi-weekly" subtitle={`$${formatCurrency(paymentAmount *
86 month`} />
87             <OptionCard type="monthly" title="Monthly" subtitle={`$${formatCurrency(paymentAmount * 4)} /
88 month`} />
89         </View>
90
91         <View style={[styles.infoCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
92             <Feather name="info" size={20} color={colors.primary} style={{ marginRight: 12, marginTop:
93 2 }} />
94             <Text style={[styles.infoText, { color: colors.foreground }]}>
95                 You can always make extra manual contributions anytime, or skip a scheduled contribution if
96                 needed.
97             </Text>
98         </View>
99     </ScrollView>
100
101     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
102 colors.card, border: 1px solid colors.border }]}>
103         <Pressable
104             style={[styles.button, { backgroundColor: colors.primary }]}
105             onPress={handleNext}
106         >
107             <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
108                 Continue
109             </Text>
110         </Pressable>
111     </View>
112 </View>
113 );
114 }
115
116 const styles = StyleSheet.create({
117     container: { flex: 1 },
118     header: {
119         flexDirection: 'row',
120         alignItems: 'center',
121         justifyContent: 'space-between',
122         paddingHorizontal: 16,
123         paddingVertical: 12,
124     },
125     backButton: { width: 40, height: 40, justifyContent: 'center' },
126     stepText: { fontSize: 14, fontWeight: '600' },
127     content: { padding: 24, paddingBottom: 40 },
128     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
129     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
130     optionsContainer: { gap: 16, marginBottom: 32 },
131     optionCard: {
132         flexDirection: 'row',
133         alignItems: 'center',

```

artifacts/mobile/app/create-jar/plan.tsx (continued)

```
128     justifyContent: 'space-between',
129     padding: 20,
130     borderWidth: 2,
131     borderRadius: 16,
132   },
133   optionContent: { flex: 1 },
134   optionTitle: { fontSize: 18, fontWeight: 'bold', marginBottom: 4 },
135   optionSubtitle: { fontSize: 15 },
136   radio: {
137     width: 24,
138     height: 24,
139     borderRadius: 12,
140     borderWidth: 2,
141     alignItems: 'center',
142     justifyContent: 'center',
143   },
144   infoCard: {
145     flexDirection: 'row',
146     padding: 16,
147     borderRadius: 12,
148     borderWidth: 1,
149   },
150   infoText: { flex: 1, fontSize: 14, lineHeight: 20 },
151   footer: {
152     paddingHorizontal: 24,
153     paddingTop: 16,
154     borderTopWidth: 1,
155   },
156   button: {
157     height: 56,
158     borderRadius: 12,
159     alignItems: 'center',
160     justifyContent: 'center',
161   },
162   buttonText: { fontSize: 18, fontWeight: 'bold' },
163 });
164
```

artifacts/mobile/app/create-jar/rules.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { Feather } from '@expo/vector-icons';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { ProgressBar } from '@components/ProgressBar';
8
9  export default function CreateJarStep7() {
10    const colors = useColors();
11    const router = useRouter();
12    const insets = useSafeAreaInsets();
13    const [agreed, setAgreed] = useState(false);
14
15    const handleNext = () => {
16      if (agreed) {
17        router.push('/create-jar/review');
18      }
19    };
20  }
```

```

20
21   return (
22     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
23       <View style={styles.header}>
24         <Pressable onPress={() => router.back()} style={styles.backButton}>
25           <Feather name="arrow-left" size={24} color={colors.foreground} />
26         </Pressable>
27         <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 7 of 8</Text>
28         <View style={{ width: 40 }} />
29       </View>
30       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
31         <ProgressBar progress={87.5} height={4} />
32       </View>
33
34       <ScrollView contentContainerStyle={styles.content}>
35         <Text style={[styles.title, { color: colors.foreground }]}>Set the rules</Text>
36         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
37           Clear expectations make group trips stress-free. Review the standard agreement below.
38         </Text>
39
40         <View style={[styles.agreementCard, { backgroundColor: colors.card, borderColor:
41           <Text style={[styles.ruleTitle, { color: colors.foreground }]}>1. The Commitment</Text>
42           <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
43             All members agree to save their designated portion by the target date.
44           </Text>
45
46           <Text style={[styles.ruleTitle, { color: colors.foreground }]}>2. Spending Approvals</Text>
47           <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
48             Major expenses (like flights and lodging) require a majority vote before the jar funds are
49           </Text>
50
51           <Text style={[styles.ruleTitle, { color: colors.foreground }]}>3. Refunds</Text>
52           <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
53             Members can withdraw uncommitted funds anytime. Once funds are committed to a booking,
54             refunds are subj
55           </Text>
56           <Text style={[styles.ruleTitle, { color: colors.foreground }]}>4. Transparency</Text>
57           <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
58             All members can see the jar's progress, member contributions, and activity.
59           </Text>
60         </View>
61
62         <Pressable
63           style={styles.checkboxRow}
64           onPress={() => setAgreed(!agreed)}
65         >
66           <View style={[
67             styles.checkbox,
68             { borderColor: agreed ? colors.primary : colors.mutedForeground },
69             agreed && { backgroundColor: colors.primary }
70           ]>
71             {agreed && <Feather name="check" size={16} color="#fff" />}
72           </View>
73           <Text style={[styles.checkboxText, { color: colors.foreground }]}>
74             I agree to these rules and will ask my group to agree as well.
75           </Text>
76         </Pressable>
77
78       </ScrollView>

```

```

79
80     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
81     <Pressable
82         style={[
83             styles.button,
84             { backgroundColor: agreed ? colors.primary : colors.muted },
85         ]}
86         onPress={handleNext}
87         disabled={!agreed}
88     >
89     <Text style={[styles.buttonText, { color: agreed ? colors.primaryForeground :
90     colors.mutedForeground }]}>
91     </Text>
92     </Pressable>
93     </View>
94 </View>
95 );
96 }
97
98 const styles = StyleSheet.create({
99     container: { flex: 1 },
100     header: {
101         flexDirection: 'row',
102         alignItems: 'center',
103         justifyContent: 'space-between',
104         paddingHorizontal: 16,
105         paddingVertical: 12,
106     },
107     backButton: { width: 40, height: 40, justifyContent: 'center' },
108     stepText: { fontSize: 14, fontWeight: '600' },
109     content: { padding: 24, paddingBottom: 40 },
110     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
111     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
112     agreementCard: {
113         padding: 20,
114         borderRadius: 16,
115         borderWidth: 1,
116         marginBottom: 32,
117     },
118     ruleTitle: { fontSize: 16, fontWeight: 'bold', marginBottom: 8, marginTop: 16 },
119     ruleText: { fontSize: 14, lineHeight: 20 },
120     checkboxRow: {
121         flexDirection: 'row',
122         alignItems: 'flex-start',
123         paddingRight: 24,
124     },
125     checkbox: {
126         width: 24,
127         height: 24,
128         borderRadius: 6,
129         borderWidth: 2,
130         alignItems: 'center',
131         justifyContent: 'center',
132         marginRight: 12,
133         marginTop: 2,
134     },
135     checkboxText: {
136         flex: 1,
137         fontSize: 16,

```

artifacts/mobile/app/create-jar/rules.tsx (continued)

```
138     lineHeight: 24,
139   },
140   footer: {
141     paddingHorizontal: 24,
142     paddingTop: 16,
143     borderTopWidth: 1,
144   },
145   button: {
146     height: 56,
147     borderRadius: 12,
148     alignItems: 'center',
149     justifyContent: 'center',
150   },
151   buttonText: { fontSize: 18, fontWeight: 'bold' },
152 });
153
```

artifacts/mobile/app/create-jar/review.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView, ActivityIndicator } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useCreateJarContext } from '@/contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9  import { useCreateJar, useQueryClient } from '@workspace/api-client-react';
10
11  export default function CreateJarStep8() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { state, resetState } = useCreateJarContext();
16    const queryClient = useQueryClient();
17
18    const [isLoading, setIsLoading] = useState(false);
19    const { mutateAsync: createJar } = useCreateJar();
20
21    const handleLaunch = async () => {
22      try {
23        setIsLoading(true);
24        // Construct API payload
25        const response = await createJar({
26          data: {
27            name: state.name!,
28            category: state.category!,
29            destination: state.destination,
30            description: state.description,
31            targetDate: state.targetDate!,
32            goalAmountCents: state.goalAmountCents!,
33            startDate: state.startDate,
34            endDate: state.endDate,
35            currency: 'USD',
36          }
37        });
38
39        // We would also create milestones and invite members here,
40        // but assuming the MVP jar creation handles it or we do it sequentially.
```

```

41
42     queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
43     queryClient.invalidateQueries({ queryKey: ['/api/jars'] });
44
45     resetState();
46     router.replace(`/jar/${response.id}`);
47   } catch (e: any) {
48     console.error('Failed to create jar', e);
49     setIsLoading(false);
50   }
51 };
52
53 const formatCurrency = (cents: number) => {
54   return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
55 };
56
57 const totalPeople = (state.invitees?.length || 0) + 1;
58
59 return (
60   <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
61     <View style={styles.header}>
62       <Pressable onPress={() => router.back()} style={styles.backButton}>
63         <Feather name="arrow-left" size={24} color={colors.foreground} />
64       </Pressable>
65       <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 8 of 8</Text>
66       <View style={{ width: 40 }} />
67     </View>
68     <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
69       <ProgressBar progress={100} height={4} />
70     </View>
71
72     <ScrollView contentContainerStyle={styles.content}>
73       <View style={styles.iconContainer}>
74         <Feather name="check-circle" size={64} color={colors.primary} />
75       </View>
76       <Text style={[styles.title, { color: colors.foreground }]}>Ready for takeoff!</Text>
77       <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
78         Review your trip details before we create your jar.
79       </Text>
80
81       <View style={[styles.summaryCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
82         <View style={styles.summaryRow}>
83           <Text style={[styles.summaryLabel, { color: colors.mutedForeground }]}>Jar Name</Text>
84           <Text style={[styles.summaryValue, { color: colors.foreground }]}>{state.name}</Text>
85         </View>
86         <View style={styles.divider, { backgroundColor: colors.border }} />
87
88         <View style={styles.summaryRow}>
89           <Text style={[styles.summaryLabel, { color: colors.mutedForeground }]}>Destination</Text>
90           <Text style={[styles.summaryValue, { color: colors.foreground }]}>{state.destination}</Text>
91         </View>
92         <View style={styles.divider, { backgroundColor: colors.border }} />
93
94         <View style={styles.summaryRow}>
95           <Text style={[styles.summaryLabel, { color: colors.mutedForeground }]}>Goal Amount</Text>
96           <Text style={[styles.summaryValue, { color: colors.foreground }]}>
97             {formatCurrency(state.goalAmount)}
98           </Text>
99         </View>

```

```

100         <View style={styles.summaryRow}>
101             <Text style={[styles.summaryLabel, { color: colors.mutedForeground }]}>Target Date</Text>
102             <Text style={[styles.summaryValue, { color: colors.foreground }]}>{state.targetDate ? new
103         </View>
104         <View style={[styles.divider, { backgroundColor: colors.border }} />
105
106         <View style={styles.summaryRow}>
107             <Text style={[styles.summaryLabel, { color: colors.mutedForeground }]}>Group Size</Text>
108             <Text style={[styles.summaryValue, { color: colors.foreground }]}>{totalPeople} members</
109         </View>
110     </View>
111 </ScrollView>
112
113     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
114     colors.background,
115         style={[styles.button, { backgroundColor: colors.primary }]}
116         onPress={handleLaunch}
117         disabled={isLoading}
118     >
119         {isLoading ? (
120             <ActivityIndicator color={colors.primaryForeground} />
121         ) : (
122             <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
123                 Launch Jar
124             </Text>
125         )}
126     </Pressable>
127 </View>
128 </View>
129 );
130 }
131
132 const styles = StyleSheet.create({
133     container: { flex: 1 },
134     header: {
135         flexDirection: 'row',
136         alignItems: 'center',
137         justifyContent: 'space-between',
138         paddingHorizontal: 16,
139         paddingVertical: 12,
140     },
141     backButton: { width: 40, height: 40, justifyContent: 'center' },
142     stepText: { fontSize: 14, fontWeight: '600' },
143     content: { padding: 24, paddingBottom: 40 },
144     iconContainer: {
145         alignItems: 'center',
146         marginBottom: 24,
147     },
148     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16, textAlign: 'center' },
149     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 40, textAlign: 'center' },
150     summaryCard: {
151         borderRadius: 16,
152         borderWidth: 1,
153         padding: 20,
154     },
155     summaryRow: {
156         flexDirection: 'row',
157         justifyContent: 'space-between',
158         paddingVertical: 12,

```



```
159   },
160   summaryLabel: { fontSize: 15, fontWeight: '500' },
161   summaryValue: { fontSize: 16, fontWeight: 'bold' },
162   divider: { height: 1, marginVertical: 4 },
163   footer: {
164     paddingHorizontal: 24,
165     paddingTop: 16,
166     borderTopWidth: 1,
167   },
168   button: {
169     height: 56,
170     borderRadius: 12,
171     alignItems: 'center',
172     justifyContent: 'center',
173   },
174   buttonText: { fontSize: 18, fontWeight: 'bold' },
175 });
176
```

Mobile App — Components

artifacts/mobile/components/CalendarPicker.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4  import { Feather } from '@expo/vector-icons';
5
6  interface CalendarPickerProps {
7    value: Date | undefined;
8    onChange: (date: Date) => void;
9    minDate?: Date;
10 }
11
12 const DAYS = ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'];
13 const MONTHS = [
14   'January', 'February', 'March', 'April', 'May', 'June',
15   'July', 'August', 'September', 'October', 'November', 'December',
16 ];
17
18 function isSameDay(a: Date, b: Date) {
19   return a.getFullYear() === b.getFullYear()
20     && a.getMonth() === b.getMonth()
21     && a.getDate() === b.getDate();
22 }
23
24 function startOfMonth(year: number, month: number) {
25   return new Date(year, month, 1).getDay(); // 0 = Sun
26 }
27
28 function daysInMonth(year: number, month: number) {
29   return new Date(year, month + 1, 0).getDate();
30 }
31
32 export function CalendarPicker({ value, onChange, minDate }: CalendarPickerProps) {
33   const colors = useColors();
34   const today = new Date();
35
36   const [viewYear, setViewYear] = useState(
37     value ? value.getFullYear() : today.getFullYear()
38   );
39   const [viewMonth, setViewMonth] = useState(
40     value ? value.getMonth() : today.getMonth()
41   );
42
43   const prevMonth = () => {
44     if (viewMonth === 0) { setViewYear(y => y - 1); setViewMonth(11); }
45     else setViewMonth(m => m - 1);
46   };
47
48   const nextMonth = () => {
49     if (viewMonth === 11) { setViewYear(y => y + 1); setViewMonth(0); }
50     else setViewMonth(m => m + 1);
51   };
52 }
```

```

53   const firstDay = startOfMonth(viewYear, viewMonth);
54   const totalDays = daysInMonth(viewYear, viewMonth);
55
56   // Build grid cells: nulls for leading blanks, then day numbers
57   const cells: (number | null)[][] = [
58     ...Array(firstDay).fill(null),
59     ...Array.from({ length: totalDays }, (_, i) => i + 1),
60   ];
61   // Pad to complete last row
62   while (cells.length % 7 !== 0) cells.push(null);
63
64   const rows: (number | null)[][] = [];
65   for (let i = 0; i < cells.length; i += 7) rows.push(cells.slice(i, i + 7));
66
67   return (
68     <View style={[styles.container, { backgroundColor: colors.card, borderColor: colors.border }]}>
69       {/* Header */}
70       <View style={styles.header}>
71         <Pressable
72           onPress={prevMonth}
73           style={({ pressed }) => [styles.navBtn, pressed && { opacity: 0.6 }]}
74           hitSlop={12}
75         >
76           <Feather name="chevron-left" size={22} color={colors.foreground} />
77         </Pressable>
78
79         <Text style={[styles.monthLabel, { color: colors.foreground }]}>
80           {MONTHS[viewMonth]} {viewYear}
81         </Text>
82
83         <Pressable
84           onPress={nextMonth}
85           style={({ pressed }) => [styles.navBtn, pressed && { opacity: 0.6 }]}
86           hitSlop={12}
87         >
88           <Feather name="chevron-right" size={22} color={colors.foreground} />
89         </Pressable>
90       </View>
91
92       {/* Day headers */}
93       <View style={styles.dayHeaders}>
94         {DAYS.map(d => (
95           <Text key={d} style={[styles.dayHeader, { color: colors.mutedForeground }]}>{d}</Text>
96         ))}
97       </View>
98
99       {/* Date grid */}
100      {rows.map((row, ri) => (
101        <View key={ri} style={styles.row}>
102          {row.map((day, ci) => {
103            if (!day) return <View key={ci} style={styles.cell} />;
104
105            // Use noon to avoid UTC-shift off-by-one when calling toISOString()
106            const cellDate = new Date(viewYear, viewMonth, day, 12, 0, 0);
107            const isSelected = value ? isSameDay(cellDate, value) : false;
108            const isToday = isSameDay(cellDate, today);
109            const isDisabled = minDate ? cellDate < minDate : false;
110
111            return (

```

```

112         <Pressable
113             key={ci}
114             style={[
115                 styles.cell,
116                 isSelected && { backgroundColor: colors.primary, borderRadius: 22 },
117                 !isSelected && isToday && { borderRadius: 22, borderWidth: 2, borderColor:
118 colors.primary }]]
119             onPress={() => !isDisabled && onChange(cellDate)}
120             disabled={isDisabled}
121         >
122             <Text style={[
123                 styles.dayText,
124                 { color: isSelected ? '#fff' : isDisabled ? colors.mutedForeground :
125 isToday && !isSelected && { color: colors.primary, fontWeight: '700' },
126                 ]}>
127                 {day}
128             </Text>
129         </Pressable>
130     );
131     }}}
132     </View>
133     )}}
134 </View>
135 );
136 }
137
138 const CELL_SIZE = 44;
139
140 const styles = StyleSheet.create({
141     container: {
142         borderRadius: 16,
143         borderWidth: 1,
144         padding: 16,
145         width: '100%',
146     },
147     header: {
148         flexDirection: 'row',
149         alignItems: 'center',
150         justifyContent: 'space-between',
151         marginBottom: 16,
152     },
153     navBtn: {
154         padding: 4,
155     },
156     monthLabel: {
157         fontSize: 17,
158         fontWeight: '700',
159     },
160     dayHeaders: {
161         flexDirection: 'row',
162         marginBottom: 4,
163     },
164     dayHeader: {
165         width: CELL_SIZE,
166         textAlign: 'center',
167         fontSize: 13,
168         fontWeight: '600',
169     },
170     row: {

```

artifacts/mobile/components/CalendarPicker.tsx (continued)

```
171     flexDirection: 'row',
172   },
173   cell: {
174     width: CELL_SIZE,
175     height: CELL_SIZE,
176     alignItems: 'center',
177     justifyContent: 'center',
178   },
179   dayText: {
180     fontSize: 15,
181     fontWeight: '500',
182   },
183 });
184
```

artifacts/mobile/components/DateInput.tsx

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, Platform } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4  import { Feather } from '@expo/vector-icons';
5  import { CalendarPicker } from '../CalendarPicker';
6  import DateTimePicker from '@react-native-community/datetimepicker';
7
8  interface DateInputProps {
9    value: Date | undefined;
10   onChange: (date: Date) => void;
11   placeholder?: string;
12   error?: string;
13   minDate?: Date;
14 }
15
16 export function DateInput({ value, onChange, placeholder = 'Select date', error, minDate }:
17   ~~~~~
18   const colors = useColors();
19   const [showPicker, setShowPicker] = useState(false);
20
21   const formatDate = (date: Date) => {
22     return date.toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' });
23   };
24
25   const handleSelect = (date: Date) => {
26     onChange(date);
27     // Keep open on web so user can see selection; they dismiss by tapping the button again
28   };
29
30   return (
31     <View style={styles.container}>
32       <Pressable
33         style={[
34           styles.inputContainer,
35           { borderColor: error ? colors.destructive : showPicker ? colors.primary : colors.input,
36         }
37       >
38         <Feather name="calendar" size={20} color={showPicker ? colors.primary : colors.mutedForeground}
39         <Text
40           style={[
41             styles.text,
42             { color: value ? colors.foreground : colors.mutedForeground },
```

```

43     }}
44     >
45     {value ? formatDate(value) : placeholder}
46   </Text>
47   <Feather
48     name={showPicker ? 'chevron-up' : 'chevron-down'}
49     size={18}
50     color={colors.mutedForeground}
51   />
52 </Pressable>
53
54 {error ? (
55   <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text>
56 ) : null}
57
58 {showPicker && (
59   Platform.OS === 'web' ? (
60     <View style={styles.calendarWrapper}>
61       <CalendarPicker
62         value={value}
63         onChange={(date) => {
64           handleSelect(date);
65           setShowPicker(false);
66         }}
67         minDate={minDate}
68       />
69     </View>
70   ) : (
71     <DateTimePicker
72       value={value || new Date()}
73       mode="date"
74       display="spinner"
75       minimumDate={minDate}
76       onChange={(event, date) => {
77         setShowPicker(false);
78         if (date) onChange(date);
79       }}
80     />
81   )
82 )}
83 </View>
84 );
85 }
86
87 const styles = StyleSheet.create({
88   container: {
89     width: '100%',
90   },
91   inputContainer: {
92     flexDirection: 'row',
93     alignItems: 'center',
94     borderWidth: 1,
95     borderRadius: 12,
96     paddingHorizontal: 16,
97     height: 56,
98   },
99   icon: {
100     marginRight: 12,
101   },

```

artifacts/mobile/components/DateInput.tsx (continued)

```
102   text: {
103     fontSize: 16,
104     flex: 1,
105   },
106   errorText: {
107     fontSize: 13,
108     marginTop: 6,
109     marginLeft: 4,
110   },
111   calendarWrapper: {
112     marginTop: 8,
113   },
114 });
115
```

artifacts/mobile/components/CurrencyInput.tsx

```
1  import React from 'react';
2  import { View, TextInput, StyleSheet, TextInputProps } from 'react-native';
3  import { useColors } from '@/hooks/useColors';
4
5  interface CurrencyInputProps extends Omit<TextInputProps, 'onChangeText' | 'value'> {
6    value: number; // in cents
7    onChangeCents: (cents: number) => void;
8    label?: string;
9    error?: string;
10 }
11
12 export function CurrencyInput({ value, onChangeCents, label, error, style, ...props }:
13   CurrencyInputProps) {
14   const colors = useColors();
15
16   const formatDisplay = (cents: number) => {
17     if (!cents) return '';
18     return (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
19   };
20
21   const handleTextChange = (text: string) => {
22     const numericOnly = text.replace(/^[^0-9]/g, '');
23     if (!numericOnly) {
24       onChangeCents(0);
25       return;
26     }
27     onChangeCents(parseInt(numericOnly, 10) * 100);
28   };
29
30   return (
31     <View style={styles.container}>
32       <View
33         style={[
34           styles.inputContainer,
35           { borderColor: error ? colors.destructive : colors.input, backgroundColor: colors.card },
36           style,
37         ]>
38         <Text style={styles.prefix}></Text>
39         <TextInput
40           style={styles.input, { color: colors.foreground }}
41           value={formatDisplay(value)}
42           onChangeText={handleTextChange}

```

artifacts/mobile/components/CurrencyInput.tsx (continued)

```
43         keyboardType="number-pad"
44         placeholder="0"
45         placeholderTextColor={colors.mutedForeground}
46         {...props}
47     />
48 </View>
49 </View>
50 );
51 }
52
53 const styles = StyleSheet.create({
54     container: {
55         width: '100%',
56     },
57     inputContainer: {
58         flexDirection: 'row',
59         alignItems: 'center',
60         borderWidth: 1,
61         borderRadius: 12,
62         paddingHorizontal: 16,
63         height: 56,
64     },
65     prefix: {
66         fontSize: 24,
67         fontWeight: '600',
68         marginRight: 4,
69     },
70     input: {
71         flex: 1,
72         fontSize: 24,
73         fontWeight: '600',
74         height: '100%',
75     },
76 });
77
```

artifacts/mobile/components/CircularProgress.tsx

```
1  import React, { useEffect } from 'react';
2  import { View, StyleSheet, Text } from 'react-native';
3  import Svg, { Circle } from 'react-native-svg';
4  import Animated, {
5      useSharedValue,
6      useAnimatedProps,
7      withTiming,
8      withSpring,
9      Easing,
10 } from 'react-native-reanimated';
11 import { useColors } from '@/hooks/useColors';
12
13 const AnimatedCircle = Animated.createAnimatedComponent(Circle);
14
15 interface CircularProgressProps {
16     size: number;
17     progress: number;
18     strokeWidth: number;
19     color?: string;
20     trackColor?: string;
21     children?: React.ReactNode;
```



```

22   }
23
24   export function CircularProgress({
25     size,
26     progress,
27     strokeWidth,
28     color,
29     trackColor,
30     children,
31   }: CircularProgressProps) {
32     const colors = useColors();
33     const radius = (size - strokeWidth) / 2;
34     const circumference = radius * 2 * Math.PI;
35     const progressValue = useSharedValue(0);
36
37     useEffect(() => {
38       progressValue.value = withTiming(Math.min(Math.max(progress, 0), 100), {
39         duration: 1000,
40         easing: Easing.out(Easing.cubic),
41       });
42     }, [progress]);
43
44     const animatedProps = useAnimatedProps(() => {
45       const strokeDashoffset = circumference - (circumference * progressValue.value) / 100;
46       return { strokeDashoffset };
47     });
48
49     return (
50       <View style={{ width: size, height: size, alignItems: 'center', justifyContent: 'center' }}>
51         <Svg width={size} height={size}>
52           <Circle
53             cx={size / 2}
54             cy={size / 2}
55             r={radius}
56             stroke={trackColor || colors.muted}
57             strokeWidth={strokeWidth}
58             fill="none"
59           />
60           <AnimatedCircle
61             cx={size / 2}
62             cy={size / 2}
63             r={radius}
64             stroke={color || colors.primary}
65             strokeWidth={strokeWidth}
66             fill="none"
67             strokeDasharray={circumference}
68             animatedProps={animatedProps}
69             strokeLinecap="round"
70             rotation="-90"
71             originX={size / 2}
72             originY={size / 2}
73           />
74         </Svg>
75         {children && <View style={StyleSheet.absoluteFill}>{children}</View>}
76       </View>
77     );
78   }
79

```

```

1  import React, { useEffect } from 'react';
2  import { View, StyleSheet } from 'react-native';
3  import Animated, {
4    useSharedValue,
5    useAnimatedStyle,
6    withTiming,
7    Easing,
8  } from 'react-native-reanimated';
9  import { useColors } from '@/hooks/useColors';
10
11 interface ProgressBarProps {
12   progress: number;
13   height?: number;
14   color?: string;
15   backgroundColor?: string;
16 }
17
18 export function ProgressBar({ progress, height = 8, color, backgroundColor }: ProgressBarProps) {
19   const colors = useColors();
20   const progressWidth = useSharedValue(0);
21
22   useEffect(() => {
23     progressWidth.value = withTiming(Math.min(Math.max(progress, 0), 100), {
24       duration: 800,
25       easing: Easing.out(Easing.cubic),
26     });
27   }, [progress]);
28
29   const animatedStyle = useAnimatedStyle(() => {
30     return {
31       width: `${progressWidth.value}%`,
32     };
33   });
34
35   return (
36     <View
37       style={[
38         styles.track,
39         { height, backgroundColor: backgroundColor || colors.muted, borderRadius: height / 2 },
40       ]
41     >
42       <Animated.View
43         style={[
44           styles.fill,
45           { backgroundColor: color || colors.primary, borderRadius: height / 2 },
46           animatedStyle,
47         ]
48       />
49     </View>
50   );
51 }
52
53 const styles = StyleSheet.create({
54   track: {
55     width: '100%',
56     overflow: 'hidden',
57   },
58   fill: {
59     height: '100%',

```

artifacts/mobile/components/ProgressBar.tsx (continued)

```
60   },
61   });
62
```

artifacts/mobile/components/JarCard.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { ImageBackground } from 'expo-image';
4  import { LinearGradient } from 'expo-linear-gradient';
5  import { useColors } from '@/hooks/useColors';
6  import { ProgressBar } from '../ProgressBar';
7  import { JarHealthBadge } from '../JarHealthBadge';
8  import { useRouter } from 'expo-router';
9  import type { JarSummary } from '@workspace/api-client-react/src/generated/api.schemas';
10 import { Feather } from '@expo/vector-icons';
11
12 interface JarCardProps {
13   jar: JarSummary;
14   hero?: boolean;
15 }
16
17 export function JarCard({ jar, hero = false }: JarCardProps) {
18   const colors = useColors();
19   const router = useRouter();
20
21   const formatCurrency = (cents: number) => {
22     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
23   };
24
25   const onPress = () => {
26     router.push(`/jar/${jar.id}`);
27   };
28
29   const imageSource = jar.coverImageUrl
30     ? { uri: jar.coverImageUrl }
31     : require('../assets/images/hawaii-cover.jpg');
32
33   return (
34     <Pressable
35       onPress={onPress}
36       style={({ pressed }) => [
37         styles.card,
38         { backgroundColor: colors.card, borderColor: colors.border },
39         pressed && { opacity: 0.95, transform: [{ scale: 0.98 }] },
40         hero ? styles.heroCard : styles.standardCard,
41       ]
42     >
43     <View style={[styles.imageContainer, hero ? { height: 180 } : { height: 120 }]}>
44       <ImageBackground
45         source={imageSource}
46         style={StyleSheet.absoluteFill}
47         contentFit="cover"
48       >
49         <LinearGradient
50           colors={['transparent', 'rgba(0,0,0,0.8)']}
51           style={StyleSheet.absoluteFill}
52         />
53       <View style={styles.imageOverlayContent}>
```

```

54      {jar.daysRemaining !== null && jar.daysRemaining !== undefined && (
55        <View style={styles.daysChip}>
56          <Text style={styles.daysText}>{jar.daysRemaining} days left</Text>
57        </View>
58      )}
59      <View style={styles.titleContainer}>
60        <Text style={styles.title} numberOfLines={1}>
61          {jar.name}
62        </Text>
63        {jar.destination && (
64          <View style={styles.locationRow}>
65            <Feather name="map-pin" size={12} color="#fff" />
66            <Text style={styles.destination} numberOfLines={1}>
67              {jar.destination}
68            </Text>
69          </View>
70        )}
71      </View>
72    </View>
73    </ImageBackground>
74  </View>
75
76  <View style={styles.content}>
77    <View style={styles.progressRow}>
78      <Text style={[styles.amountText, { color: colors.foreground }]}>
79        <Text style={{ fontWeight: 'bold' }}>{formatCurrency(jar.totalSavedCents)}</Text>
80        <Text style={{ color: colors.mutedForeground }}> of {formatCurrency(jar.goalAmountCents)}</
81      </Text>
82      <Text style={[styles.percentText, { color: colors.primary }]}>
83        {jar.percentFunded}%
84      </Text>
85    </View>
86
87    <ProgressBar progress={jar.percentFunded} height={hero ? 10 : 6} />
88
89    <View style={styles.footerRow}>
90      <View style={styles.membersRow}>
91        <Feather name="users" size={14} color={colors.mutedForeground} />
92        <Text style={[styles.membersText, { color: colors.mutedForeground }]}>
93          {jar.memberCount} members
94        </Text>
95      </View>
96      {jar.health && <JarHealthBadge status={jar.health.status} />}
97    </View>
98  </View>
99  </Pressable>
100 );
101 }
102
103 const styles = StyleSheet.create({
104   card: {
105     borderRadius: 16,
106     overflow: 'hidden',
107     borderWidth: 1,
108     shadowColor: '#000',
109     shadowOffset: { width: 0, height: 4 },
110     shadowOpacity: 0.05,
111     shadowRadius: 8,
112     elevation: 2,

```

```
113   },
114   heroCard: {
115     marginBottom: 24,
116   },
117   standardCard: {
118     marginBottom: 16,
119   },
120   imageContainer: {
121     width: '100%',
122     backgroundColor: '#178B57', // Fallback
123   },
124   imageOverlayContent: {
125     flex: 1,
126     justifyContent: 'space-between',
127     padding: 16,
128   },
129   daysChip: {
130     alignSelf: 'flex-end',
131     backgroundColor: 'rgba(0,0,0,0.5)',
132     paddingHorizontal: 10,
133     paddingVertical: 4,
134     borderRadius: 12,
135     backdropFilter: 'blur(4px)', // web
136   },
137   daysText: {
138     color: '#fff',
139     fontSize: 12,
140     fontWeight: '600',
141   },
142   titleContainer: {
143     marginTop: 'auto',
144   },
145   title: {
146     color: '#fff',
147     fontSize: 22,
148     fontWeight: 'bold',
149     marginBottom: 4,
150   },
151   locationRow: {
152     flexDirection: 'row',
153     alignItems: 'center',
154     gap: 4,
155   },
156   destination: {
157     color: '#E8F6EF',
158     fontSize: 14,
159     fontWeight: '500',
160   },
161   content: {
162     padding: 16,
163   },
164   progressRow: {
165     flexDirection: 'row',
166     justifyContent: 'space-between',
167     alignItems: 'flex-end',
168     marginBottom: 10,
169   },
170   amountText: {
171     fontSize: 16,
```

artifacts/mobile/components/JarCard.tsx (continued)

```
172   },
173   percentText: {
174     fontSize: 16,
175     fontWeight: 'bold',
176   },
177   footerRow: {
178     flexDirection: 'row',
179     justifyContent: 'space-between',
180     alignItems: 'center',
181     marginTop: 16,
182   },
183   membersRow: {
184     flexDirection: 'row',
185     alignItems: 'center',
186     gap: 6,
187   },
188   membersText: {
189     fontSize: 13,
190   },
191 });
192
```

artifacts/mobile/components/JarHealthBadge.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet } from 'react-native';
3  import { useColors } from '@/hooks/useColors';
4  import { JarHealthStatus } from '@workspace/api-client-react/src/generated/api.schemas';
5
6  interface JarHealthBadgeProps {
7    status: typeof JarHealthStatus[keyof typeof JarHealthStatus];
8  }
9
10 export function JarHealthBadge({ status }: JarHealthBadgeProps) {
11   const colors = useColors();
12
13   let bgColor = colors.muted;
14   let textColor = colors.mutedForeground;
15   let dotColor = colors.mutedForeground;
16   let label = 'Unknown';
17
18   switch (status) {
19     case 'Ahead':
20       bgColor = colors.secondary;
21       textColor = colors.primary;
22       dotColor = colors.success;
23       label = 'Ahead of Schedule';
24       break;
25     case 'OnTrack':
26       bgColor = colors.secondary;
27       textColor = colors.primary;
28       dotColor = '#3B82F6'; // Blue
29       label = 'On Track';
30       break;
31     case 'NeedsAttention':
32       bgColor = '#FEF3C7';
33       textColor = '#92400E';
34       dotColor = colors.warning;
35       label = 'Needs Attention';

```

artifacts/mobile/components/JarHealthBadge.tsx (continued)

```
36     break;
37   case 'AtRisk':
38     bgColor = '#FEE2E2';
39     textColor = '#991B1B';
40     dotColor = colors.destructive;
41     label = 'At Risk';
42     break;
43   }
44
45   return (
46     <View style={[styles.badge, { backgroundColor: bgColor }]}>
47       <View style={[styles.dot, { backgroundColor: dotColor }]} />
48       <Text style={[styles.text, { color: textColor }]}>{label}</Text>
49     </View>
50   );
51 }
52
53 const styles = StyleSheet.create({
54   badge: {
55     flexDirection: 'row',
56     alignItems: 'center',
57     paddingHorizontal: 8,
58     paddingVertical: 4,
59     borderRadius: 12,
60     alignSelf: 'flex-start',
61   },
62   dot: {
63     width: 6,
64     height: 6,
65     borderRadius: 3,
66     marginRight: 6,
67   },
68   text: {
69     fontSize: 12,
70     fontWeight: '600',
71   },
72 });
73
```

artifacts/mobile/components/MemberAvatar.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet } from 'react-native';
3  import { Image } from 'expo-image';
4  import { useColors } from '@hooks/useColors';
5  import { JarMemberHealthStatus } from '@workspace/api-client-react/src/generated/api.schemas';
6
7  interface MemberAvatarProps {
8    displayName: string;
9    avatarUrl?: string | null;
10    size?: number;
11    showStatus?: boolean;
12    status?: typeof JarMemberHealthStatus[keyof typeof JarMemberHealthStatus];
13  }
14
15  export function MemberAvatar({
16    displayName,
17    avatarUrl,
18    size = 40,
```

```

19   showStatus,
20   status,
21 }: MemberAvatarProps) {
22   const colors = useColors();
23
24   const getInitials = (name: string) => {
25     return name
26       .split(' ')
27       .map((n) => n[0])
28       .join('')
29       .substring(0, 2)
30       .toUpperCase();
31   };
32
33   let ringColor = 'transparent';
34   if (showStatus && status) {
35     switch (status) {
36       case 'Ahead':
37         ringColor = colors.success;
38         break;
39       case 'OnTrack':
40         ringColor = '#3B82F6';
41         break;
42       case 'NeedsAttention':
43         ringColor = colors.warning;
44         break;
45       case 'AtRisk':
46         ringColor = colors.destructive;
47         break;
48       case 'NotStarted':
49         ringColor = colors.mutedForeground;
50         break;
51     }
52   }
53
54   return (
55     <View
56       style={[
57         styles.container,
58         {
59           width: size,
60           height: size,
61           borderRadius: size / 2,
62           borderWidth: showStatus ? 2 : 0,
63           borderColor: ringColor,
64           padding: showStatus ? 2 : 0,
65         },
66       ]}
67     >
68       {avatarUrl ? (
69         <Image
70           source={{ uri: avatarUrl }}
71           style={{ width: '100%', height: '100%', borderRadius: size / 2 }}
72         />
73       ) : (
74         <View
75           style={[
76             styles.initialsContainer,
77             { backgroundColor: colors.muted, borderRadius: size / 2 },

```


artifacts/mobile/components/MemberAvatar.tsx (continued)

```
78         ]}
79       >
80       <Text style={[styles.initialsText, { color: colors.primary, fontSize: size * 0.4 }]}>
81         {getInitials(displayName)}
82       </Text>
83     </View>
84   )}
85 </View>
86 );
87 }
88
89 const styles = StyleSheet.create({
90   container: {
91     justifyContent: 'center',
92     alignItems: 'center',
93   },
94   initialsContainer: {
95     width: '100%',
96     height: '100%',
97     justifyContent: 'center',
98     alignItems: 'center',
99   },
100   initialsText: {
101     fontWeight: 'bold',
102   },
103 });
104
```

artifacts/mobile/components/ScheduleSetupSheet.tsx

```
1  /**
2   * ScheduleSetupSheet
3   *
4   * A bottom-sheet-style modal that lets a jar member create or edit their
5   * contribution schedule (frequency + amount).
6   */
7
8  import React, { useEffect, useState } from 'react';
9  import {
10    Modal,
11    View,
12    Text,
13    StyleSheet,
14    Pressable,
15    ScrollView,
16    ActivityIndicator,
17    Platform,
18    KeyboardAvoidingView,
19  } from 'react-native';
20  import { Feather } from '@expo/vector-icons';
21  import { useColors } from '@/hooks/useColors';
22  import { CurrencyInput } from '@components/CurrencyInput';
23  import {
24    useGetContributionSchedule,
25    useCreateContributionSchedule,
26    useUpdateContributionSchedule,
27  } from '@workspace/api-client-react';
28  import { useQueryClient } from '@tanstack/react-query';
29  import type { ContributionSchedule } from '@workspace/api-client-react';
```

```

30 import { useSafeAreaInsets } from 'react-native-safe-area-context';
31
32 type Frequency = 'weekly' | 'biweekly' | 'monthly';
33
34 const FREQUENCY_OPTIONS: { value: Frequency; label: string; description: string }[] = [
35   { value: 'weekly', label: 'Weekly', description: 'Every week on the same day' },
36   { value: 'biweekly', label: 'Every 2 weeks', description: 'Every other week' },
37   { value: 'monthly', label: 'Monthly', description: 'Once a month' },
38 ];
39
40 interface Props {
41   visible: boolean;
42   jarId: string;
43   onClose: () => void;
44 }
45
46 export function ScheduleSetupSheet({ visible, jarId, onClose }: Props) {
47   const colors = useColors();
48   const insets = useSafeAreaInsets();
49   const queryClient = useQueryClient();
50
51   const { data: existingSchedule, isLoading: loadingSchedule } = useGetContributionSchedule(jarId, {
52     query: { enabled: visible && !!jarId },
53   });
54
55   const { mutateAsync: createSchedule, isPending: creating } = useCreateContributionSchedule();
56   const { mutateAsync: updateSchedule, isPending: updating } = useUpdateContributionSchedule();
57
58   const [frequency, setFrequency] = useState<Frequency>('monthly');
59   const [amountCents, setAmountCents] = useState(0);
60   const [isPaused, setIsPaused] = useState(false);
61   const [error, setError] = useState('');
62   const [success, setSuccess] = useState(false);
63
64   // Pre-fill from existing schedule
65   useEffect(() => {
66     if (existingSchedule) {
67       const freq = existingSchedule.frequency as Frequency;
68       if (['weekly', 'biweekly', 'monthly'].includes(freq)) setFrequency(freq);
69       setAmountCents(existingSchedule.amountCents);
70       setIsPaused(existingSchedule.isPaused);
71     }
72   }, [existingSchedule]);
73
74   const isSaving = creating || updating;
75
76   const handleSave = async () => {
77     if (amountCents <= 0) {
78       setError('Please enter an amount greater than $0.');
```

```

89     } else {
90         // Create new
91         const today = new Date();
92         const startDate = `${today.getFullYear()}-${String(today.getMonth() + 1).padStart(2, '0')}-${
93             String(today.getDate() + 1).padStart(2, '0')}
94         `;
95         await createSchedule({
96             jarId,
97             data: { frequency, amountCents, startDate, endCondition: 'targetDate' },
98         });
99         // Invalidate dashboard so next-contribution chip updates
100         queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
101         queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}/schedule`] });
102         setSuccess(true);
103         setTimeout(() => {
104             setSuccess(false);
105             onClose();
106         }, 1400);
107     } catch (e: any) {
108         setError(e?.message || 'Failed to save schedule. Please try again.');
```

```

108     }
109 };
110

```

```

111 const handleTogglePause = async () => {
112     if (!existingSchedule) return;
113     try {
114         await updateSchedule({ jarId, data: { isPaused: !isPaused } });
115         setIsPaused((p) => !p);
116         queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
117         queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}/schedule`] });
118     } catch {
119         // ignore
120     }
121 };
122

```

```

123 const formatAmount = (cents: number) =>
124     '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
125

```

```

126 return (
127     <Modal visible={visible} animationType="slide" presentationStyle="pageSheet" onRequestClose={onClose}>
128     <KeyboardAvoidingView
129         behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
130         style={[styles.container, { backgroundColor: colors.background }]}
131     >
132         {/* Header */}
133         <View style={[styles.header, { paddingTop: insets.top + 16, borderBottomColor: colors.border }]}>
134             <Pressable onPress={onClose} style={styles.closeBtn}>
135                 <Feather name="x" size={24} color={colors.foreground} />
136             </Pressable>
137             <Text style={[styles.headerTitle, { color: colors.foreground }]}>
138                 {existingSchedule ? 'Edit Schedule' : 'Set Up Schedule'}
139             </Text>
140             <View style={{ width: 40 }} />
141         </View>
142
143         <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}>
144             {loadingSchedule ? (
145                 <ActivityIndicator color={colors.primary} style={{ marginTop: 40 }} />
146             ) : success ? (
147                 <View style={styles.successContainer}>

```

```

148         <Feather name="check-circle" size={64} color={colors.primary} />
149         <Text style={[styles.successText, { color: colors.foreground }]}>Schedule saved!</Text>
150     </View>
151 ) : (
152     <>
153         <Text style={[styles.sectionLabel, { color: colors.mutedForeground }]}>
154             How often would you like to contribute?
155         </Text>
156
157         <View style={styles.optionsRow}>
158             {FREQUENCY_OPTIONS.map((opt) => {
159                 const isSelected = frequency === opt.value;
160                 return (
161                     <Pressable
162                         key={opt.value}
163                         onPress={() => setFrequency(opt.value)}
164                         style={[
165                             styles.freqOption,
166                             {
167                                 backgroundColor: isSelected ? colors.primary : colors.card,
168                                 borderColor: isSelected ? colors.primary : colors.border,
169                             },
170                         ]}
171                     >
172                         <Text style={[styles.freqLabel, { color: isSelected ? colors.primaryForeground :
173                             colors.mutedF...
174                             {opt.label}
175                         </Text>
176                         <Text style={[styles.freqDesc, { color: isSelected ? 'rgba(255,255,255,0.75)' :
177                             colors.mutedF...
178                             {opt.description}
179                         </Text>
180                     </Pressable>
181                 );
182             })}
183         </View>
184
185         <Text style={[styles.sectionLabel, { color: colors.mutedForeground, marginTop: 28 }]}>
186             Amount per contribution
187         </Text>
188
189         <CurrencyInput
190             value={amountCents}
191             onChangeCents={setAmountCents}
192             autoFocus={!existingSchedule}
193         />
194
195         {/* Summary chip */}
196         {amountCents > 0 && (
197             <View style={[styles.summaryChip, { backgroundColor: colors.secondary }]}>
198                 <Feather name="calendar" size={14} color={colors.primary} />
199                 <Text style={[styles.summaryText, { color: colors.secondaryForeground }]}>
200                     {formatAmount(amountCents)}{' '}
201                     {frequency === 'weekly'
202                     ? 'every week'
203                     : frequency === 'biweekly'
204                     ? 'every two weeks'
205                     : 'every month'}
206                 </Text>
207             </View>
208         )}

```

```

207
208         {error ? (
209             <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text>
210         ) : null}
211
212         <Pressable
213             style={[
214                 styles.saveButton,
215                 { backgroundColor: colors.primary, opacity: isSaving ? 0.7 : 1 },
216             ]}
217             onPress={handleSave}
218             disabled={isSaving}
219         >
220             {isSaving ? (
221                 <ActivityIndicator color={colors.primaryForeground} />
222             ) : (
223                 <Text style={[styles.saveButtonText, { color: colors.primaryForeground }]}>
224                     {existingSchedule ? 'Update Schedule' : 'Start Schedule'}
225                 </Text>
226             )}
227         </Pressable>
228
229         {/* Pause / Resume toggle for existing schedule */}
230         {existingSchedule && (
231             <Pressable
232                 style={[styles.secondaryButton, { borderColor: colors.border }]}
233                 onPress={handleTogglePause}
234             >
235                 <Feather name={isPaused ? 'play' : 'pause'} size={16} color={colors.foreground} />
236                 <Text style={[styles.secondaryButtonText, { color: colors.foreground }]}>
237                     {isPaused ? 'Resume Schedule' : 'Pause Schedule'}
238                 </Text>
239             </Pressable>
240         )}
241     </>
242 )}
243 </ScrollView>
244 </KeyboardAvoidingView>
245 </Modal>
246 );
247 }
248
249 const styles = StyleSheet.create({
250     container: { flex: 1 },
251     header: {
252         flexDirection: 'row',
253         alignItems: 'center',
254         justifyContent: 'space-between',
255         paddingHorizontal: 16,
256         paddingBottom: 16,
257         borderBottomWidth: StyleSheet.hairlineWidth,
258     },
259     closeBtn: { padding: 8, marginLeft: -8 },
260     headerTitle: { fontSize: 18, fontWeight: '700' },
261     content: { padding: 20 },
262     sectionLabel: { fontSize: 14, fontWeight: '500', marginBottom: 12 },
263     optionsRow: { gap: 12 },
264     freqOption: {
265         padding: 16,

```

artifacts/mobile/components/ScheduleSetupSheet.tsx (continued)

```
266     borderRadius: 16,
267     borderWidth: 1.5,
268   },
269   freqLabel: { fontSize: 16, fontWeight: '700', marginBottom: 2 },
270   freqDesc: { fontSize: 13 },
271   summaryChip: {
272     flexDirection: 'row',
273     alignItems: 'center',
274     gap: 8,
275     padding: 12,
276     borderRadius: 12,
277     marginTop: 16,
278   },
279   summaryText: { fontSize: 14, fontWeight: '600' },
280   errorText: { marginTop: 12, fontSize: 13 },
281   saveButton: {
282     height: 56,
283     borderRadius: 16,
284     alignItems: 'center',
285     justifyContent: 'center',
286     marginTop: 24,
287   },
288   saveButtonText: { fontSize: 16, fontWeight: '700' },
289   secondaryButton: {
290     height: 52,
291     borderRadius: 16,
292     borderWidth: 1,
293     alignItems: 'center',
294     justifyContent: 'center',
295     marginTop: 12,
296     flexDirection: 'row',
297     gap: 8,
298   },
299   secondaryButtonText: { fontSize: 15, fontWeight: '600' },
300   successContainer: { alignItems: 'center', paddingTop: 60, gap: 16 },
301   successText: { fontSize: 20, fontWeight: '700' },
302 });
303
```

artifacts/mobile/components/EmptyState.tsx

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { Feather } from '@expo/vector-icons';
4  import { useColors } from '@hooks/useColors';
5
6  interface EmptyStateProps {
7    icon: keyof typeof Feather.glyphMap;
8    title: string;
9    description?: string;
10   action?: {
11     label: string;
12     onPress: () => void;
13   };
14 }
15
16 export function EmptyState({ icon, title, description, action }: EmptyStateProps) {
17   const colors = useColors();
18
```

```

19   return (
20     <View style={styles.container}>
21       <View style={[styles.iconContainer, { backgroundColor: colors.secondary }]}>
22         <Feather name={icon} size={32} color={colors.primary} />
23       </View>
24       <Text style={[styles.title, { color: colors.foreground }]}>{title}</Text>
25       {description && (
26         <Text style={[styles.description, { color: colors.mutedForeground }]}>
27           {description}
28         </Text>
29       )}
30       {action && (
31         <Pressable
32           style={[styles.button, { backgroundColor: colors.primary }]}
33           onPress={action.onPress}
34         >
35           <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
36             {action.label}
37           </Text>
38         </Pressable>
39       )}
40     </View>
41   );
42 }
43
44 const styles = StyleSheet.create({
45   container: {
46     flex: 1,
47     alignItems: 'center',
48     justifyContent: 'center',
49     padding: 32,
50   },
51   iconContainer: {
52     width: 64,
53     height: 64,
54     borderRadius: 32,
55     alignItems: 'center',
56     justifyContent: 'center',
57     marginBottom: 24,
58   },
59   title: {
60     fontSize: 20,
61     fontWeight: 'bold',
62     marginBottom: 8,
63     textAlign: 'center',
64   },
65   description: {
66     fontSize: 15,
67     textAlign: 'center',
68     marginBottom: 24,
69     lineHeight: 22,
70   },
71   button: {
72     paddingHorizontal: 24,
73     paddingVertical: 12,
74     borderRadius: 12,
75   },
76   buttonText: {
77     fontSize: 16,

```

artifacts/mobile/components/EmptyState.tsx (continued)

```
78     fontWeight: '600',
79   },
80 });
81
```

artifacts/mobile/components/SkeletonLoader.tsx

```
1  import React, { useEffect } from 'react';
2  import { View, StyleSheet } from 'react-native';
3  import Animated, {
4    useSharedValue,
5    useAnimatedStyle,
6    withRepeat,
7    withTiming,
8    Easing,
9  } from 'react-native-reanimated';
10 import { useColors } from '@/hooks/useColors';
11
12 interface SkeletonLoaderProps {
13   width?: number | string;
14   height?: number;
15   borderRadius?: number;
16   style?: any;
17 }
18
19 export function SkeletonLoader({ width = '100%', height = 20, borderRadius = 8, style }:
20   SkeletonLoaderProps, colors) {
21   const opacity = useSharedValue(0.3);
22
23   useEffect(() => {
24     opacity.value = withRepeat(
25       withTiming(0.7, { duration: 800, easing: Easing.inOut(Easing.ease) }),
26       -1,
27       true
28     );
29   }, []);
30
31   const animatedStyle = useAnimatedStyle(() => ({
32     opacity: opacity.value,
33   }));
34
35   return (
36     <Animated.View
37       style={[
38         { width, height, borderRadius, backgroundColor: colors.muted },
39         animatedStyle,
40         style,
41       ]}
42     />
43   );
44 }
45
```

artifacts/mobile/components/ErrorBoundary.tsx

```
1  import React, { Component, ComponentType, PropsWithChildren } from 'react';
2  import { ErrorFallback, ErrorFallbackProps } from '@components/ErrorFallback';
3
4  export type ErrorBoundaryProps = PropsWithChildren<{
```


artifacts/mobile/components/ErrorBoundary.tsx (continued)

```
5   FallbackComponent?: ComponentType<ErrorFallbackProps>;
6   onError?: (error: Error, stackTrace: string) => void;
7 }>;
8
9 type ErrorBoundaryState = { error: Error | null };
10
11 /**
12  * This is a special case for for using the class components. Error boundaries must be class components
13  * https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary
14  */
15 export class ErrorBoundary extends Component<
16   ErrorBoundaryProps,
17   ErrorBoundaryState
18 > {
19   state: ErrorBoundaryState = { error: null };
20
21   static defaultProps: {
22     FallbackComponent: ComponentType<ErrorFallbackProps>;
23   } = {
24     FallbackComponent: ErrorFallback,
25   };
26
27   static getDerivedStateFromError(error: Error): ErrorBoundaryState {
28     return { error };
29   }
30
31   componentDidCatch(error: Error, info: { componentStack: string }): void {
32     if (typeof this.props.onError === 'function') {
33       this.props.onError(error, info.componentStack);
34     }
35   }
36
37   resetError = (): void => {
38     this.setState({ error: null });
39   };
40
41   render() {
42     const { FallbackComponent } = this.props;
43
44     return this.state.error && FallbackComponent ? (
45       <FallbackComponent
46         error={this.state.error}
47         resetError={this.resetError}
48       />
49     ) : (
50       this.props.children
51     );
52   }
53 }
54
```

artifacts/mobile/components/ErrorFallback.tsx

```
1 import React, { useState } from 'react';
2 import {
3   Modal,
4   Platform,
5   Pressable,
6   ScrollView,
```

```

7   StyleSheet,
8   Text,
9   View,
10  } from 'react-native';
11  import { useSafeAreaInsets } from 'react-native-safe-area-context';
12  import { useColors } from '@hooks/useColors';
13  import { Feather } from '@expo/vector-icons';
14  import { reloadAppAsync } from 'expo';
15
16  export type ErrorFallbackProps = {
17    error: Error;
18    resetError: () => void;
19  };
20
21  export function ErrorFallback({ error, resetError }: ErrorFallbackProps) {
22    const colors = useColors();
23    const insets = useSafeAreaInsets();
24
25    const [isModalVisible, setIsModalVisible] = useState(false);
26
27    const handleRestart = async () => {
28      try {
29        await reloadAppAsync();
30      } catch (restartError) {
31        console.error('Failed to restart app:', restartError);
32        resetError();
33      }
34    };
35
36    const formatErrorDetails = (): string => {
37      let details = `Error: ${error.message}\n\n`;
38      if (error.stack) {
39        details += `Stack Trace:\n${error.stack}`;
40      }
41      return details;
42    };
43
44    const monoFont = Platform.select({
45      ios: 'Menlo',
46      android: 'monospace',
47      default: 'monospace',
48    });
49
50    return (
51      <View style={[styles.container, { backgroundColor: colors.background }]}>
52        {__DEV__ ? (
53          <Pressable
54            onPress={() => setIsModalVisible(true)}
55            accessibilityLabel="View error details"
56            accessibilityRole="button"
57            style={({ pressed }) => [
58              styles.topButton,
59              {
60                top: insets.top + 16,
61                backgroundColor: colors.card,
62                opacity: pressed ? 0.8 : 1,
63              },
64            ]}
65          >

```

```

66         <Feather name="alert-circle" size={20} color={colors.foreground} />
67     </Pressable>
68     ) : null}
69
70     <View style={styles.content}>
71         <Text style={[styles.title, { color: colors.foreground }]}>
72             Something went wrong
73         </Text>
74
75         <Text style={[styles.message, { color: colors.mutedForeground }]}>
76             Please reload the app to continue.
77         </Text>
78
79         <Pressable
80             onPress={handleRestart}
81             style={({ pressed }) => [
82                 styles.button,
83                 {
84                     backgroundColor: colors.primary,
85                     opacity: pressed ? 0.9 : 1,
86                     transform: [{ scale: pressed ? 0.98 : 1 }],
87                 },
88             ]}
89         >
90             <Text
91                 style={[styles.buttonText, { color: colors.primaryForeground }]}
92             >
93                 Try Again
94             </Text>
95         </Pressable>
96     </View>
97
98     {__DEV__ ? (
99         <Modal
100             visible={isModalVisible}
101             animationType="slide"
102             transparent={true}
103             onRequestClose={() => setIsModalVisible(false)}
104         >
105             <View style={styles.modalOverlay}>
106                 <View
107                     style={[
108                         styles.modalContainer,
109                         { backgroundColor: colors.background },
110                     ]}
111                 >
112                     <View
113                         style={[
114                             styles.modalHeader,
115                             { borderBottomColor: colors.border },
116                         ]}
117                     >
118                         <Text style={[styles.modalTitle, { color: colors.foreground }]}>
119                             Error Details
120                         </Text>
121                     <Pressable
122                         onPress={() => setIsModalVisible(false)}
123                         accessibilityLabel="Close error details"
124                         accessibilityRole="button"

```

```

125         style={{ { pressed } } => [
126             styles.closeButton,
127             { opacity: pressed ? 0.6 : 1 },
128         ]}
129     >
130     <Feather name="x" size={24} color={colors.foreground} />
131 </Pressable>
132 </View>
133
134 <ScrollView
135     style={styles.modalScrollView}
136     contentContainerStyle={[
137         styles.modalScrollContent,
138         { paddingBottom: insets.bottom + 16 },
139     ]}
140     showsVerticalScrollIndicator
141 >
142     <View
143         style={[
144             styles.errorContainer,
145             { backgroundColor: colors.card },
146         ]}
147     >
148         <Text
149             style={[
150                 styles.errorText,
151                 {
152                     color: colors.foreground,
153                     fontFamily: monoFont,
154                 },
155             ]}
156             selectable
157         >
158             {formatErrorDetails()}
159         </Text>
160     </View>
161 </ScrollView>
162 </View>
163 </View>
164 </Modal>
165 ) : null}
166 </View>
167 );
168 }
169
170 const styles = StyleSheet.create({
171     container: {
172         flex: 1,
173         width: '100%',
174         height: '100%',
175         justifyContent: 'center',
176         alignItems: 'center',
177         padding: 24,
178     },
179     content: {
180         alignItems: 'center',
181         justifyContent: 'center',
182         gap: 16,
183         width: '100%',

```

```
184     maxWidth: 600,
185   },
186   title: {
187     fontSize: 28,
188     fontWeight: '700',
189     textAlign: 'center',
190     lineHeight: 40,
191   },
192   message: {
193     fontSize: 16,
194     textAlign: 'center',
195     lineHeight: 24,
196   },
197   topButton: {
198     position: 'absolute',
199     right: 16,
200     width: 44,
201     height: 44,
202     borderRadius: 8,
203     flexDirection: 'row',
204     alignItems: 'center',
205     justifyContent: 'center',
206     zIndex: 10,
207   },
208   button: {
209     paddingVertical: 16,
210     borderRadius: 8,
211     paddingHorizontal: 24,
212     minWidth: 200,
213     shadowColor: '#000',
214     shadowOffset: {
215       width: 0,
216       height: 2,
217     },
218     shadowOpacity: 0.1,
219     shadowRadius: 4,
220     elevation: 3,
221   },
222   buttonText: {
223     fontWeight: '600',
224     textAlign: 'center',
225     fontSize: 16,
226   },
227   modalOverlay: {
228     flex: 1,
229     backgroundColor: 'rgba(0, 0, 0, 0.5)',
230     justifyContent: 'flex-end',
231   },
232   modalContainer: {
233     width: '100%',
234     height: '90%',
235     borderTopLeftRadius: 16,
236     borderTopRightRadius: 16,
237   },
238   modalHeader: {
239     flexDirection: 'row',
240     justifyContent: 'space-between',
241     alignItems: 'center',
242     paddingHorizontal: 16,
```

```
243     paddingTop: 16,  
244     paddingBottom: 12,  
245     borderBottomWidth: 1,  
246   },  
247   modalTitle: {  
248     fontSize: 20,  
249     fontWeight: '600',  
250   },  
251   closeButton: {  
252     width: 44,  
253     height: 44,  
254     alignItems: 'center',  
255     justifyContent: 'center',  
256   },  
257   modalScrollView: {  
258     flex: 1,  
259   },  
260   modalScrollContent: {  
261     padding: 16,  
262   },  
263   errorContainer: {  
264     width: '100%',  
265     borderRadius: 8,  
266     overflow: 'hidden',  
267     padding: 16,  
268   },  
269   errorText: {  
270     fontSize: 12,  
271     lineHeight: 18,  
272     width: '100%',  
273   },  
274   });  
275
```

Mobile App — Constants & Hooks

artifacts/mobile/constants/colors.ts

```
1  /**
2   * Semantic design tokens for the mobile app.
3   */
4
5  const colors = {
6    light: {
7      text: '#17211B',
8      tint: '#178B57',
9      background: '#F7FAF8',
10     foreground: '#17211B',
11     card: 'FFFFFF',
12     cardForeground: '#17211B',
13     primary: '#178B57',
14     primaryForeground: 'FFFFFF',
15     secondary: '#E8F6EF',
16     secondaryForeground: '#0E5F3B',
17     muted: '#E8F6EF',
18     mutedForeground: '#657069',
19     accent: '#E8F6EF',
20     accentForeground: '#178B57',
21     destructive: '#C94B4B',
22     destructiveForeground: 'FFFFFF',
23     border: '#DDE7E1',
24     input: '#DDE7E1',
25     success: '#2E9D63',
26     warning: '#D99A25',
27     darkGreen: '#0E5F3B',
28     lightGreen: '#E8F6EF',
29   },
30   radius: 12,
31 };
32
33 export default colors;
34
```

artifacts/mobile/hooks/useColors.ts

```
1  import { useColorScheme } from 'react-native';
2  import colors from '@constants/colors';
3
4  /**
5   * Returns the design tokens for the current color scheme.
6   *
7   * The returned object contains all color tokens for the active palette
8   * plus scheme-independent values like `radius`.
9   *
10   * Falls back to the light palette when no dark key is defined in
11   * constants/colors.ts (the scaffold ships light-only by default).
12   * When a sibling web artifact's dark tokens are synced into a `dark`
13   * key, this hook will automatically switch palettes based on the
14   * device's appearance setting.
15   */
```

artifacts/mobile/hooks/useColors.ts (continued)

```
16 export function useColors() {  
17   const scheme = useColorScheme();  
18   const palette =  
19     scheme === 'dark' && 'dark' in colors  
20       ? (colors as Record<string, typeof colors.light>).dark  
21       : colors.light;  
22   return { ...palette, radius: colors.radius };  
23 }  
24
```


